GEMILANG SATRIA PERKASA
PT Gemilang Satria Perkasa · Divisi Engineering

STANDARD · TECHNICAL DOCUMENTATION

DEMS Technical Documentation

Digital Evidence Management System: architecture, development and operations

Commit `59a683ed` is on `main`. It has the same content as branch `audit/fix-20261105`, which was merged into `main` at the end of the 2026-11-05/06 audit.

NOMOR DOKUMEN	001/GSP/STD/XI/2026	REVISI	H · 06-XI-2026
REPOSITORI	commit 59a683ed · main	BAHASA	English
KLASIFIKASI	BIASA	HALAMAN	127
REGISTER	Register Induk GSP	JENIS	STD · Standar / SOP

REVISION HISTORY

Revision History

REV	DATE	SECTIONS AFFECTED	WHAT CHANGED	TRIGGER	IMPACT	BY
H	06-XI-2026	Whole document	First registered issue; English and Indonesian editions; set in the GSP DATUM house style	User request, 2026-11-06	Number 001/GSP/STD/XI/2026 assigned in the Register Induk; technical content as in the source chapters	DEMS documentation task, row A1
A–G	–	–	Earlier issues by the user, outside this register	–	–	User

SUMBER Register Induk GSP, 001/GSP/STD/XI/2026 **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

CHANGE NOTE

RevH · 06-XI-2026 · 001/GSP/STD/XI/2026 · whole document

What	First registered issue of the manual, English and Indonesian editions, in the GSP DATUM house style: IBM Plex embedded, register block, ruled asides, DATUM tables, three figures redrawn with datum_diagram.py.
Why	User request 2026-11-06, "rebuild dengan gsp standart dokumen, ini sudah rev H". Rev A-G were used earlier by the user, outside this register.
Impact	Technical content unchanged against the source chapters at commit 59a683ed; no table, figure or number moved. The cover version line "1.0" is replaced by Rev H; two front-matter headings were reworded for lint check 04.
By	DEMS documentation task, row A1 · 2026-11-06

CONTENTS

Contents

Reading this document 9
The root README and this manual 9
Sources of this manual 10
Conventions 11
Text 11
Operations and decisions 11
Verdict words 11
Callouts, figures and tables 12
Sanitization 12
1 Introduction and architecture 13
1.1 What DEMS is 13
1.1.1 Where it came from 13
1.2 Operating modes 14
1.3 High-level architecture 14
1.4 End-to-end data flow 16
2 Repository tour 19
2.1 Top-level layout 19
2.1.1 npm workspaces 19
2.1.2 The two packages 20
2.2 deploy/ 20
2.3 docs/ 21
2.4 READMEs everywhere 21
2.5 scripts/ 22
3 Core server 23
3.1 Overview 23
3.2 Source layout 24
3.3 Request pipeline 25
3.4 Modules and operations 26
3.4.1 API reference 28
3.5 Capture pipeline 28
3.5.1 The capture operation 28
3.5.2 Tracks and channels 28
3.5.3 Session start 28
3.5.4 Cameras 28
3.5.5 Clocks 29

3.5.6 Stream loss 29
3.5.7 Pre-check 29
3.5.8 Live preview and microphone meters 29
3.5.9 "Not reported" 29
3.6 Camera and microphone pairs (audit D5) 30
3.7 Boot checks 30
3.7.1 Database checks 31
3.7.2 Capture refusals 31
3.7.3 Production key and operator checks 32
3.7.4 Platform and host checks 33
3.7.5 A healthy production start 33
3.8 Database and migrations 33
3.8.1 Server, roles and databases 33
3.8.2 Connection files 34
3.8.3 Migrations 34
3.8.4 Database commands 35
3.9 Sealing, verify and export 36
3.9.1 The sealing pipeline 36
3.9.2 The package 37
3.9.3 Verify 37
3.9.4 Two keys, opposite risks 38
3.9.5 Export 39
3.10 Pairing and realtime 39
3.10.1 Pairing a tablet 39
3.10.2 The realtime socket 39
3.11 LCD bridge 40
3.12 Transcription in the Core 40
3.13 Configuration summary 41
4 Web console 42
4.1 What it is 42
4.2 Data modes and the proxy 42
4.2.1 How LIVE reaches the Core 43
4.2.2 Which mode wins 43
4.3 Routes 45
4.4 Features 46
4.4.1 Session flow 46
4.4.2 Devices & Inputs 46
4.4.3 Evidence 46

4.4.4 Paired Devices 47
4.4.5 Administration and the demo panel 47
4.5 Live previews 47
4.6 Production lock 47
4.7 Sign-in behaviour 48
4.8 Commands and environment 48
5 Tablet app 50
5.1 Runtime 50
5.1.1 Data sources 50
5.1.2 The runtime folder 50
5.1.3 Folder layout 51
5.2 Core server address 51
5.2.1 Validation 51
5.2.2 What a change does 52
5.3 Pairing 53
5.4 Features 53
5.5 Debug vs release APK 54
5.5.1 Building the debug APK 54
5.5.2 Building the release APK 54
5.5.3 Library patch 55
5.5.4 Cleartext to the Core 55
5.6 Installing on a real tablet 56
5.7 Commands and toolchain 57
6 Capture worker, LCD backend, transcriber and mock Core 58
6.1 Capture worker 58
6.1.1 What it is 58
6.1.2 How it talks 58
6.1.3 Source layout and make targets 59
6.1.4 Configuration 60
6.1.5 Build stamp 60
6.1.6 The e2e series 61
6.1.7 The capture relay 62
6.1.8 The capture guard 62
6.1.9 Linux status 62
6.1.10 Production notes 63
6.2 LCD backend 63
6.2.1 What it is 63
6.2.2 Routes 63

6.2.3 How the Core drives it ... 64
6.2.4 Channel behaviour ... 65
6.2.5 Unit and operations ... 66
6.2.6 Tests ... 66
6.2.7 Configuration ... 66
6.3 Transcriber ... 67
6.3.1 Configuration ... 67
6.3.2 Commands ... 68
6.4 Mock Core ... 68
7 Development workflow ... 70
7.1 Requirements ... 70
7.2 Setup ... 70
7.2.1 The pg:* scripts ... 71
7.2.2 Database commands ... 71
7.2.3 Lanes ... 71
7.2.4 Development capture (lane 2) ... 72
7.3 Typecheck, lint and tests ... 72
7.3.1 What npm test does not run ... 73
7.3.2 Test counts ... 73
7.4 The DB test window ... 73
7.5 Branches ... 74
8 Deploy and operations ... 76
8.1 Bench host layout ... 76
8.1.1 dems-core.service ... 77
8.1.2 dems-console.service ... 78
8.1.3 dems-lcd.service ... 78
8.1.4 The nft loopback rule ... 78
8.1.5 What's not in the repository ... 78
8.2 Prerequisites ... 79
8.3 Runbook: fresh install ... 79
8.3.1 Roles, database and connection files ... 80
8.3.2 Seed and operator passwords ... 81
8.3.3 Production signing key ... 81
8.3.4 Units and the nft rule ... 82
8.3.5 Keep the secrets private ... 82
8.4 Runbook: core.env from the template ... 82
8.5 Runbook: migrations ... 84
8.5.1 On a fresh install ... 84

8.5.2 In a deploy 84
8.5.3 Rules 85
8.6 Runbook: DB test window 85
8.7 Runbook: lockstep deploy 86
8.8 Runbook: smoke checks 89
8.9 Runbook: rollback 89
8.10 Docker 90
8.11 Ops facts 90
9 Security notes 91
9.1 Network exposure 91
9.2 Request security 91
9.3 Device credentials and tokens 92
9.4 Evidence at rest 93
9.5 Host hardening 94
9.6 Secrets handling 95
9.7 Demo-era leftovers 95
9.8 Open security items 95
10 Design decisions and history 97
10.1 How to read this chapter 97
10.2 Timeline 98
10.3 Decision records 98
10.3.1 ADR-1: Camera N to Mic N pairing 98
10.3.2 ADR-2: Evidence manifest /2.1, and quarantine instead of deletion 99
10.3.3 ADR-3: Live previews on every surface 100
10.3.4 ADR-4: One mic status reader, and mic reach stated by the Core 101
10.3.5 ADR-5: Production console lock and "Not reported" 102
10.3.6 ADR-6: Worker version, build stamp and the lockstep rule 102
10.3.7 ADR-7: Mission order swapped 103
10.3.8 ADR-8: Fixes first (the W4 wave) 103
10.3.9 ADR-9: Release builds are LIVE-only 104
10.3.10 ADR-10: Configurable Core address, with LAN cleartext 104
10.3.11 ADR-11: The D5 proof limit 105
10.3.12 ADR-12: Pairing and credential binding 106
10.3.13 ADR-13: Biometric sign-in off in release 106
10.3.14 ADR-14: Public repository for the team audit 107

10.3.15 ADR-15: Environment published as a template, not as secrets 107
10.3.16 ADR-16: How changes reach production 108
10.3.17 ADR-17: All missions stopped 108
11 Audit changelog 2026-11-05/06 110
11.1 Missions 110
11.2 Method 110
11.3 The first fix: a stale worker 110
11.4 Waves W1-W3 110
11.5 Wave W4 111
11.6 Mission 3: tablet rows 111
11.7 Deploys 112
11.8 Results 113
11.9 Commit chain 113
11.10 APKs 114
11.11 Publication 114
11.12 Deviations and lapses 115
12 Known issues and roadmap 116
12.1 Open items 116
12.2 Blocked items, by kind 117
12.3 Long-standing product limits worth knowing 117
12.4 Roadmap 118
12.5 Later hygiene 118
13 Appendix 119
13.1 Ports 119
13.2 Environment variables 119
13.3 systemd units 122
13.4 npm scripts 123
13.5 Error and log code index 124
13.6 Glossary 125

FRONT MATTER

Reading this document

This manual is for engineers joining the DEMS team, for the people auditing the code, and for whoever installs the bench or production host. It assumes you can read TypeScript, run `npm` and `systemctl`, and find your way around a Linux box. You don't need to have read the old README.

The chapters fall into three groups:

CHAPTERS	WHAT THEY COVER	TIME FRAME
1-9	The system as it stands at commit `59a683ed`: architecture, repository, Core server, web console, tablet app, the smaller services, the development workflow, deploy and operations, security notes	Current state only
10	Why the system looks the way it does: the design decisions that changed the original design, with the request behind each one	History, 2026-10-23 to 2026-11-06
11-12	The audit record of 2026-11-05/06 and what is still open	Audit record and open items
13	Appendix: ports, environment variables, units, scripts, codes, glossary	Reference

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

If you're new, read chapters 1 and 2 first, then the chapter for the part you'll work on. If you're installing a host, read chapter 1, then chapter 8, and keep chapter 13 open.

WARNING Know where the system stands before you rely on it.

- The bench host runs the Core in `REAL_PRODUCTION`: real cameras, real microphones, the production signing key.
- The camera/microphone pairing of audit decision D5 (camera N with microphone N) is built from contract to evidence, but **no real recording with four cameras and four microphones has proven it yet**. The first real 4+4 session is a mandatory acceptance test before operational use (audit D153).
- The full end-of-mission verification gate that the audit planned never ran. Each fix wave was gated with targeted tests only.

The root README and this manual

The repository's root `README.md` runs to more than 1,300 lines. Only its first block, **Overview** (added 2026-11-06), describes the current repository. Everything after it, from the section **Ringkasan** on, is the history of earlier tasks (the first demo, Live mode, real capture, task 4, the audit), kept as written with dated corrections.

This manual replaces the root README for orientation. It does **not** replace the per-app READMEs (`apps/<app>/README.md`) or the folder READMEs under `apps/*/src/`. Those stay the place for file-level detail, and this manual points to them.

Sources of this manual

The facts in this manual come from the repository at `59a683ed`: the Overview block of the root README, the per-app and folder READMEs, `deploy/bench-config/`, and `docs/AUDIT_REPORT_2026-11-06.md`. Where a README left a fact open, the code was read, and the text names the file. Chapter 10 also draws on the audit's decision log, which is kept outside the repository; chapter 10 says so where it matters. Nothing was measured or run for this manual. Where the sources disagree or are silent, you'll find an **Unknown:** marker instead of a guess.

FRONT MATTER

Conventions

Text

CONVENTION	MEANING
`monospace`	Anything you type or read literally: commands, file paths, ports, environment variables, codes, operation names, commit hashes
`<placeholder>`	A value you fill in, for example `<data directory>` or `<sessionId>`
`<repo root>`	The root of a clone of the repository. On the bench host the deployed tree is `/opt/dems/core`
Bold	A term on first use; the glossary in chapter 13 defines it
"Not reported"	A literal UI word of the product, quoted as it appears

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

Paths inside the repository are written from the repository root, for example `apps/core-server/src/config.ts`. A line reference is written `file:line`.

`AUDIT §n` points to section n of the public audit report `docs/AUDIT_REPORT_2026-11-06.md` in the repo.

Operations and decisions

- **Operation numbers** ("op 41", "operation 81") are the numbers of the shared operation registry in `packages/shared`. Each Core module README names the numbers it serves.
- **Decision numbers.** Two decision logs use the same `Dnn` numbers for different things:
  - the team's **bench log** (kept on the bench host), for example bench D60, which made the Core record four video tracks;
  - the **audit log** of 2026-11-05/06 (D1 to D283), for example audit D5, the camera/microphone pairs.

  Wherever the series isn't obvious, this manual writes "bench D60" or "audit D60". A bare `Dnn` inside a quoted file name or code comment is left as the source wrote it. Neither log is in the repository. Chapter 10 explains what is available instead.
- **Residuals** such as `R263-TLS` or `R225-F1` are open risks the audit recorded. Chapter 12 lists them.

Verdict words

The sources are careful about how something is known, and this manual keeps the same distinction:

PHRASE	MEANING
proven by tests	Automated tests show it, on a development machine or the bench, often with test doubles
proven on hardware	Shown on the bench host with real cameras, microphones or panel
not proven	Built, but no test or hardware run has shown it yet

PHRASE	MEANING
Unknown:	The sources of this manual don't settle the fact. The text says what is missing and doesn't guess

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Callouts, figures and tables

WARNING Data loss, a security exposure or production downtime.

HEADS-UP Something that will trip you up if you don't know it.

TIP A shortcut or a better way to do something.

Figures are box-and-arrow drawings, numbered per chapter (Figure 3.2 is the second figure of chapter 3). Tables are numbered the same way, with the caption above the table.

Sanitization

The repository is public, and so is this manual. It contains no network addresses except `127.0.0.1`, `0.0.0.0` and the placeholder `192.168.x.x` (subnets as `192.168.x.0/24`), no passwords, PINs or pairing codes, no e-mail addresses, no account names and no home-folder paths. Where a source held such a value, the manual describes it instead of quoting it.

CHAPTER 1

1 Introduction and architecture

1.1 What DEMS is

DEMS is a Digital Evidence Management System for interview-room recording. A **Core server** runs on the interview-room host and does four jobs (root `README.md`, Overview):

- it records up to four cameras, each with its paired microphone;
- it seals each recording into a signed **evidence package**;
- it keeps an audit trail;
- it serves the operators, through a web console and a tablet app.

The system has six parts:

TABLE 1.1 Parts of DEMS

PART	WHERE	WHAT IT IS
Core server	`apps/core-server`	Fastify/TypeScript server. Owns the PostgreSQL database, operators, roles and permissions, tablet pairing, capture supervision, sealing, export, transcription and the audit trail
Web console	`apps/core-web`	React + Vite browser app for operators and administrators
LCD front-panel service	`apps/lcd-backend`	Small Python service that drives the ESP32 status panel from the Core's state
Capture worker	`apps/capture-worker`	C program (plus a relay and a guard) that the Core starts to record cameras and microphones
Tablet app	`apps/mobile`	The operator's Android app, paired with one Core
Shared contract	`packages/shared`, `packages/capture-protocol`	Types, operations, UI words and rules shared by the Core and both clients; the Core-to-worker message contract

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

One rule shapes everything else: **the Core holds the recording truth**. The console, the tablet and the LCD panel only display it (`apps/core-server/README.md`; `apps/lcd-backend/README.md`). The LCD service even has an HTTP `rec=start` endpoint, and it does not start a DEMS recording. If you're ever unsure where a piece of state lives, the answer is the Core.

1.1.1 Where it came from

DEMS didn't start as an evidence system. It started as a synthetic demo where both clients ran a built-in simulator (**Simulated** mode). A real backend followed on 2026-10-06 (**Live** mode: the Core server with PostgreSQL, roles and pairing). Task 3 added real capture and sealing; task 4 added the production signing key, real ONVIF cameras and host commissioning; the audit of 2026-11-05/06 fixed the team's updates and built the camera/microphone pairs. Chapter 10 tells that story decision by decision.

HEADS-UP That history explains odd names you'll meet: the root package is still called `dems-demo`, the database of the Core on port 4420 is `dems_demo`, and the tablet's package id is `com.dems.demo.tablet`. They're names, not a statement about what the bench runs.

1.2 Operating modes

There are two different "mode" ideas in DEMS. Keep them apart.

Data mode belongs to a client (console or tablet). It says where the client gets its data from (`apps/core-web/README.md`):

TABLE 1.2 Data modes of the clients

DATA MODE	SOURCE OF TRUTH	USED FOR
`SIMULATED`	A simulator inside the client (`@dems/shared`)	Demos and development
`LIVE`	The Core server	Real use; the only mode on the bench

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Capture operation belongs to the Core. It says what the Core records. The Core decides it once, at start, from three environment variables (`apps/core-server/src/capture/fence/operation.ts`):

TABLE 1.3 Capture operations of the Core

CAPTURE OPERATION	SET BY	WHAT IT RECORDS	DEMO CONTROLS
`SIMULATED` (default)	`DEMS_CAPTURE_MODE` unset or `simulated`	Nothing. No capture worker starts; sessions and evidence come from the simulator	Only here, with `DEMS_DEMO_CONTROLS=1`
`REAL_DEVELOPMENT`	`DEMS_CAPTURE_MODE=real` and `DEMS_CAPTURE_DEV_SOURCES=1`	Development sources only: a generated test camera and microphone, or the host's own camera and microphone	Never
`REAL_PRODUCTION`	`DEMS_CAPTURE_MODE=real`, `DEMS_CAPTURE_DEV_SOURCES` unset	Production sources only, such as cameras added through the camera search and connect flow	Never

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Two rules hold across all three: demo controls (scenarios, fault injection, reset from the console) exist only in `SIMULATED`, and a single session never mixes development and production sources. Bad combinations, such as `real` together with `DEMS_DEMO_CONTROLS=1`, stop the boot; section 3.7 lists the refusal codes.

The bench host runs `REAL_PRODUCTION`. On such a Core, anything that has no real data source shows **"Not reported"** instead of a simulated value (audit D25).

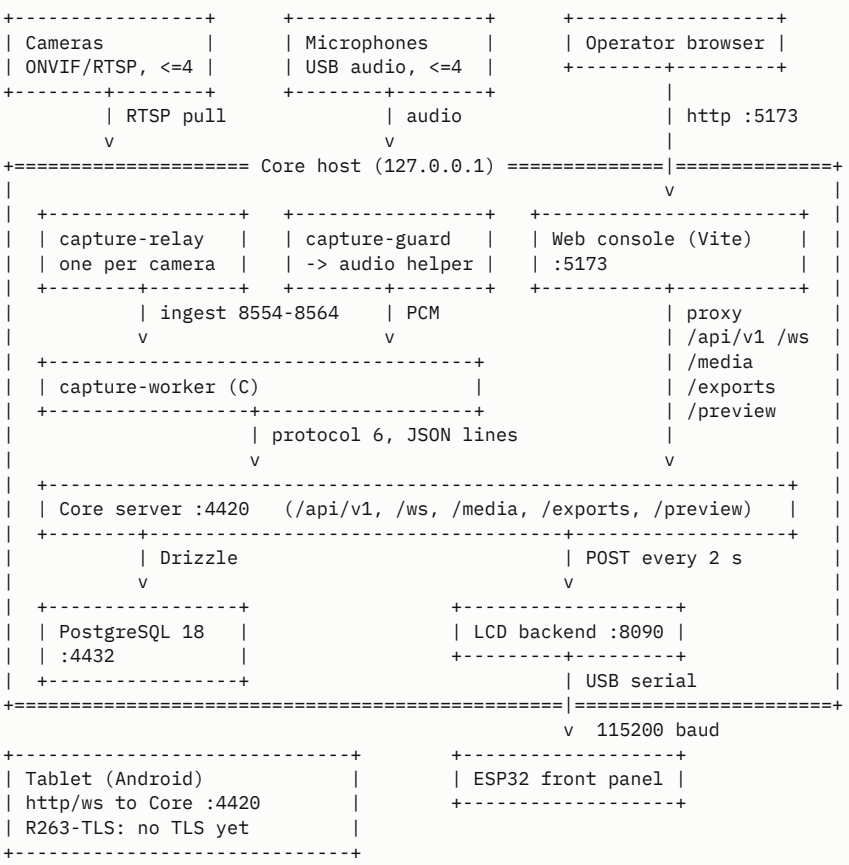
The clients follow suit on the bench. The console runs with the **production lock** (`VITE_DEMS_PRODUCTION_LOCK=1`), which keeps it in `LIVE` and hides the Simulated choice on the login page (audit D47). Release builds of the tablet app carry the Live source only (audit D163). The Simulated paths still exist for development and tests, which is why you'll keep running into them in the code.

HEADS-UP Simulated data and Live data are separate worlds. Switching a client between them signs you out, and nothing is copied across.

1.3 High-level architecture

Figure 1.1 shows every process and every link of a running installation.

FIGURE 1.1 System context. Every listener on the Core host binds 127.0.0.1.



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Walk through the arrows:

- Camera to worker: for each camera, the Core starts one `capture-relay`. The relay dials the camera over RTSP and pushes the stream into the worker's loopback ingest, a TCP port in `127.0.0.1:8554-8564`. The relay never listens; the worker never dials out (`apps/capture-worker/README.md`). On the bench an nftables rule lets only the Core's service user reach the ingest ports.
- Microphone to worker: the Core starts the audio helper through `capture-guard`. The helper delivers PCM audio to the worker. The guard exists to make sure the helper dies when its parent does.
- Worker and Core: the worker is a child process of the Core. They exchange one JSON object per line on the worker's stdin and stdout: commands in, frames out (`HELLO`, `HEARTBEAT`, `ACK`, `WARNING`, `RECORD`, `CRASH`, `TELEMETRY`). This is **capture protocol 6** (`CAPTURE_PROTOCOL_VERSION = 6` in `packages/capture-protocol`). A Core and a worker with different protocol numbers refuse each other at `HELLO`.
- Core and database: PostgreSQL 18 on `127.0.0.1:4432`, TCP only, through Drizzle. The Core refuses port 5432.
- Browser to Core: the operator opens the console at `http://127.0.0.1:5173`. The Vite server proxies `/api/v1`, `/ws`, `/media`, `/exports` and `/preview` to the Core on port 4420, so the browser sees one origin and the session cookie stays same-origin.

- Tablet to Core: the tablet uses the same `/api/v1` API and `/ws` socket over plain `http` and `ws`. Traffic between tablet and Core is not encrypted; TLS is the open residual `R263-TLS`.
- Core to panel: every 2 s the Core posts each channel's state to the LCD backend (`/api/dems/ch/<n>/status` on port 8090) with a 6 s lease. The backend talks to the ESP32 panel over USB serial at 115200 baud, one JSON line per message.

HEADS-UP One link is not documented. **Unknown:** how a tablet on the LAN reaches the Core. The Core's listen host is the constant `127.0.0.1` (`LISTEN_HOST` in `apps/core-server/src/config.ts`, no variable changes it), yet the tablet's setup tells you to enter the Core host's LAN address (`192.168.x.x`) and port 4420. No forwarder, proxy or bind change is documented in the repository, and the audit records LAN `http`/`ws` from a real tablet as not proven. Don't assume a mechanism; check the host you're working on.

TABLE 1.4 Processes and ports on a Core host

PROCESS	PORT (127.0.0.1)	STARTED BY	BENCH UNIT
Core server (Node)	4420	systemd	`dems-core`
PostgreSQL 18	4432	systemd	`postgresql@18-main`
Web console (Vite server)	5173	systemd	`dems-console`
LCD backend (Python)	8090	systemd	`dems-lcd`
capture-worker	8554-8564 (ingest)	the Core, per session	none
capture-relay, one per camera	none (never listens)	the Core	none
capture-guard and audio helper	none	the Core	none

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

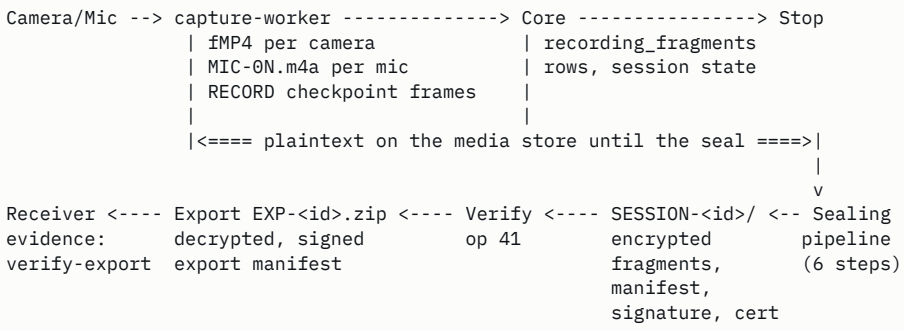
On a development machine you start the same processes by hand (`npm run pg:start`, `npm run start:server`, `npm run dev:web`), and lanes add Core instances on 4421-4423 with consoles on 5176-5178. Chapter 7 covers that setup, chapter 8 the bench units.

The transcriber doesn't appear in the figure because it isn't a process: `apps/transcriber` is a library that the Core imports in-process. The mock Core (`apps/mock-core`, port 4400) is also absent; it belongs to the first demo's Simulated mode only.

1.4 End-to-end data flow

Figure 1.2 follows one recording from the camera to a receiver who checks an export.

FIGURE 1.2 End-to-end data flow of one recording



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Step by step:

1. **Start.** The UI shows "Connecting cameras..." and switches to recording only after the first **durable fragment** is on disk. A start with no camera ends as a failed start with no evidence item.
2. **Capture.** The worker writes one fragmented MP4 (**fMP4**) file per camera, up to four (`VIDEO-01` to `VIDEO-04`). Each camera file carries the audio of its paired microphone. Each microphone is also written to its own file `MIC-0N.m4a`, stored as track `AUDIO-0N` (`apps/capture-worker/README.md`). As fragments become durable, the worker sends `RECORD` checkpoint frames, and the Core writes one `recording_fragments` row per fragment per camera, in order, without a gap.
3. **At rest before the seal.** The recording lies in plaintext on the Core's media store until the seal completes. Only file permissions protect it in that window. If the Core or the machine stops mid-session, the recording stays plaintext until the Core starts again and recovery seals it, up to the interruption.
4. **Sealing.** After stop, the sealing pipeline runs its six steps (`apps/core-server/src/sealing/pipeline/contract.ts`). It encrypts each fragment as one AES-256-GCM message, builds and signs the manifest. Packages with the camera/microphone pairs use manifest schema `dems.evidence-manifest/2.1`.
5. **The package.** The sealed package is the folder `SESSION-<sessionId>/` under the media store root. It holds `session-manifest.json`, `signature.sig`, `device-certificate.pem`, the encrypted camera files `fragments/VIDEO-0N.mp4` with `fragments/fragments.json`, the audio files that /2.1 adds, `audit-session-extract.jsonl`, `transcript/` when a transcript was sealed, and `export/` once it was exported.
6. **Verify.** The Core verifies a package on request (op 41). At task 3 verify ran 15 checks and named an altered fragment exactly.
7. **Export.** An export writes a decrypted archive `EXP-<id>.zip` with its own signed export manifest. In the web console the download link is signed, bound to the operator's sign-in, and lives 60 s.
8. **Receiver.** Whoever receives an export can check it without a Core:

```
npm run evidence:verify-export -- <archive.zip>
```

HEADS-UP The audit extract inside a package, `audit-session-extract.jsonl`, must be present but is **not** covered by the package signature. Verify reports this on every package as the notice `AUDIT_EXTRACT_NOT_SIGNED`. Anything recorded only in that extract (for example microphone fault rows) is unsigned.

Chapter 3 goes into each of these stages from the Core's side. Chapter 6 covers the worker.

CHAPTER 2

2 Repository tour

This chapter is the map. It tells you what each top-level folder is for and where to look first. File-level detail lives in the READMEs next to the code (section 2.4).

2.1 Top-level layout

TABLE 2.1 Top-level folders at commit 59a683ed

FOLDER	WHAT IT HOLDS
`apps/core-server`	The Core server: Fastify on port 4420, PostgreSQL storage with Drizzle migrations, roles and permissions, device pairing, capture supervision, sealing, export, and the host commands for keys and operator passwords
`apps/core-web`	The web console: React + Vite
`apps/lcd-backend`	The LCD front-panel service: Python HTTP service on port 8090 that writes to the ESP32 panel over USB serial
`apps/capture-worker`	The capture worker, capture relay and capture guard: C programs against FFmpeg 9.0.2, built with `make`
`apps/mobile`	The tablet app: Expo / React Native Android app `com.dems.demo.tablet`
`apps/mock-core`	The first demo's mock Core: health check and WebSocket heartbeat on `127.0.0.1:4400`, used by Simulated mode only
`apps/transcriber`	The speech-to-text pipeline, used by the Core as a library
`packages/shared`	The shared contract (`@dems/shared`): API client and constants, operations, i18n words, simulator, theme tokens, OpenAPI generator
`packages/capture-protocol`	The message contract between the Core and the capture worker (`@dems/capture-protocol`, protocol 6)
`scripts/`	Project-local PostgreSQL, test runners, host helpers, capture setup, release packaging (section 2.5)
`deploy/`	Bench host configuration and the Docker installer sources (section 2.2)
`docs/`	Task records of the earlier tasks, plus the public audit report (section 2.3)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

A few smaller items sit at the root as well: `THIRD_PARTY_NOTICES.md` (licence records, including the FFmpeg licence check), `ui_design/` (five design images), and `delegation_progress/` (one leftover agent progress note).

2.1.1 npm workspaces

The repository is one npm workspace tree. The root `package.json` declares:

```
{
  "name": "dems-demo",
  "version": "0.1.0",
  "packageManager": "npm@11.19.1",
  "engines": { "node": ">=26.8.2 <27" },
  "workspaces": ["packages/*", "apps/*"]
}
```

(Excerpt of the root `package.json`*, other keys left out.)*

Development uses Node 26.8.2 (`.node-version`). `.npmrc` sets `save-exact=true`, so every dependency is pinned. Run `npm install` once at the root; it also applies the mobile library patch through `postinstall`. You address one workspace with `-w`, for example `npm test -w @dems/core-server`.

2.1.2 The two packages

`@dems/shared` (`packages/shared`, version 0.1.0) is what the Core and both clients agree on. It has no build step for TypeScript consumers: each sub-export points straight at a source folder. Besides the root export, the sub-exports are (`packages/shared/package.json`, `exports`):

TABLE 2.2 Sub-exports of `@dems/shared`

SUB-EXPORT	SUB-EXPORT	SUB-EXPORT
`@dems/shared/domain`	`@dems/shared/api`	`@dems/shared/realtime`
`@dems/shared/tokens`	`@dems/shared/sim`	`@dems/shared/i18n`
`@dems/shared/brand`	`@dems/shared/demo-data`	`@dems/shared/client`
`@dems/shared/live`	`@dems/shared/live/source`	`@dems/shared/live/testing`
`@dems/shared/redaction`	`@dems/shared/evidence-package`	`@dems/shared/evidence-verify`
`@dems/shared/evidence-rules`	`@dems/shared/assets/*`	three theme CSS files (`theme/heroui-web.css`, `theme/heroui-native.css`, `theme/scalar.css`)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The operation registry lives under `packages/shared/src/api/`. The OpenAPI document is generated from it:

```
npm run openapi:gen -w @dems/shared
```

`@dems/capture-protocol` (`packages/capture-protocol`, version 0.1.0) holds the message contract between the Core and the worker. Its only runtime dependency is `zod`. It defines `CAPTURE_PROTOCOL_VERSION = 6`, the same number the C worker carries in `apps/capture-worker/src/worker.h`.

HEADS-UP Neither package has a README. What's written here comes from their `package.json` files and the root README. To see what a sub-export offers, open its `index.ts`.

2.2 deploy/

`deploy/` has two folders, and only one of them describes the bench.

`deploy/bench-config/` is current. It holds byte-for-byte copies of the systemd units and the nftables rule installed on the bench host on 2026-11-06, plus `core.env.example`, a template of the Core's environment file built from the code. Its README is the full deploy-time guide: environment file, PostgreSQL roles, connection files, migrations, unit install, boot verification. Chapter 8 walks through it. The real `/etc/dems/core.env` is never in the repository.

`deploy/docker/` holds a Docker image and an offline installer from task 4.

WARNING The Docker package is marked **UNVERIFIED** (`deploy/docker/README.md`). It was built from a task-4 work-in-progress checkpoint; only the TypeScript compile check ran on that tree, and no test suite. It is not how the bench is deployed (the bench runs native systemd units). The Core's own host checks refuse `REAL_PRODUCTION` inside a container, so a container can't be a production Core as the code stands. **Unknown:** how the Docker installer is meant to relate to the bench deployment; the sources don't say. Treat it as a demo/bench package and nothing more.

2.3 docs/

`docs/` keeps the task records of earlier tasks, one folder per task: identification, research, plans, implementation and test reports.

TABLE 2.3 Task folders in `docs/`

FOLDER	TASK
`docs/2026-09-30_dems_demo_stack_setup`	The first demo stack
`docs/2026-10-04_dems_backend_rbac_pairing`	The backend: Core server, roles, pairing (Live mode)
`docs/2026-10-07_dems_real_capture_sealing`	Real capture and sealing (task 3)
`docs/2026-10-12_input_av_core_adapter_plan`	Task 4: production key, real cameras, commissioning
`docs/2026-10-19_lcd_installer`, `docs/2026-10-19_lcd_integration`	The LCD panel integration and its installer
`docs/2026-10-21_bare_metal_bench`	The bare-metal bench host
`docs/AUDIT_REPORT_2026-11-06.md`	The public, sanitized report of the 2026-11-05/06 audit

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP The audit's own task folder (its decision log, row reports and test reports) is **not** in the repository. Only `docs/AUDIT_REPORT_2026-11-06.md` is. When the root README or a code comment cites a file such as `implementation/w4_deploy.md` of the audit task folder, you won't find it in the clone.

2.4 READMEs everywhere

There are 156 `README.md` files under `apps/`. Each app has one (`apps/<app>/README.md`), and most source folders under `apps/*/src/` have their own. A source-folder README is a generated inventory with a fixed shape:

- the folder's purpose;
- a table of its files and subfolders with their roles;
- its main exports and entry points;
- its dependencies, inbound (who imports it) and outbound;
- its tests;
- the related audit decisions.

They're good for "what's in this folder and who uses it". They're not prose, and they don't explain design. A few contain generator artifacts; when a sentence reads oddly, check the code.

TIP Start at `apps/<app>/README.md`, then read `apps/<app>/src/README.md`, which indexes every top-level source folder with one line each. From there, drill into the folder you need.

2.5 scripts/

Every script you'll use day to day is wired as an `npm run` entry in the root `package.json`. The folders behind them:

TABLE 2.4 The `scripts/` folders

FOLDER	WHAT IT DOES	ENTRY POINTS
`scripts/pg`	The project-local PostgreSQL 18 cluster on `127.0.0.1:4432`, data in the git-ignored `.data/`	`npm run pg:init`, `pg:start`, `pg:stop`, `pg:status`, `pg:remove-data`, `pg:remove`, `test:pg-scripts`
`scripts/test`	The full regression run and the kill matrix	`npm run test:regression`, `npm run test:kill-matrix`
`scripts/host`	Host helpers: the numbered host-preparation scripts, the Ubuntu 24.04 host profile, the installer note for the first production boot, and `netalias`, the optional address helper	`npm run test:host-helper`
`scripts/lane`	`lane.sh`: start, stop, status and reset of the development Core instances (lanes), each with its own media store root	`scripts/lane/lane.sh start <n>`
`scripts/capture`	Development capture setup: a private FFmpeg copy as the helper program of the development sources	`npm run capture:dev-setup`
`scripts/release`	Release packaging of the web packages and licence collection	none at the root
`scripts/hardware-acceptance`	The hardware acceptance procedures (`HW-ACCEPTANCE-*.md`) and their collector scripts	none at the root
`scripts/secret-scan`	A count-only secret scan	none at the root

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP Some scripts were written on the development Mac and use macOS tools. `test:seal-e2e` and `test:dev-session-e2e`, for example, take their locks with `/usr/bin/lockf`, which Linux doesn't have. Chapter 7 covers the workflow and the lock files under `.locks/`.

CHAPTER 3

3 Core server

Most DEMS work happens in `apps/core-server`. This chapter goes from the outside (peers, the request pipeline) to the inside (capture, database, sealing, keys).

3.1 Overview

The Core is a Fastify server in TypeScript, run directly with `tsx`; there's no build step (`npm run start:server` runs `tsx src/index.ts`, `npm run dev:server` adds `watch`). The package is `@dems/core-server`, package version 0.2.0 (`apps/core-server/package.json`). **Unknown:** the runtime version word in `src/version.ts`; it wasn't read for this manual, so don't take 0.2.0 as what the server reports at runtime.

The listen host is the constant `127.0.0.1` (`LISTEN_HOST` in `src/config.ts`); no variable changes it. The port is 4420 (`DEMS_CORE_PORT`), with 4421 to 4423 for development lanes; any other value stops the boot.

HEADS-UP Why it's like this: since Live mode, the Core has been built loopback-only. The root README's rule is that no DEMS listener is exposed on another interface, through a port forward or through a tunnel.

TABLE 3.1 What the Core talks to

PEER	HOW
Web console (`apps/core-web`)	JSON API under `/api/v1` and the socket `GET /ws`, through the console's Vite proxy (one origin for the browser)
Tablet (`apps/mobile`)	The same API and socket over plain `http`/`ws` (residual `R263-TLS`). The signed paths `/media/...`, `/exports/...` and `/preview/...` carry their own bearer token
Capture worker, relay, guard (`apps/capture-worker`)	Child processes. The worker speaks capture protocol 6: one JSON object per line on stdin/stdout
LCD backend (`apps/lcd-backend`)	Channel state posted every 2 s with a 6 s lease; panel setup proxied
Transcriber (`apps/transcriber`)	A library, imported in-process by `src/transcription/`
PostgreSQL 18	`127.0.0.1:4432`, database `dems_demo` for port 4420, through Drizzle

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Figure 3.1 shows how the code is put together.

FIGURE 3.1 Internal structure of the Core server



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Modules call the top-level units only through each unit's `index.ts` (`apps/core-server/src/README.md`).

3.2 Source layout

TABLE 3.2 Folders of `apps/core-server/src`

FOLDER OR FILE	WHAT IT HOLDS
`index.ts`	Entry point: configuration, app, boot steps, listener
`app.ts`	The Fastify app factory
`config.ts`	Constants and pure parsers of the configuration
`plugins/`	Cross-cutting plugins: `config`, `db`, `logger`, and the pipeline stages (section 3.3)
`modules/`	The API operations, one folder per area (section 3.4)
`core/`	Core services and their boot checks
`engine/`	The session engine host: rehydration at boot, the committed state
`realtime/`	The `GET /ws` socket and the publish seam
`db/`	Schema, migrations, commands, seed, reset, store (section 3.8)
`capture/`	Capture supervisor, camera and microphone adapters, development fence, real-capture boot checks (`capture/boot/checks.ts`)
`capture-composition.ts`	Composition of production capture

FOLDER OR FILE	WHAT IT HOLDS
`capture-session/`	The capture of one session
`capture-notices/`	The no-device notices
`camera-preview/`	Live camera preview and idle microphone meters
`media-stream/`	Signed media sessions for playback
`credential-store/`	The one store of keys for data at rest
`sealing/`, `sealing-host/`	The sealing pipeline as a library, and its composition in the running Core
`evidence-export/`, `evidence-verify/`, `evidence-facts/`	Export, verify, and the facts the Core states about an item
`transcription/`, `transcript-package/`	Transcriber composition; transcript files of a sealed package
`lcd/`	The bridge to the LCD front panel
`host-alias-socket.ts`, `segment-probe-socket.ts`	Clients of the host helper sockets
`version.ts`	The server's version word

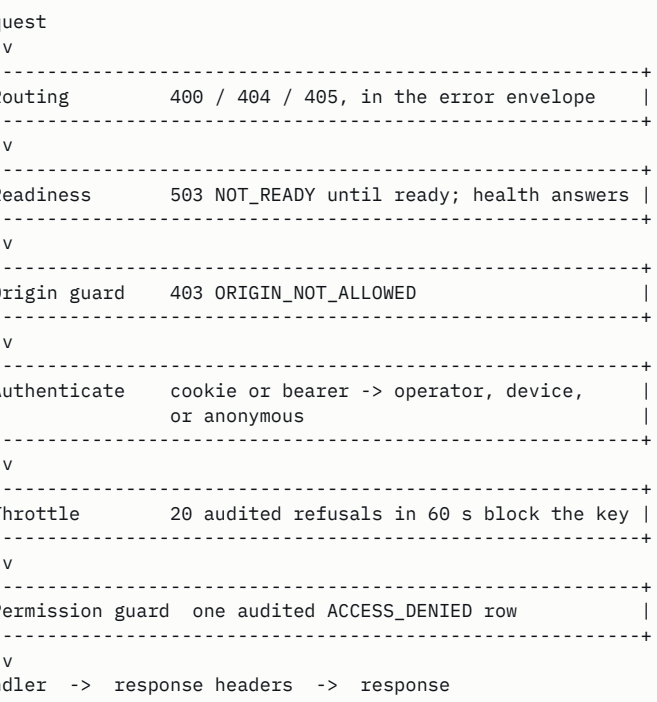
SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Outside `src/`, `cli/` holds the host commands (section 3.9) and `test/` the DB global setup, acceptance tests, static rules and test support such as the fake ONVIF camera.

3.3 Request pipeline

Every HTTP request passes the same stages in the same order (`src/plugins/README.md`). A request that fails a stage never reaches the next one.

FIGURE 3.2 The request pipeline



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 3.3 Pipeline stages

STAGE	PLUGIN	WHAT IT DOES
1 Routing	`error-envelope`	Unknown path or method: 400 `INVALID_PATH`, 404 `NOT_FOUND`, 405 `METHOD_NOT_ALLOWED` with `allow`
2 Readiness	`readiness`	Until the database and engine flags are set, everything but the health check answers 503 `NOT_READY`. Named holds (demo reset, shutdown) refuse too
3 Origin guard	`origin-guard`	Checks the `Origin` header, the session cookie and the `Authorization` header against a decision table; refuses with 403 `ORIGIN_NOT_ALLOWED`
4 Authenticate	`authenticate`	Reads at most one credential, a session cookie or a bearer token, and resolves it to an operator, a device or anonymous
5 Throttle	`throttle`	Throttles requests per caller key. A denial budget blocks a key after 20 audited refusals within 60 s
6 Permission guard	`permission-guard`	Applies the route's access rule from the shared registry; refuses with one audited `ACCESS_DENIED` row

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

After the six stages come body parsing and validation (413 `PAYLOAD_TOO_LARGE`, 400 `INVALID_JSON`, 422 `INVALID_INPUT`), the handler, and the response headers. **Unknown:** the plugin READMEs don't number these later steps or state the exact order of the response headers relative to the handler, so the figure leaves them unnumbered.

`response-headers` adds fixed security headers to every answer (`Cache-Control`, `X-Content-Type-Options`, `Content-Security-Policy`, `Cross-Origin-Resource-Policy`, `Referrer-Policy`) and a request ID. Anything unexpected becomes 500 `INTERNAL_ERROR` with no detail in the body; the error itself goes to the logs.

TIP Access rules don't live in the modules. If you add an operation, its access rule goes into the shared registry, and `permission-guard` enforces it. Don't add a permission check inside a module.

3.4 Modules and operations

Each folder in `src/modules/` registers the routes of one area from the shared operation registry. Operations have numbers, and you'll see them everywhere: in code comments, in decision logs, in test names.

TABLE 3.4 Modules and their operation numbers

MODULE	OPERATIONS	WHAT THEY COVER
`health`	`GET /api/v1/health`	Liveness and readiness
`identity`	2-6	Sign-in, sign-out, auth sessions, lockout, password policy
`rbac`	10	The role matrix as stored in PostgreSQL
`admin`	11-14	User and role administration
`pairing`	15-23	Codes, redeem, device list, revoke, unpair, credential rotation
`sessions`	24-38, 81-82	The recording lifecycle: create, pre-check, start, stop, cancel, recover, markers, notes, history; the retired audio switch (81) and the audio-source intervals (82)

MODULE	OPERATIONS	WHAT THEY COVER
`evidence`	39-43	List, get, verify, export, archive an evidence item
`devices`	44-52	The Core's recording hardware: health, audio and video status, commands
`storage`	53-55	Storage status, detail, policy
`network-time`	56-57	Network and time status
`security-audit`	58-63	Security status, audit rows, chain verification, audit and log export
`system`	64-68	System status, diagnostics, service restart, software update
`alerts`	69-70	List and acknowledge alerts
`demo`	71-75	Demo status, scenarios, faults, reset; only with the demo flag
`capture-config`	76-80	The selector, the capture-device registry, camera credentials and network, commissioning
`sealing`	83	Sealing progress
`transcripts`	84-86	Sealed transcripts and signed revisions
`evidence-export`	87, `GET /exports/:token`	Export job status and the signed download
`media-stream`	88, `GET /media/:token`	Signed playback of sealed media
`stt-config`	90-92	Speech-to-text settings and connection test
`capture-discovery`	93-94	Camera search and the candidate list
`capture-inputs`	97	Re-scan of the local microphone inputs
`camera-preview`	112-113, `GET /preview/:token`	Live preview token and idle microphone meters
LCD (`src/lcd/`)	114-116	Panel firmware word, panel setup read and write

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Two module folders serve no operation of their own: `recording-fragments` (the fragment store fed by the worker's checkpoints) and `transcript-chunks` (the transcriber's chunk checkpoints).

HEADS-UP The ranges are as the module READMEs state them at `59a683ed`. **Unknown:** the numbers of the individual admin operations, on which the sources of this manual disagree; check the shared operation registry (`packages/shared/src/api/`). **Unknown:** the contract's total size at `59a683ed`. The last count, 110 operations and 83 audit types, is from task-4 commit `1fbeada7` and wasn't redone for this document.

Besides op 41 (verify, section 3.9), op 81 (retired, section 3.6) and op 112 (preview token, section 3.5), three operations are worth knowing by number:

TABLE 3.5 Operations worth knowing by number

OPERATION	WHAT TO KNOW
op 66 `restartServices`	Refused on a Core that does its own capture

OPERATION	WHAT TO KNOW
op 78	Capture configuration update. A microphone swap between two pairs is one op 78 call with a two-line body (audit D125, D126)
op 80	"Make Mic of Camera 1 (transcribed)" in the web selector (audit D125)

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

3.4.1 API reference

The OpenAPI document is generated from the shared registry with `npm run openapi:gen -w @dems/shared`. The web console shows it as an API reference page at `/developer/api`.

3.5 Capture pipeline

3.5.1 The capture operation

The Core decides its capture operation once, at start, from three variables: the capture mode (`DEMS_CAPTURE_MODE`), the demo flag (`DEMS_DEMO_CONTROLS`) and the development flag (`DEMS_CAPTURE_DEV_SOURCES`). The one function that reads them together is `captureOperationDecisionOf` in `src/capture/fence/operation.ts`. Table 1.3 in chapter 1 shows the three operations. The boot log prints the result in the line `capture fence passed`.

3.5.2 Tracks and channels

A Core records up to four video tracks, `VIDEO-01` to `VIDEO-04`, and up to four microphone channels, `MIC-01` to `MIC-04` (bench D60 and bench D61; migrations `0039_four_video_tracks`, `0040_four_audio_channels`, `0041_capture_widening_privileges`). Section 3.6 explains how they pair up.

3.5.3 Session start

The UI shows "Connecting cameras..." and switches to recording only after the first durable fragment is on disk. A start with no camera ends as a failed start with no evidence item.

3.5.4 Cameras

- An adopted ONVIF camera has source class PRODUCTION. It can be assigned only under `REAL_PRODUCTION`; a development-capture Core refuses it.
- An assignment needs a stored stream profile, one that a connection test read from the camera's own stream. The Core picks no default (`CAMERA_PROFILE_NOT_STATED`).
- The Core doesn't follow a camera to a new address on its own. An operator with the capture configuration permission confirms the change in the web console (operation `confirmCaptureDeviceAddress`, audit row `CAPTURE_DEVICE_ADDRESS_CHANGED`).

Smart attach (bench D62) sweeps the camera segment and adopts the cameras it finds. Turn it on with `DEMS_CAMERA_AUTOATTACH=1`; `DEMS_CAMERA_SEGMENT_CIDR` must then be one IPv4 `/24` such as `192.168.x.0/24`, or the boot stops. The default camera credential (`DEMS_CAMERA_DEFAULT_USER`, `DEMS_CAMERA_DEFAULT_PASSWORD`) is offered only inside that `/24` (audit D64).

Camera link watch (audit D61). The Core polls the camera interface's link every 5 s. A link-up triggers an immediate sweep. While the link is down the sweep

doesn't run. Each link change writes one `CAMERA_LINK_DOWN` or `CAMERA_LINK_UP` audit row, instead of about 60 empty sweep rows an hour (audit D23).

3.5.5 Clocks

The product doesn't set or correct a camera's clock. Set every camera to the same time as the Core.

TABLE 3.6 Camera clock offsets

CAMERA CLOCK OFFSET	WHAT HAPPENS
below 500 ms	Recorded, not corrected; nothing is shown
500 ms to 2 s	Recorded, not corrected; the operator sees an advisory that the recording keeps the camera's own timeline and records the offset
more than 2 s behind	The camera is given up and named in a CRITICAL fault. Up to 1 s of wrongly timed video may already be in its file

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Up to 2 s, the measured offset goes into the sealed manifest per track (`tracks[].clockOffsetMs`).

3.5.6 Stream loss

When a camera's stream drops, its relay keeps its connection to the worker and reopens the camera after 2, 2, 4, 8, 16 and 30 s (six tries per loss). It accepts the reopened stream only if the parameter sets come back byte-identical; otherwise that camera's capture ends with `STREAM_INVALID`. A camera that restarts with other encoder settings mid-session isn't recorded any further.

3.5.7 Pre-check

Before a recording, the pre-check runs 12 items covering cameras 1 to 4 and microphones 1 to 4 (audit fix wave W1).

3.5.8 Live preview and microphone meters

- The live preview is MJPEG through `GET /preview/:token`. The token comes from op 112, and the URL it returns is a bearer value: never logged or stored.
- The contract caps previews with `CAMERA_PREVIEW_LIMITS` (`packages/shared/src/api/devices.ts`): 1 pipeline per camera, 4 pipelines in all, 4 viewers per camera, 16 viewers in all. Hitting the cap answers op 112 with `429 RATE_LIMITED`, reason `PREVIEW_LIMIT`.
- Op 113 serves the idle microphone meters, the levels you see while nothing is recording.
- On the bench, 4 preview pipelines measured 26.5 to 31.5 % of one CPU core each (audit D249). The load during a recording hasn't been measured.

Unknown: whether the preview pipelines on the bench are separate FFmpeg processes, and which program they run. This manual calls them "preview pipelines" and nothing more.

3.5.9 "Not reported"

On `REAL_PRODUCTION`, any value without a real data source is shown as "Not reported" (audit D25). A simulated value on a production screen is a defect; the audit's fix waves W1 and W2 removed several of them.

3.6 Camera and microphone pairs (audit D5)

Each camera is paired with one microphone: `VIDEO-01` with `MIC-01`, and so on up to `VIDEO-04` with `MIC-04`. The pairs are set in the capture setup. An operator can't move a microphone to another camera during a session: op 81, which used to switch the audio source, now answers `409 AUDIO_SOURCE_FIXED_BY_PAIRING`. A microphone change in the selector is a swap, so no camera is left without its pair.

FIGURE 3.3 Camera/microphone pairs and the missing-mic case

```
VIDEO-01 file  <---- audio ----+  MIC-01 --+--> MIC-01.m4a  (AUDIO-01)
                               |           |
                               +-----------+--> transcription tap
VIDEO-02 file  <---- audio ------- MIC-02 ----> MIC-02.m4a  (AUDIO-02)
VIDEO-03 file  <---- audio ------- MIC-03 ----> MIC-03.m4a  (AUDIO-03)
VIDEO-04 file  <---- audio ------- MIC-04 ----> MIC-04.m4a  (AUDIO-04)

- - - - - - - - - - - - paired mic missing - - - - - - - - - - - - -
VIDEO-0N file  <-- silent track    MIC-0N  x    no AUDIO-0N file
                                   manifest: NOT_ASSIGNED / NOT_OPENED
                                   notice:   PAIRED_MIC_MISSING
```

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Audio per pair. The worker opens one encoder per microphone and writes its audio twice: into its paired camera's file (`VIDEO-0N`) and into its own audio file `MIC-0N.m4a`, stored as track `AUDIO-0N`. The seal declares every audio track (manifest /2.1, section 3.9), and the verifier checks them. `MIC-01` also feeds the transcription.

Paired mic missing. A camera whose paired microphone isn't assigned (`NOT_ASSIGNED`) or couldn't be opened (`NOT_OPENED`) still records: its file gets a silent audio track, there's no `AUDIO-0N` file for that pair, and the manifest records the missing channel. The Core logs `CAPTURE_PAIRED_MIC_MISSING` at warn level and raises the notice `PAIRED_MIC_MISSING`, with an extra sentence for Pair 1 because `MIC-01` feeds the transcription. Pre-check normally catches an unassigned slot, so the usual case is a microphone that didn't open.

HEADS-UP Why it's like this: an interview must never be lost to one failed microphone. The cost is a silent track, and the manifest and notice say so.

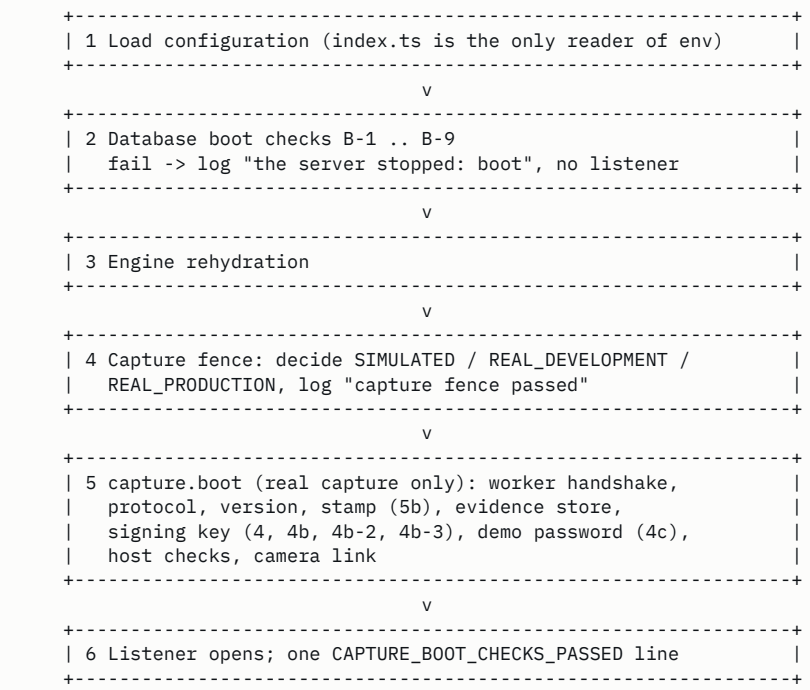
Migration `0044_recording_fragments_audio_tracks` lets `recording_fragments` accept the tracks `AUDIO-01` to `AUDIO-04`.

WARNING The pairing is proven by its parts only: unit tests, database tests, real-worker media pairs, the verifier on real packages, sealing tests with four cameras and four microphones including a missing mic and tampering. The gated end-to-end tests can't run on the bench host, because the product's own trust rules refuse their test setups (audit D153). **The first real session with four cameras and four microphones is a mandatory acceptance test before operational use.**

3.7 Boot checks

A refused start opens no listener at all; you never get a half-started Core answering requests. Figure 3.4 shows the order of the boot steps.

FIGURE 3.4 Boot sequence



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

3.7.1 Database checks

At start the Core runs boot checks B-1 to B-9: the database answers, version, schema version, runtime role, audit protection, seed, chain head, owner role. A failed check stops the start with one log line `"the server stopped: boot"` and a `"check"` field naming the step.

TABLE 3.7 Database boot check failures and their usual fix

CHECK FAILS	USUAL FIX
`B-1` the database doesn't answer	`npm run pg:start`
`B-3` schema version	`npm run db:migrate`
`B-6` not seeded	`npm run db:seed`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

3.7.2 Capture refusals

The capture fence refuses a bad combination of flags, and in real capture (`REAL_DEVELOPMENT` or `REAL_PRODUCTION`) the boot step `capture.boot` adds its own checks. Each refusal logs `"the server stopped: boot"` with the check and a code, and no listener opens. In `SIMULATED` none of the real-capture checks run.

TABLE 3.8 Boot refusal codes

CODE	REFUSED WHEN
`REAL_CAPTURE_WITH_DEMO_CONTROLS`	`DEMS_CAPTURE_MODE=real` together with `DEMS_DEMO_CONTROLS=1`
`DEV_SOURCES_NEED_REAL_CAPTURE`	`DEMS_CAPTURE_DEV_SOURCES=1` with capture mode `simulated`
`DEV_FLAG_WITH_LAN_LISTENER`	The development flag is set while a listener outside loopback is configured

CODE	REFUSED WHEN
DEV_SOURCE_WITHOUT_FLAG	REAL_PRODUCTION, and the selector assigns a development source
CAPTURE_WORKER_NOT_CONFIGURED, CAPTURE_WORKER_PROGRAM_UNUSABLE	DEMS_CAPTURE_WORKER unset, or the program isn't a private executable (owned by root or the Core's user, not writable by group or others)
CAPTURE_EVIDENCE_STORE_UNUSABLE	The media store fails a rule (readable by others, wrong owner, no full sync, no hard links); the reason is named
CAPTURE_WORKER_NO_HANDSHAKE	The worker doesn't answer
CAPTURE_WORKER_PROTOCOL_MISMATCH	The worker speaks another protocol version
CAPTURE_WORKER_PROOF_BUILD	The worker is a proof build (a negative control of the tests)
CAPTURE_WORKER_VERSION_UNKNOWN	The worker names an unknown version
CAPTURE_WORKER_MEDIA_LIBRARY_MISMATCH	The worker runs with another libavformat than the one it was built against
CAPTURE_WORKER_BUILD_STAMP_STALE	Check 5b: the stamp file doesn't name this binary's sha256 and source
CAPTURE_WORKER_VERSION_MISMATCH	The worker's version doesn't match what the Core expects
TRANSCRIBER_CONFIG_UNRESOLVABLE	The stored speech-to-text configuration can't be resolved on this host

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The signing-key and operator codes (CAPTURE_SIGNING_KEY_UNUSABLE, CAPTURE_DEVELOPMENT_SIGNING_KEY_IN_PRODUCTION, CAPTURE_DEMO_OPERATOR_PASSWORD_IN_PRODUCTION and the 4b-2/4b-3 codes) are in Table 3.9.

The **build stamp** exists because a stale worker once passed its boot check while refusing every 4+4 plan (audit D15). The worker now names its version and writes a stamp next to its binary, and the Core checks both (audit D48).

HEADS-UP The root README's deploy section abbreviates these as STAMP_STALE, VERSION_MISMATCH and PROTOCOL_MISMATCH. The codes in the log are the full CAPTURE_WORKER_... names above.

3.7.3 Production key and operator checks

These run in capture.boot too, after the fence and before the worker starts. All of them apply to REAL_PRODUCTION only, except check 4.

TABLE 3.9 Production key and operator checks

CHECK	REFUSED WHEN	CODE
4	No signing key loads; also a key marked compromised, or a certificate that doesn't cover the current time	CAPTURE_SIGNING_KEY_UNUSABLE
4b	The key is a development key	CAPTURE_DEVELOPMENT_SIGNING_KEY_IN_PRODUCTION
4b-2	The certificate's organisational unit isn't exactly Evidence signing <installation id> for the configured DEMS_INSTALLATION_DEVICE_ID, or no id is configured	CAPTURE_SIGNING_KEY_INSTALLATION_MISMATCH
4b-3	The configured installation id is one of the reserved ids DEMS-TEST-01, DEMS-IR-01, DEMS-JKT-07	CAPTURE_INSTALLATION_ID_RESERVED_FOR_TESTS

CHECK	REFUSED WHEN	CODE
4c	An operator still has the published demo password, or the operators can't be read	`CAPTURE_DEMO_OPERATOR_PASSWORD_IN_PRODUCTION`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

WARNING The old part of the root README contains published demo credentials for synthetic accounts. Production refuses to boot while any operator still has that password (check 4c). That's why a production unit uses the factory seed and `operator:set-password`, never the demonstration seed (chapter 8).

These checks are necessary conditions, not proof of correct provisioning: a start can't show that the right person made the key.

3.7.4 Platform and host checks

- `REAL_PRODUCTION` starts on Linux and macOS only. Anything else is refused with `CAPTURE_HOST_PLATFORM_NOT_SUPPORTED`.
- Linux asks every question of the host profile (`scripts/host/HOST-PROFILE-UBUNTU-24.04.md`) at every start; each missing prerequisite refuses with its own code. This hardened form is unproven on hardware.
- macOS doesn't ask the host profile. The Core logs the warning `CAPTURE_HOST_PROFILE_NOT_ASKED`, and packages sealed there carry a signed member saying so.

The address helper (`scripts/host/netalias/`) is an optional root service on Linux that adds and removes one temporary host address for camera commissioning. It's all or none:

- absent entirely: the Core logs the warning `CAPTURE_HOST_ALIAS_HELPER_ABSENT` and starts;
- present but wrong (a socket of another owner or mode, a configured path that isn't the installed one, or a partial install): the start is refused.

3.7.5 A healthy production start

A healthy production start logs exactly one line with the code `CAPTURE_BOOT_CHECKS_PASSED`, `"operation":"REAL_PRODUCTION"`, `"checks":"real"` and the worker version, `capture-worker/0.8.0` at this commit. On the bench:

```
journalctl -u dems-core -b --no-pager | grep CAPTURE_BOOT_CHECKS_PASSED
```

A start without that line is not a healthy production start, whatever else the log says.

3.8 Database and migrations

3.8.1 Server, roles and databases

The Core uses PostgreSQL 18 on `127.0.0.1:4432`, TCP only. It refuses port 5432 with `FOREIGN_DATABASE_PORT`.

HEADS-UP Why it's like this: on the development Mac, another PostgreSQL server already owned port 5432. The Core refuses that port outright instead of trusting whatever answers there.

TABLE 3.10 Database roles and databases

NAME	WHAT IT IS
`dems_owner`	Owner role. Owns the database, runs migrations, seed, reset, and the DB tests
`dems_app`	Runtime role. The running Core connects as this role, with least privilege
`dems_admin`	Superuser of the development cluster only, used for provisioning
`dems_demo`	Database of the Core on port 4420
`dems_lane_1`, `dems_lane_2`, `dems_lane_3`	Databases of the lanes on ports 4421, 4422, 4423

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

3.8.2 Connection files

The Core takes its database credentials from a file, not from the environment. The files live in the data directory, `<repo root>/.data` by default (`/opt/dems/core/.data` on the bench):

TABLE 3.11 Connection files

FILE	READ BY	KEYS
`pg.env`	The Core	`DEMS_PG_HOST`, `DEMS_PG_PORT`, `DEMS_PG_DATABASE`, `DEMS_PG_APP_ROLE`, `DEMS_PG_APP_PASSWORD`
`pg.owner.env`	The `db:*` commands	`DEMS_PG_HOST`, `DEMS_PG_PORT`, `DEMS_PG_DATABASE`, `DEMS_PG_OWNER_ROLE`, `DEMS_PG_OWNER_PASSWORD`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

`pg.env` must be mode 0600; the Core refuses it with any group or other bit set. Any `PG*` variable in the environment is removed at start. In development `npm run pg:init` writes these files; for the bench, see `deploy/bench-config/README.md`.

3.8.3 Migrations

The migrations are SQL files in `src/db/migrations/`, from `0000_prelude` to `0044_recording_fragments_audio_tracks`, applied by `db:migrate` in the order of the Drizzle journal (`meta/_journal.json`). A table migration is usually followed by a separate `_privileges` migration that grants the runtime role exactly what it needs. The bench's migration ledger holds 45 entries, one per file.

TABLE 3.12 Migrations, grouped

MIGRATIONS	THEME
0000-0003	Prelude, tables, audit guards (the audit table is append-only with a hash chain, `0002_audit_guards`), runtime privileges
0004-0005	Capture tables and their privileges, including the write-only credential store
0006-0007, 0011-0012	Speech-to-text configuration, global and per session
0008-0010	Capture sources and credential replacement
0013-0014	Key id, fault reason, media root
0015-0016	Capture streams and audio inputs
0017-0020	Network commissioning; credential-replaced reason

MIGRATIONS	THEME
0021-0023	Profile codes
0024-0025	Installation acceptance
0026-0031	Address helper removal reasons; ONVIF ports of the camera network
0032-0034	Capture device retirement; capture-config permission removed from Supervisor
0035-0038	Operator administration, operator purge, operator credential creation, purge reseal correction
0039	Four video tracks (bench D60)
0040	Four audio channels (bench D61)
0041	Privileges for the widening of 0039 and 0040
0042	Capture config constraint repair, so a production database equals a fresh install (audit D22)
0043	Operator purge scope: the purge keeps evidence-chain audit types
0044	`recording_fragments` accepts `AUDIO-01` to `AUDIO-04`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

WARNING An applied migration is never edited. Repairs go forward as a new migration (audit D22). 0042 exists because that rule was once broken. No down migrations are documented; on the bench, rolling back a migration means restoring the dump taken before the deploy.

3.8.4 Database commands

Run them from the repository root. Each database command takes exactly one database name (`--database dems_demo`, or a lane database):

TABLE 3.13 Database commands

COMMAND	WHAT IT DOES
`npm run db:create`	Creates the database; leaves an existing one as it is
`npm run db:migrate`	Applies pending migrations; applies nothing the second time
`npm run db:seed`	Seeds the demo data; `--factory` seeds a production unit with no operator password. Refuses a database that is already seeded
`npm run db:reset`	Returns the database to the seed. Keeps 18 tables, among them paired devices, sessions, settings, capture devices and the media store root (`src/db/reset/kept-tables.ts`)
`npm run db:status`	Shows the migration state
`npm run db:generate -w @dems/core-server`	Generates a migration with drizzle-kit

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

`db:reset` refuses, with nothing changed, in these cases (`src/db/reset/`):

TABLE 3.14 Reset refusals

REFUSAL	WHEN

REFUSAL	WHEN
RESET_BLOCKED_REAL_EVIDENCE	Real sealed evidence exists in the database or on disk, or an unsealed production recording, any quarantine folder, an unreadable manifest, or a media store root that can't be listed
RESET_STORE_ROOT_CONFLICT	DEMS_MEDIA_DIR names one media root and the database stores another
RESET_STORE_ROOT_UNKNOWN	No DEMS_MEDIA_DIR and no root stored in the database
RESET_TARGET_IN_USE	A Core of that database is running

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

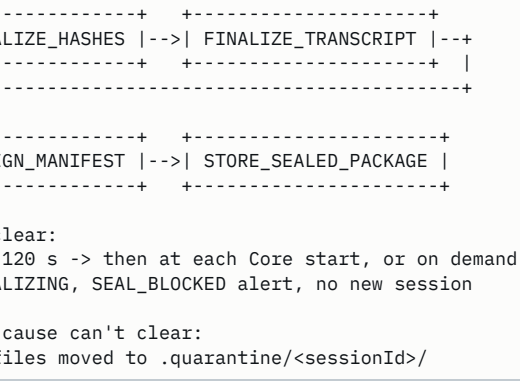
HEADS-UP Why it's like this: sealed evidence is under legal hold. The product has no way to erase a sealed package, and a reset that could do so by the back door is refused.

3.9 Sealing, verify and export

3.9.1 The sealing pipeline

After a session stops, the Core seals it in six steps (SEALING_STEP_ORDER in src/sealing/pipeline/contract.ts).

FIGURE 3.5 Sealing pipeline



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Each fragment becomes one AES-256-GCM message. The largest fragment the pipeline seals is the verifier's own bound (SEALING_MAX_FRAGMENT_BYTES), so the Core never produces a package its own verifier refuses.

When sealing fails. A cause that may clear is retried after 5 s, 30 s and 120 s, then at every Core start and on demand ("Retry sealing" in the web console). Meanwhile the session stays FINALIZING, the Core accepts no new session, and one CRITICAL alert SEAL_BLOCKED names the cause. The Core doesn't notice by itself when the cause heals: someone presses "Retry sealing" or restarts the Core.

When nothing can be sealed, or a permanent cause remains, the session ends **INCOMPLETE** with no evidence item. Its files are moved, never deleted, to <media store>/.quarantine/<sessionId>/ (folders 0700, files 0600). The product can't bring them back into a seal.

Plaintext removal needs proof: the manifest names the file, the sealed counterpart exists, and the bytes hash to the manifest's plaintext hash. An unproven file goes to the quarantine.

WARNING Until its seal completes, a recording lies in plaintext on the media store; the quarantine stays plaintext for good. Only file permissions protect it (media store root and hidden folders owned by the Core's user, no bit for group or others; a real-capture Core refuses to start otherwise). A stolen or imaged disk exposes every unsealed recording.

3.9.2 The package

TABLE 3.15 Files of a sealed package `SESSION-<sessionId>/`

FILE	WHAT IT IS
`session-manifest.json`	The manifest: tracks, hashes, facts, installation identity
`signature.sig`	The signature over the manifest
`device-certificate.pem`	The self-signed certificate of the signing key
`fragments/VIDEO-0N.mp4`	The encrypted camera files, one per recorded camera
`fragments/fragments.json`	The fragment list
audio files of the pairs	The `AUDIO-0N` tracks that manifest /2.1 declares
`audit-session-extract.jsonl`	The session's audit rows. Required, but **not** covered by the signature
`transcript/`	The sealed transcript and `transcript-revisions.json`, when a transcript was sealed
`export/`	Exports of this package, once exported

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Packages with camera/microphone pairs use manifest schema `dems.evidence-manifest/2.1` (audit D45). It adds a signed statement of missing microphones, `audioTracksMissing`, with the values `NOT_ASSIGNED` and `NOT_OPENED`. Older /2 packages still verify, byte for byte as before (audit D119).

Two optional signed facts may appear in a manifest:

- `audio.absentOnRestart[]`: no sound was recorded across a worker restart inside the session (verify notice `AUDIO_ABSENT_ON_RESTART`);
- `tracks[].absentInSession[]`: a camera stopped delivering inside the session, with the bounds of each absence (notice `VIDEO_ABSENT_IN_SESSION`).

A package without such an event carries neither member.

3.9.3 Verify

Op 41 verifies a sealed package at the Core: it opens the fragments with the session keys, runs the checks, and records the verdict. At task 3 verify ran 15 checks and named an altered fragment exactly. Notices report a fact without failing the package:

TABLE 3.16 Verify notices

NOTICE	MEANING
`AUDIT_EXTRACT_NOT_SIGNED`	The audit extract is present but not covered by the signature (every package)
`HOST_PROFILE_NOT_ASKED`	Sealed on a host where the Linux host profile wasn't asked (macOS)
`SIGNED_BY_COMPROMISED_KEY`	Signed by a key marked compromised later; passes on the Core only if it matches the digest stored at the seal

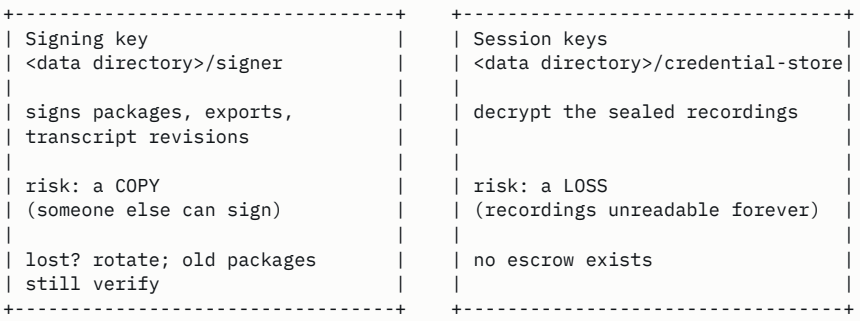
NOTICE	MEANING
REVISION_APPEND_PENDING	A transcript revision is staged and not yet appended
EXPORT_FOLDER_UNEXPECTED_FILE	A stray file in the package's export/ folder

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

3.9.4 Two keys, opposite risks

The Core holds two kinds of key, and they fail in opposite ways.

FIGURE 3.6 The signing key and the session keys



SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

The signing key is a software Ed25519 key with a self-signed certificate valid 1,825 days. There's no certificate authority behind it and no revocation channel; a receiver trusts a package only through the certificate fingerprint handed over out of band.

The session keys live under <data directory>/credential-store/session-keys-<scope>/, where the scope is derived from the absolute path of the media store root.

WARNING Moving the media store to another path means renaming its session-key folder by hand. A Core started on the moved store looks for its keys under the new scope; if the folder isn't renamed, the sealed packages can't be decrypted. No command does this rename. And since there's no escrow of the session keys, losing the credential store loses every sealed recording.

TABLE 3.17 Key and evidence commands

COMMAND	WHAT IT DOES
npm run evidence:production-key -w @dems/core-server -- --data-dir <dir>	Makes the production signing key in <dir>/signer (organisation and installation id typed at prompts). Refuses a non-empty store (KEY_EXISTS)
npm run evidence:rotate-key -w @dems/core-server -- --data-dir <dir>	Makes a new signing key and makes it active (signer/ACTIVE); the old key is kept
npm run evidence:mark-compromised -w @dems/core-server -- --data-dir <dir> --key-id <key id>	Marks one named key compromised. No command takes a mark back
npm run evidence:dev-key -w @dems/core-server	A development signing key, for development capture only
npm run evidence:verify-export -- <archive.zip>	The receiver's check of an exported package, without a Core

COMMAND	WHAT IT DOES
`npm run operator:set-password -w @dems/core-server -- --database <name> --data-dir <dir> --username <username>`	Sets an operator's password at a hidden prompt

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Run the key commands as the Core's user, never as root, with the Core stopped. After a rotation, restart the Core: until then seals, exports and transcript revisions are refused with `ACTIVE_KEY_CHANGED`. After a suspected compromise: mark, rotate, start the Core, tell every receiver.

3.9.5 Export

An export writes a decrypted archive, `SESSION-<sessionId>/export/EXP-<id>.zip`, with an export manifest signed by the active key. The archive stays on the Core's disk after the download. The download link (section 1.4) lives 60 s, and every download is audited.

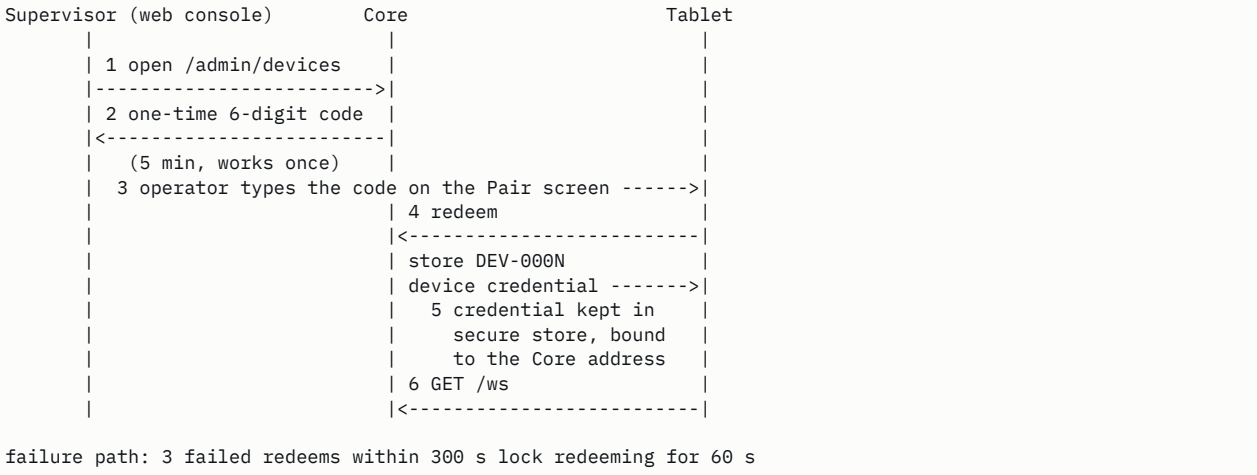
Composite video export isn't available in this build: it ends as a failed job with `COMPOSITE_ENCODER_NOT_SELECTED`. It waits on an open licence decision about which FFmpeg build a distributed Core carries.

3.10 Pairing and realtime

3.10.1 Pairing a tablet

A tablet must be paired with a Core before it can sign in.

FIGURE 3.7 Pairing sequence



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The pairing operations are 15 to 23 (`src/modules/pairing`). A Supervisor issues the code under **Admin > Paired Devices** (`/admin/devices`); the paired tablet then appears there as `DEV-000N` and can be revoked. The code length and lifetime above are as the READMEs state them. No README contains a code; each one is issued fresh.

3.10.2 The realtime socket

There's one socket route, `GET /ws`. An admission table decides who a socket is when it upgrades; the hub then decides which sockets get which message. Units of work publish through the `publish` seam after their commit (`src/realtime/`).

- At most four tabs per sign-in. A fifth console tab of the same sign-in displaces the oldest.
- The tablet sees two deliberate close codes: `4403` when its device was revoked, `4401` when its session was ended. Neither is an outage: the tablet's socket client doesn't reconnect by itself, and the app first deletes the retired credential or forgets the session, then opens a new socket.

HEADS-UP A Live recording needs a paired tablet connected. After a Core start, the Core's tablet link stays offline until the app on a paired device connects, and until then every pre-check is blocked on the Core link, even when you start the recording from the web console.

A restart of the Core ends every sign-in.

3.11 LCD bridge

`src/lcd/` is the Core's side of the front panel. The panel channels map to the video tracks: CH1 to CH4 are `VIDEO-01` to `VIDEO-04`.

- `bridge.ts` builds each channel's state and posts it to the LCD backend (`/api/dems/ch/<n>/status` on `DEMS_LCD_BACKEND_URL`) every 2 s with a 6 s lease. Each request has a 1 s timeout, and updates don't overlap.
- The payload per channel is `rec`, `cam`, `dev`, `lease:6`, `clear_file:true`. No evidence file name or path, case name, operator identity, camera credential or session token is sent.
- If the Core stops posting, the recording state expires with the lease. An orderly shutdown sends STOP on every channel.
- The camera lamp is lit only for a camera stated reachable or with an open stream (audit fix wave W4, audit D234).
- Op 114 reads the panel's firmware word and link verdict (cached 5 s). Ops 115 and 116 proxy the panel setup behind the vendor-account gate (bench D57); the Core stores none of it.

`DEMS_LCD_BACKEND_URL` must be `http://127.0.0.1:<port>`; unset, the bridge is off. Chapter 6 covers the LCD backend and the panel.

3.12 Transcription in the Core

`src/transcription/` runs the `apps/transcriber` library on the `MIC-01` tap, with the Core's database and credential store.

- Settings are ops 90 to 92 (`getSttConfig`, `updateSttConfig`, `testSttConnection`). Only the web console can set the credential field, which is write-only and holds a reference, never the key.
- The transcriber host file `<data directory>/transcriber/host.json` tells the Core where the engine and models are. It's written by hand and holds paths only.
- If the stored configuration can't be resolved on the host, a real-capture Core stops at boot with `TRANSCRIBER_CONFIG_UNRESOLVABLE`.
- Speaker labels are "unavailable" in this build: the running Core composes no speaker-labelling engine, whatever the setting says.

Chapter 6 covers the transcriber itself.

3.13 Configuration summary

The Core reads its environment once, at start, in `index.ts`. Section 13.2 of the appendix lists every variable; the template `deploy/bench-config/core.env.example` is built from the code.

A few things are deliberately **not** configuration:

- There's no session secret. The session cookie name is derived from the port: `dems_session_<port>`.
- No variable names a signing-key path. The signing key and the session keys live in the Core's key store.
- The database password isn't in the environment; it's in `pg.env` (section 3.8).
- `NODE_OPTIONS` is only reported in the boot log (set or not set). Don't set it.

Persistent errors also go to `apps/core-server/logs/errors.log`.

WARNING Never commit a filled `core.env`, `pg.env` or `pg.owner.env`; the repository is public (chapter 8).

CHAPTER 4

4 Web console

The web console is where operators run sessions and where administrators manage users, devices and the installation. It lives in `apps/core-web` (package `@dems/core-web`). This chapter covers how it finds the Core, which pages it has, and how it behaves on the bench host.

Decision numbers in this chapter come from the audit log unless they're marked "bench".

4.1 What it is

It's a React app built with Vite, TanStack Router, Tailwind CSS and TanStack Query (`apps/core-web/README.md:7`). The pages are:

- Dashboard, Session Monitor, Start Session;
- Evidence;
- Devices and Inputs, Paired Devices;
- Users and Roles, Security and Audit;
- Storage, Network Time, System.

The shared contract (API client, operations, words, theme tokens) comes from `packages/shared` (`@dems/shared`). The console has no secrets of its own. It uses the operator's session cookie, which the Core sets (`apps/core-web/README.md:70`).

Source layout, briefly (`apps/core-web/README.md:22-35`):

TABLE 4.1 Folders of `apps/core-web/src`

FOLDER OR FILE	WHAT IT HOLDS
`main.tsx`, `router.tsx`, `routeTree.gen.ts`	App entry, router setup, the generated route tree
`routes/`	File routes: `login.tsx`, `_app.tsx` (the signed-in shell) and `_app/` (every page)
`features/`	One folder per page area (`devices`, `evidence`, `session-monitor`, `session-new`, `users-roles` and so on)
`components/`	`shell` (layout, navigation, top bar, notifications), `ui`, `composed`, `media`
`runtime/`	Boot, data mode resolution, the production lock, error sink, owed sign-out storage
`lib/`	Core endpoint and queries, error words, download, CSP of the packaged build
`motion/`, `i18n.ts`, `i18n-web.ts`, `assets/`, `test/`	Motion policy, words, brand assets, page-level tests

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

`public/` holds `favicon.svg` and `kiosk.html`. Every source folder has its own README listing files, exports and tests. Start at `apps/core-web/src/README.md`.

4.2 Data modes and the proxy

The console runs in one of two data modes (`apps/core-web/README.md:9-12`):

TABLE 4.2 Data modes of the console

MODE	SOURCE OF TRUTH
LIVE	The Core server. Everything you see comes from `apps/core-server`
SIMULATED	An in-browser simulator from `@dems/shared`. Demos and development only

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

4.2.1 How LIVE reaches the Core

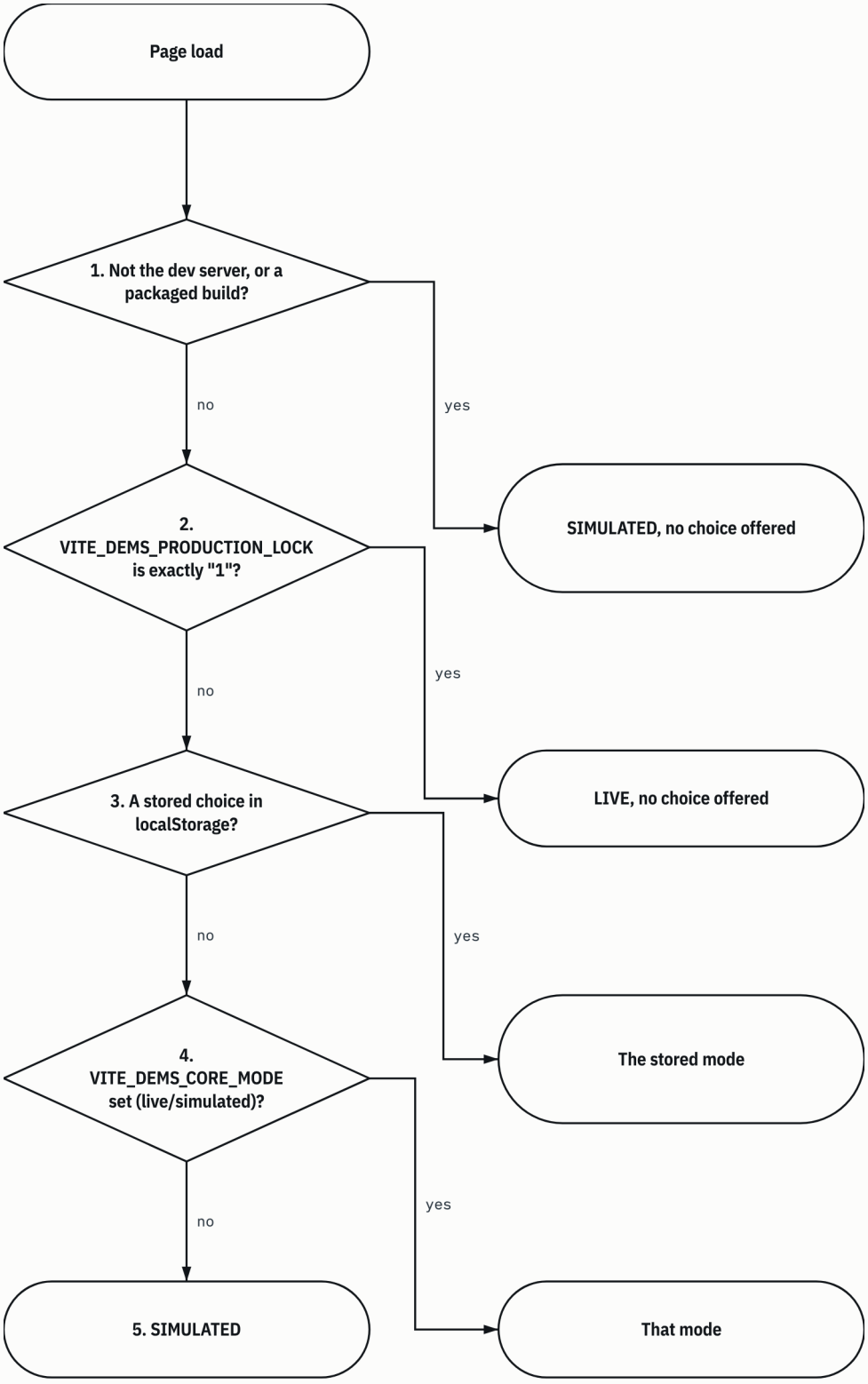
In LIVE mode the browser never talks to the Core directly. The Vite server proxies `/api/v1`, `/ws`, `/media`, `/exports` and `/preview` to `127.0.0.1:<DEMS_CORE_PROXY_PORT>` (`vite.config.ts`). The browser sees one origin, so the session cookie stays same-origin. The proxy forwards `Origin` and `Cookie` unchanged, and the Core's origin guard checks them (`apps/core-web/README.md:11`).

HEADS-UP Open the console as `http://127.0.0.1:5173`, never `http://localhost:5173`. The Core accepts only the exact `127.0.0.1` origins (`WEB_CONSOLE_ORIGINS` in `apps/core-server/src/config.ts`). On `localhost` the console shows "Live mode needs another address" and any sign-in is refused. Development lanes use ports 5176 to 5178 (`apps/core-web/README.md:16`).

4.2.2 Which mode wins

The mode of a page load is resolved once, by `resolveCoreMode` in `apps/core-web/src/runtime/core-mode.ts:91-104`. The first rule that matches wins.

FIGURE 4.1 How the console picks its data mode (resolveCoreMode).



SUMBER DEMS repository, commit 59a683ed. Redrawn with datum_diagram.py from the box drawing of the source chapter. Arrows without a label show the order of steps, as in the source drawing. **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The stored choice lives per origin in `localStorage['dems.core.mode.v1']` (README.md:539). Under the production lock (rule 2) neither the stored choice nor the env preselection is read.

HEADS-UP A console built with `vite build` is always SIMULATED. Rule 1 says LIVE exists only on the dev server. That's why production runs the Vite server as a systemd unit instead of serving a static build (`apps/core-web/src/runtime/core-mode.ts:92-94`; `deploy/docker/README.md:40`).

4.3 Routes

Every page inside the signed-in shell is a file route under `apps/core-web/src/routes/_app/`. The layout `_app.tsx` sends a signed-out visit to `/login?redirect=...` before any of these load. `/` redirects to `/dashboard` (`apps/core-web/src/routes/_app/README.md:7-40`).

TABLE 4.3 Console routes

ROUTE	PAGE	GUARD OR NOTES
`/login`	Sign-in	Shows the data mode selector unless the production lock is on
`/dashboard`	Dashboard	
`/devices`	Devices & Inputs	
`/evidence`	Evidence list	Search: `q`, `status`, `caseType`, `investigator`, `from`, `to`, `page`
`/evidence/$evidenceId`	Evidence detail	`tab`: `media`, `metadata` or `integrity`
`/session`	Session Monitor	Shows NO SESSION when idle
`/session/new`	Start Session	`step` 1, 2 or 3; needs `SESSION_START`, otherwise sent to `/dashboard`
`/session/history`	Session history	`q`, `state` (`SEALED` or `INCOMPLETE`); every role may open it
`/alerts`, `/storage`, `/system`, `/network-time`, `/security-audit`	Alerts, Storage, System & Maintenance, Network & Time, Security & Audit	`/system` hosts the Demo Control Panel
`/stack-check`	Developer diagnostic of the console build	Reachable by URL only, for every signed-in operator
`/developer/api`	API reference	Lazy chunk; the Scalar reference loads only here
`/admin/users`	Users & Roles	`USER_ADMIN`
`/admin/devices`	Paired Devices	`PAIRING_MANAGE`
`/admin/commissioning`	Commissioning	`CAPTURE_COMMISSION` (Technician only)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The admin guard is `requireCorePermission` in `routes/_app/admin/-guard.ts`. In LIVE mode an operator without the permission goes to `/dashboard`, and `noteDeniedPage` records which page was refused. The `-` prefix keeps that file out of the route tree.

Why it's like this: the guard on the client is a convenience. The real check is the Core's permission guard, which refuses the operation and writes one audited `ACCESS_DENIED` row (chapter 3).

4.4 Features

4.4.1 Session flow

Start Session is a three-step page (`/session/new?step=1..3`); an operator without `SESSION_START` never sees it. A session shows "recording" only after the first durable fragment is on disk. Before that the console says "Connecting cameras...". A start with no camera ends as a failed start with no evidence item (README.md:706). The Session Monitor (`/session`) follows the running session, and Session history (`/session/history`) lists the past ones with their end state.

The pre-check has 12 items covering cameras 1-4 and mics 1-4 (AUDIT §4, W1).

HEADS-UP In LIVE mode a recording needs a paired tablet that is connected to the Core. Until one connects, every pre-check is blocked on the Core link, even when you start from the console. The message is "No paired tablet is connected to the Core server. Open the DEMS app on the paired tablet so that it connects, then re-run the checks." (README.md:570).

4.4.2 Devices & Inputs

This page (`/devices`) holds the capture setup:

- Pair rows: each row reads "Camera N <-> Mic N", the D5 pair of VIDEO-0N and MIC-0N. You can't move a mic to another camera mid-session. Changing a mic in the selector is a swap, and the console asks you to confirm it (README.md:142; AUDIT §4 W3, D125 and D126).
- One mic status reader: Audio inputs and Registered microphones read the same status since W4, so one mic can't show two states (AUDIT §5, W4-2).
- Camera search: `Search for cameras` finds ONVIF devices. The search connects to nothing; make, model and name are what the device announced and aren't checked. Connecting a camera asks for the user name and password set on that camera. They're typed in the console only, are write-only and are never shown again (README.md:704).
- Registered device lists for cameras and microphones (AUDIT §4, W2).
- Front Panel / LCD connection: under Devices & Inputs, this view shows the LCD backend's configuration, channel state, HTTP reachability and USB responses (`apps/lcd-backend/README.md:7`; chapter 6).

W4-2 (D238) also fixed the camera search's "View device" route, and stopped counting the PoE switch and the recording lamp on a Core that captures for itself.

4.4.3 Evidence

The Evidence list (`/evidence`) filters by text, status, case type, investigator and date range. The detail page has three tabs: media, metadata and integrity. Verify runs on the Core (op 41). The integrity tab lists the per-pair audio tracks AUDIO-0N next to the video tracks (`apps/core-web/README.md:82`).

Export writes a decrypted archive `EXP-<id>.zip` on the Core. When the job finishes, the console offers **Download export**. The link is signed, bound to your sign-in, and lives 60 seconds. Every download is audited. The tablet only says "Download from the web console." (README.md:774).

4.4.4 Paired Devices

A Supervisor (permission `PAIRING_MANAGE`) opens **Admin > Paired Devices** (`/admin/devices`), issues a one-time six-digit code for a tablet, and can revoke a paired device (`DEV-000N`) (README.md:136). Chapter 5 covers the tablet's side.

4.4.5 Administration and the demo panel

Users & Roles (`/admin/users`), Security & Audit (`/security-audit`) and System & Maintenance (`/system`) cover operators, the audit trail and host settings. The STT settings (transcription on or off, backend, model, credential reference, **Test connection**) sit under System & Maintenance → Settings → **Transcription (STT)** (README.md:784). Only the web console may set the STT credential field; the same request from a tablet is refused (security finding S99-09).

The Demo Control Panel shows on `/system` only when the Core was started with demo controls on (`DEMS_DEMO_CONTROLS=1`) (README.md:410, 578). Demo controls don't exist on a Core in real capture.

4.5 Live previews

Camera tiles on Devices, Dashboard, Session Monitor and Start Session show the Core's live MJPEG preview (bench D51/D53; W4, D246). The tile asks the Core for a preview token (op 112) and opens `GET /preview/<token>` (`apps/core-web/README.md:81`).

How the tiles behave:

TABLE 4.4 How the live preview tiles behave

RULE	DETAIL
Reach-gated	No token is requested while the Core's last reading says the camera is not reachable, until the operator presses "Try again" (`features/devices/reach-gated-preview.tsx`)
Backoff	A failed picture is retried with a delay that grows from 2 s to 30 s
Hidden tab	Streams close when the tab is hidden (Devices, Dashboard, Session Monitor, Start Session)
Pipeline cap	Devices tiles open automatically within the Core's pipeline cap (`features/devices/camera-live-preview.ts`)
Monitor re-check	The Session Monitor re-checks, once per mount, a camera that has been unreachable for 60 s or more (D239)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Before W4, only Devices and Start Session had live tiles. W4-1 added Dashboard and Session Monitor (AUDIT §5).

WARNING The preview URL is a bearer value. Anyone who has it can watch that camera until the token expires. The console never logs or stores it. Don't paste it into tickets or chat either.

4.6 Production lock

On the bench host the console runs with `VITE_DEMS_PRODUCTION_LOCK=1`. The value must be exactly `1`; anything else is off (`isProductionLockOn` in `core-mode.ts`). With the lock on (D25, D47):

- the console is always LIVE;

- it ignores any stored choice and `VITE_DEMS_CORE_MODE`;
- the login page offers no Simulated option.

On a `REAL_PRODUCTION` Core, any value without a real data source shows "Not reported" instead of a simulator value (`apps/core-web/README.md:80`).

The lock is set by a systemd drop-in, not by the unit itself (`deploy/bench-config/README.md:13`):

```
# /etc/systemd/system/dems-console.service.d/production-lock.conf
[Service]
Environment=VITE_DEMS_PRODUCTION_LOCK=1
```

Chapter 8 shows the full `dems-console.service` and how to install it.

4.7 Sign-in behaviour

TABLE 4.5 Sign-in behaviour of the console

EVENT	EFFECT
15 minutes without activity	The web sign-in expires (README.md:540)
8 hours after sign-in	The web sign-in expires, active or not (README.md:540)
Fifth console tab on the same sign-in	Displaces the oldest tab, which reconnects and displaces the next; at most four tabs per sign-in (README.md:572)
Restart of `dems-console`	Ends every web sign-in (README.md:85; D115, D246)
Restart of `dems-core`	Ends every sign-in, web and tablet (README.md:85)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP Stopping `dems-console` leaves the unit in state `failed` with status 143. That's the Vite server exiting on SIGTERM. It starts cleanly again; you don't need to fix anything (`apps/core-web/README.md:76`).

4.8 Commands and environment

All commands run from the repository root (`apps/core-web/README.md:49-57`).

TABLE 4.6 Console commands

WHAT	COMMAND
Dev server, SIMULATED	`npm run dev:web`
Dev server, LIVE against the Core on 4420	`DEMS_CORE_PROXY_PORT=4420 VITE_DEMS_CORE_MODE=live npm run dev:web`
Build	`npm run build -w @dems/core-web` (`vite build && tsc -b`; the root `npm run build` builds shared, mock-core and the console)
Preview a build	`npm run preview -w @dems/core-web` (serves on `127.0.0.1:4173`)
Tests	`npm test -w @dems/core-web` (Vitest, jsdom)
Typecheck	`npm run typecheck -w @dems/core-web` (`tsc -b`)
Lint	`npm run lint -w @dems/core-web` (oxlint)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The dev server binds `127.0.0.1:5173` with a strict port.

The console reads these variables at dev-server start or at build time. No `.env` file is used (`apps/core-web/README.md:61-68`).

TABLE 4.7 Console environment variables

VARIABLE	READ BY	PURPOSE
`DEMS_CORE_PROXY_PORT`	`vite.config.ts` (dev server only)	`4420` to `4423` turns on the proxy to that Core. Unset: no proxy. Any other value stops the dev server
`VITE_DEMS_CORE_MODE`	`src/runtime/boot.ts`	`live` or `simulated`: the default mode when nothing is stored
`VITE_DEMS_PRODUCTION_LOCK`	`src/runtime/boot.ts`	Exactly `1` turns the production lock on
`VITE_DEMS_CORE_ORIGIN`	`src/lib/core-endpoint.ts`	`same-origin`, for the packaged build only

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TIP If the LIVE console says "Core server unreachable", check two things: the Core is running, and Vite was started with `DEMS_CORE_PROXY_PORT=4420`. Without the variable there's no proxy and the console can't reach anything (README.md:1262, troubleshooting table).

CHAPTER 5

5 Tablet app

The tablet app is the operator's Android app. It lives in `apps/mobile` (package `@dems/mobile`). This chapter covers the runtime, the Core server address setting, pairing, the two APK builds, and how to put a build on a real tablet.

Decision numbers in this chapter come from the audit log unless they're marked "bench".

5.1 Runtime

TABLE 5.1 Tablet app runtime

FACT	VALUE	SOURCE
Android package	`com.dems.demo.tablet`	`apps/mobile/README.md:7` (`app.json`)
App name, version	"DEMS Tablet Demo", 0.1.0	same
Framework	Expo `~57.0.26`, React Native 0.86.3, Expo Router	same
Styling and state	Uniwind; TanStack Query and Zustand; i18next for words	same
Peer	One Core server, over plain `http`/`ws` on port 4420	`apps/mobile/README.md:9`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

One tablet pairs with one Core. The app uses the Core's HTTP JSON API under `/api/v1` and the realtime socket `/ws`. Media playback and live previews go through the Core's signed paths.

5.1.1 Data sources

TABLE 5.2 Data sources by build

SOURCE	DEBUG BUILD	RELEASE BUILD
Live (the Core at the configured address)	yes	yes, the only one (D163)
Simulated (the in-app simulator)	yes	no
Heartbeat (the first demo's mock Core on `127.0.0.1:4400`)	yes	no

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Simulated and Heartbeat stay in debug builds for development links and tests (`apps/mobile/README.md:11-14`).

5.1.2 The runtime folder

`apps/mobile/src/runtime/` is where the app's link to the Core lives. The pieces worth knowing (`apps/mobile/src/runtime/README.md`):

- `core-address.ts` is the single source of the Core origin. The typed client (HTTP), the Live socket (WS), the media transport and the preview transport are all built from it (`:22`).
- `camera-stream-transport.ts` makes the one request of a live camera preview. The stream `GET /preview/<token>` is `multipart/x-mixed-replace` (`:16`).
- `keep-awake.ts` holds the screen awake while a session is PRECHECK, ARMED, RECORDING, STOPPING or FINALIZING, and releases it otherwise (`:35`).

- `owed-sign-out.ts` keeps a sign-out the tablet owes the Core when the link was down, and pays it on link-up.
- `ended-link.ts` treats a socket the Core closed on purpose as "ended", not as an outage: close code 4403 means the device was revoked, 4401 means the session ended (`:31`).

5.1.3 Folder layout

TABLE 5.3 Folders of `apps/mobile/src`

FOLDER	WHAT IT HOLDS
`app/`	Expo Router routes: `_layout.tsx`, `index.tsx`, `pair.tsx`, the `(auth)` group (login) and the `(app)` group (tabs and session screens)
`features/`	One folder per screen area: `alerts`, `audit`, `capture-selector`, `dashboard`, `devices`, `evidence`, `home`, `login`, `more`, `network-time`, `pairing`, `rbac`, `session-finalizing`, `session-live`, `session-new`, `session-recovery`, `session-summary`, `sessions`, `settings`, `storage`, `transcription`
`runtime/`	Data source and wiring (above)
`auth/`	Sign-in provider, operator session, biometric sign-in (off in release)
`navigation/`, `providers/`	Root navigator, tabs, stacks, route guards; app providers, boot gate, toast area
`components/`, `layout/`, `lib/`, `i18n/`, `motion/`	UI kit, pane layout, helpers, words, animation builders

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Outside `src/`: `plugins/` holds three Expo config plugins (`with-dems-core-cleartext.js`, `with-dems-library-patches.js`, `with-dems-release-signing.js`), `patches/` holds one library patch, and `scripts/` holds the build, signing, patch and device helpers (`apps/mobile/README.md:20-34`).

5.2 Core server address

Since D263 the tablet doesn't hard-code where the Core is. The Pair screen has a **Core server address** card with two fields, Host and Port (`apps/mobile/README.md:78-81`; README.md:130-133).

How it works:

1. The address in force starts as `127.0.0.1:4420`. That default reaches the Core through `adb reverse` (`npm run mobile:reverse`), on an emulator or on a tablet connected by USB to the Core host.
2. For a tablet on the interview-room LAN, type the Core host's LAN address (for example `192.168.x.x`) and port `4420`.
3. Tap **Use this address**. The card now shows that address as "In use".
4. Pair stays disabled while the fields hold an address that isn't the one in force. You can't pair against an address you haven't confirmed.

Settings shows the address read-only. The login screen's offline message names it, so an operator can see which Core the tablet is trying to reach.

5.2.1 Validation

TABLE 5.4 Core server address validation

FIELD	ACCEPTED	REFUSED (EACH WITH ITS OWN MESSAGE)

FIELD	ACCEPTED	REFUSED (EACH WITH ITS OWN MESSAGE)
Host	An IPv4 address in a private, loopback or link-local range, or a host name	A public IPv4 address; a scheme (`http://...`) or a path; `host:port` typed into Host; IPv6
Port	1 to 65535	Anything else

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

5.2.2 What a change does

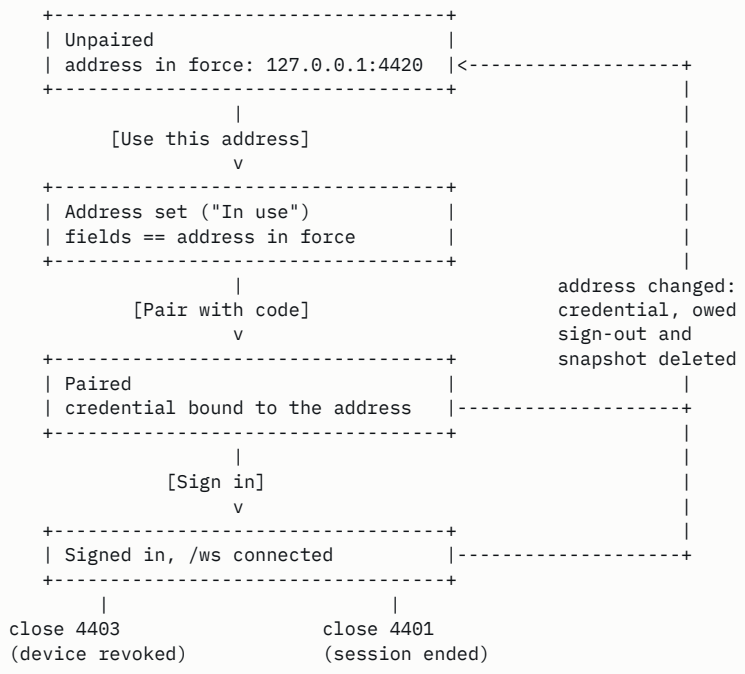
Changing the address first deletes three things:

- the stored device credential;
- any owed sign-out;
- the stored Core snapshot.

A different Core means a new pairing. Why it's like this: the device credential is bound to the address, so the tablet never sends a credential meant for one Core to another one. That covers part of residual R225-F3 (AUDIT §6, Core address feature).

The address is stored in the prefs store `dems.mobile.prefs.v1`, field `coreAddress`. It's not a secret, so it's not in the secure store (`apps/mobile/README.md:81`).

FIGURE 5.1 Tablet address and pairing states



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP **Unknown:** how a tablet on the LAN reaches the Core. The Core's listener is fixed in code to `127.0.0.1` (`LISTEN_HOST`, `apps/core-server/README.md:22`), yet a LAN tablet is told to enter the host's LAN address on port 4420 (`apps/mobile/README.md:78`). No forwarder, proxy or bind change is documented in the repository, and LAN `http`/`ws` from a real device has not been proven (AUDIT §6). The `adb reverse` path over USB is the only documented way a tablet reaches this Core today.

5.3 Pairing

Pairing gives the tablet a device credential for one Core and one database. The steps (README.md:136, 554):

1. In the web console, a Supervisor (permission `PAIRING_MANAGE`) opens **Admin > Paired Devices** (`/admin/devices`) and issues a code.
2. The code is six digits, works once, and is valid for 5 minutes. It's shown once.
3. On the tablet, check the Core server address (5.2), type the code on the Pair screen and tap Pair.
4. The Core stores the device as `DEV-000N` and returns a device credential. The tablet keeps it in its secure store, bound to the address.
5. Sign in on the tablet. `/ws` connects.

If three redeem attempts fail within 300 s, redeeming is locked for 60 s (`apps/core-server/README.md:145`). The code length and lifetime above come from the README; the constants behind them (`PAIRING_CODE_LENGTH`, `PAIRING_CODE_LIFETIME_MS`) weren't read for this manual.

To undo it:

- **On the Core:** revoke the device on the Paired Devices page. Its socket closes with 4403.
- **On the tablet:** Settings → Pairing.

A pairing belongs to one database. A tablet paired with `dems_demo` on port 4420 isn't paired with a lane database (README.md:555).

WARNING A tablet can lose its pairing without being told (residual R225, README.md:176):

- **F1:** a database rollback (`pg_restore`) to a dump taken before the tablet paired;
- **F2:** moving the tablet to another Core, because Cores don't have a unique id per installation yet;
- **F3:** a reset of the pairing tables.

In the one case the audit saw (a credential from another Core), the Core answered 401 and the app deleted the credential (D204, which corrects D202). Either way the tablet has to pair again. Before any database-restoring rollback, list the tablets that paired after the dump (chapter 8).

5.4 Features

The screen areas follow `src/features/` (Table 5.3). What the audit's Mission 3 changed to match the current Core (README.md:166-169; AUDIT §6):

TABLE 5.5 Tablet behaviour after the audit's Mission 3

AREA	BEHAVIOUR NOW
Previews	Four camera previews, each shown with its D5 pair
Paired mic missing	The `PAIRED_MIC_MISSING` notice shows on Devices, in the bell and as a toast
Mic selector	A mic change is a swap with a confirm dialog; the retired op 81 is no longer called
Core identity	Shown on login and on the Pair screen ("Not reported" on a Core that captures for itself)

AREA	BEHAVIOUR NOW
Device commands	Blocked on a Core that captures for itself, instead of pretending to run
Verify and integrity	Show the per-pair audio tracks next to the video tracks
Pair words	Home and the microphone cards name each mic's camera pair
Sign-out	An owed sign-out is kept and paid when the link comes back

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP A tablet build older than the W1-W3 contracts can't talk to this Core. Replace it (AUDIT §6, real-device caveats; D90).

5.5 Debug vs release APK

TABLE 5.6 The two Android builds (`apps/mobile/README.md:61-67`)

	DEBUG APK	RELEASE APK
Build	`npm run build:android`	Once: `apps/mobile/scripts/create-demo-keystore.sh`. Then: `apps/mobile/scripts/build-release-apk.sh`
Output	`apps/mobile/android/app/build/outputs/apk/debug/app-debug.apk`	`release/android/DEMS-Demo-0.1.0.apk` with `.sha256`, `SHA256SUMS.txt` and `BUILD-INFO.txt` (git-ignored)
Signed with	The Android debug key	The DEMO release keystore in the git-ignored `apps/mobile/signing/` ("DEMO ONLY - NOT FOR PRODUCTION")
JavaScript	From Metro (`npm run dev:mobile`)	Compiled Hermes bundle inside the APK
Data sources	Live, Simulated, Heartbeat	Live only (D163)
Biometric sign-in	Available	Off (residual R174-G5)

SUMBER apps/mobile/README.md:61-67; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

5.5.1 Building the debug APK

```
npm run build:android
```

That runs `expo prebuild --platform android --no-install`, then `./gradlew assembleDebug` in `apps/mobile/android`. The audit ran the Gradle step for a single architecture to save time:

```
cd apps/mobile/android
./gradlew --no-daemon assembleDebug -PreactNativeArchitectures=arm64-v8a
```

The debug APK loads its JavaScript from Metro, so start `npm run dev:mobile` and forward the ports with `npm run mobile:reverse` before you open it.

5.5.2 Building the release APK

```
apps/mobile/scripts/create-demo-keystore.sh   # once per machine
apps/mobile/scripts/build-release-apk.sh
```

`create-demo-keystore.sh` writes the DEMO keystore and its properties file into `apps/mobile/signing/` (mode 600). It refuses to run when either file already

exists, generates the password itself, and prints only the certificate's SHA-256 fingerprint and owner name. It never prints the password.

`build-release-apk.sh` checks the signing material and the `app.json` plugin, then runs:

```
npx expo prebuild --platform android --no-install
./gradlew --no-daemon assembleRelease -PreactNativeArchitectures=arm64-v8a,armeabi-v7a,x86_64
```

It holds `.locks/gradle.lock` for both steps, copies the APK to `release/android/` and writes its checksums. It never runs `prebuild --clean` and never installs anything (`apps/mobile/README.md:69`).

HEADS-UP `build-release-apk.sh` was written for the development Mac. `JAVA_HOME` and `ANDROID_HOME` default to that machine's paths, and the lock uses `/usr/bin/lockf`, a macOS tool. On another machine, set both variables before you run it. Run it outside any agent sandbox: prebuild and Gradle fetch from the network.

Things that bite (`apps/mobile/README.md:70-72`):

- Back up the debug APK first. Prebuild has cleared `android/` once even without `--clean`, and that deletes `app-debug.apk`.
- The two builds have different signatures. Installing one build over the other needs an uninstall first, and the uninstall deletes the tablet's pairing.
- Neither is a production build. Both use demo signing.

5.5.3 Library patch

The app is built with a patched `react-native-reanimated` 4.5.1 (`apps/mobile/patches/react-native-reanimated+4.5.1.patch`, one Kotlin file). Without it, on React Native 0.86 the library logged a ~190-frame stack on the UI thread for every animated view that no longer existed, which froze a long-lived app during a recording (`apps/mobile/patches/README.md`).

- The root `postinstall` script (`apps/mobile/scripts/apply-patches.sh`) applies it on `npm install`. It refuses any library version other than exactly 4.5.1.
- The Gradle task `demsCheckLibraryPatches`, added by `plugins/with-dems-library-patches.js`, fails every Android build when the patch is missing.
- After `npm install --ignore-scripts`, run `npm run patches:apply -w @dems/mobile` yourself.

5.5.4 Cleartext to the Core

The app talks plain `http` and `ws` to the Core. Android's network security config can't allow cleartext by address range, so `plugins/with-dems-core-cleartext.js` permits cleartext in the base config, in release builds too (`apps/mobile/README.md:89`; D263, D264). The compensations:

- the app refuses public IPv4 addresses;
- the device credential is bound to the address;
- the interview-room LAN is a closed network.

Traffic between tablet and Core is not encrypted. TLS is the proper fix and is open as residual R263-TLS (chapter 9).

WARNING Proof status of the tablet build (`apps/mobile/README.md:90`; AUDIT §6):

- No real-device test of the release APK against a LAN Core has run. The audit's row 238 proves the configuration only.
- The cycle-2 fixes (commit `d6146ce4`) and the address setting (commit `f41972f9`) are proven by jest, tsc and eslint only. The emulator re-judge never ran on a fresh bundle, so they're unverified on a device.

5.6 Installing on a real tablet

This is the procedure the sources support for the release APK built from `f41972f9` or later. Nothing below has been run end to end on a real tablet yet (5.5, proof status).

1. **Build and check the APK.** Run `build-release-apk.sh` (5.5). Compare the APK's SHA-256 with the `.sha256` file next to it before you copy it anywhere.
2. **Check what's already on the tablet.** A debug build and a release build have different signatures. If a debug build is installed, uninstall it first. That deletes its pairing; plan to pair again.
3. **Leave other tablets alone.** A tablet already paired with production belongs to its operators. Don't uninstall or re-pair it to test a build (AUDIT §6).
4. **Install over ADB.** With the tablet connected and USB debugging allowed:

   ```
   adb install -r release/android/DEMS-Demo-0.1.0.apk
   ```

   `-r` replaces an installed build with the same signature and keeps its data, which is how the debug APK is reinstalled on emulators (README.md:1250). It can't replace a build signed with the other key (step 2).
5. **Set the Core address.** Open the app. On the Pair screen, enter the address (5.2) and tap **Use this address**.
   - Tablet on USB to the Core host: keep `127.0.0.1:4420` and run `adb reverse tcp:4420 tcp:4420` (part of `npm run mobile:reverse`).
   - Tablet on the LAN: enter `192.168.x.x` and `4420`. See the **Unknown** note in 5.2 before you count on this path.
6. **Pair.** Have a Supervisor issue a code on `/admin/devices`, type it on the Pair screen within 5 minutes, and tap Pair (5.3).
7. **Sign in and check.** Sign in as an operator. The login screen should show the Core identity and "paired as DEV-000N". The device should appear on the console's Paired Devices page.
8. **Check the link before a session.** In LIVE mode the Core's pre-check needs a paired tablet connected (README.md:570). If the pre-check blocks on the Core link, the tablet isn't connected.

TIP If the tablet shows "Core server unreachable" on USB, the `tcp:4420` reverse rule is usually missing. Run `npm run mobile:reverse` again (or `ANDROID_SERIAL=<serial> npm run mobile:reverse` with several devices attached).

5.7 Commands and toolchain

TABLE 5.7 Tablet commands (`apps/mobile/README.md:48-57`)

WHAT	COMMAND (REPOSITORY ROOT)
Metro dev server (`expo start --localhost --port 8081`)	`npm run dev:mobile`
Build and run on a device or emulator	`npm run android`
Forward ports 4400, 8081 and 4420 to the device	`npm run mobile:reverse`
Debug APK	`npm run build:android`
Tests	`npm test -w @dems/mobile` (jest)
Typecheck	`npm run typecheck -w @dems/mobile` (`tsc --noEmit`)
Lint	`npm run lint -w @dems/mobile` (eslint)
Patch check	`npm run patches:check -w @dems/mobile`
Expo checks	`npm run doctor -w @dems/mobile`, `npm run deps:check -w @dems/mobile`

SUMBER apps/mobile/README.md:48-57; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The app has no environment configuration for production; everything it needs is set on the tablet. Development-only build variables: `EXPO_PUBLIC_USE_RN_FETCH` (debug probes), `METRO_PORT` (the `start:lane` script), and `JAVA_HOME` / `ANDROID_HOME` (read by `build-release-apk.sh`) (`apps/mobile/README.md:84`).

The toolchain:

TABLE 5.8 Tablet toolchain (README.md:45)

ITEM	VERSION
Node, npm	26.8.2, 11.19.1
JDK	17
Android SDK	platform 36, build-tools 36.0.0, NDK 27.1.12297006
Gradle	the wrapper that `expo prebuild` writes
Device	an emulator, or a USB-connected tablet with `adb`

SUMBER README.md:45; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The app isn't a server and has no systemd unit. On the Core host, all it needs is port 4420 reachable from the tablet's network (`apps/mobile/README.md:88`), which brings you back to the open question in 5.2.

CHAPTER 6

6 Capture worker, LCD backend, transcriber and mock Core

The Core does the deciding. The four parts in this chapter do narrower jobs around it:

TABLE 6.1 The parts covered in this chapter

PART	FOLDER	RUNS AS	JOB
Capture worker, relay, guard	`apps/capture-worker`	Child processes of `dems-core`	Record cameras and microphones to disk
LCD backend	`apps/lcd-backend`	Unit `dems-lcd`	Drive the ESP32 front panel
Transcriber	`apps/transcriber`	A library inside `dems-core`	Speech to text on the MIC-01 audio
Mock Core	`apps/mock-core`	Nothing in production	The first demo's stand-in server

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Decision numbers in this chapter come from the audit log unless they're marked "bench".

6.1 Capture worker

6.1.1 What it is

The capture worker is a C program linked against FFmpeg 9.0.2: `libavformat`, `libavcodec`, `libavfilter`, `libswresample` and `libavutil` (`LIBS_PC` in the `Makefile`). The Core starts and supervises it. It records one fMP4 file per camera, up to four (`VIDEO-01` to `VIDEO-04`, bench D60), each carrying the audio of that camera's paired microphone (`apps/capture-worker/README.md:5-18`).

Current version: `capture-worker/0.8.0` on capture protocol 6. Version history: 0.6.0 spoke protocols 4 and 5; 0.7.0 added the build stamp the Core checks at boot; 0.8.0 moved to protocol 6 for the D5 pairs (D117).

What 0.8.0 does with audio (D5, D117, D132):

- each camera VIDEO-0N carries the audio of its paired mic MIC-0N;
- each mic is also written to its own file `MIC-0N.m4a`, stored as track AUDIO-0N;
- a camera whose paired mic is missing gets a silent audio track (`micMissing` in the plan);
- MIC-01 alone feeds the transcription tap.

6.1.2 How it talks

TABLE 6.2 How the capture worker talks

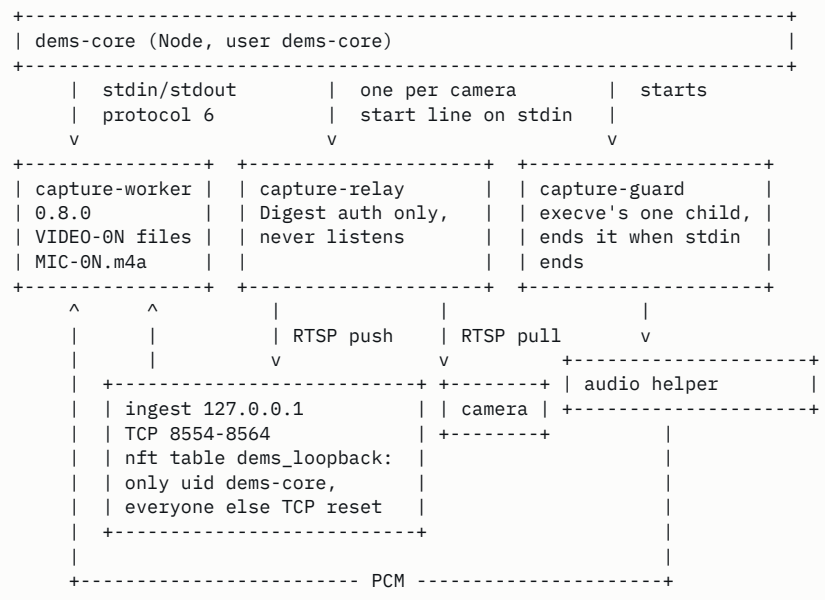
PEER	HOW	SOURCE
Core	Capture protocol 6: one JSON object per line. Commands on the worker's stdin; frames `HELLO`, `HEARTBEAT`, `ACK`, `WARNING`, `RECORD`, `CRASH`, `TELEMETRY` on its stdout, and nothing else ever on stdout. Contract: `packages/capture-protocol`, `CAPTURE_PROTOCOL_VERSION = 6`, the same number as in `src/worker.h`	`apps/capture-worker/README.md:22-28`

PEER	HOW	SOURCE
Cameras	RTSP ingest on `127.0.0.1`, TCP ports 8554-8564. The relay pulls each camera and pushes it here	:29-32
Microphones	PCM inputs from the audio helper, which the Core starts through the guard	:33-34
Log	A JSON-lines file named by `--log`. stderr isn't used	:28

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

A Core and a worker with different protocol numbers refuse each other at `HELLO`. You can't run a mismatched pair by accident.

FIGURE 6.1 Capture process tree on the bench host



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

6.1.3 Source layout and make targets

TABLE 6.3 `apps/capture-worker` folders (`apps/capture-worker/README.md:38-45`)

FOLDER	CONTENT
`src/`	The worker: `capture_worker.c` (entry, plan, threads), `ipc.c` (frames, bounded log), `ingest.c` (RTSP ingest per camera), `audio.c` (one AAC encoder per paired mic), `sink.c` (the fMP4 writer), `anchor.c`, `micwatch.c`, `announce.c`, `worker.h` (version word and protocol number)
`src/relay/`	The capture relay
`src/guard/`	The capture guard
`test/`	Unit checks, the end-to-end harness `m3worker.py`, the series runner `e2e.py`
`test-fixtures/`	Fake camera and mics, analysers, a vendored old worker used as a negative control
`build/`	Every binary (git-ignored)

SUMBER apps/capture-worker/README.md:38-45; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 6.4 Make targets (`apps/capture-worker/README.md:64-78`)

COMMAND	WHAT IT DOES

COMMAND	WHAT IT DOES
`make -C apps/capture-worker`	Static rules, their self-test, unit checks, `build/capture-worker` with its stamp, the relay, the guard (also `npm run test:capture-worker`)
`make -C apps/capture-worker check`	The worker's static rules (`check.sh`)
`make -C apps/capture-worker selftest`	Self-test of the static rules
`make -C apps/capture-worker unit`	Unit checks
`make -C apps/capture-worker relay`	`relay-check`, `relay-unit`, `relay-asan`, `relay-fuzz` and `build/capture-relay`
`make -C apps/capture-worker guard`	`build/capture-guard` and its tests
`make -C apps/capture-worker proof`	Proof builds (negative controls; the Core never starts them)
`make -C apps/capture-worker fixtures`	Fake camera, fake mics, the vendored skeleton
`make -C apps/capture-worker versions`	The FFmpeg libraries this build links
`make -C apps/capture-worker e2e`	The end-to-end series, about 35 minutes (also `npm run test:capture-worker-e2e`)
`make -C apps/capture-worker clean`	Removes `build/`

SUMBER apps/capture-worker/README.md:64-78; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

On the bench host, point `pkg-config` at the FFmpeg build:

```
PKG_CONFIG_PATH=/opt/dems/ffmpeg-9.0.2/lib/pkgconfig make -C apps/capture-worker
```

6.1.4 Configuration

The worker, the relay and the guard read **no environment variables**. The worker gets its settings as command-line arguments and through the Core's start message; the relay reads one start line on stdin. What you configure lives on the Core side, in `/etc/dems/core.env` (`apps/capture-worker/README.md:86-98`):

TABLE 6.5 Core variables for the capture programs

CORE VARIABLE	WHAT IT NAMES
`DEMS_CAPTURE_MODE`	`real` to start the worker at all; `simulated` starts none
`DEMS_CAPTURE_WORKER`	Absolute path of `capture-worker`
`DEMS_CAPTURE_RELAY`	Absolute path of `capture-relay`
`DEMS_CAPTURE_GUARD`	Absolute path of `capture-guard`
`DEMS_CAPTURE_AUDIO_HELPER`	Absolute path of the audio helper the guard starts
`DEMS_MEDIA_DIR`	The media root the worker writes camera files into

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

6.1.5 Build stamp

Every link of `build/capture-worker` writes `build/capture-worker.stamp` beside it: four ASCII lines (`apps/capture-worker/README.md:240`).

```
workerVersion=...
protocolVersion=...
```

```
sourceSha256=...
binarySha256=...
```

`sourceSha256` is the sha256 of the listing `sha256sum $(SRC) $(HDR) Makefile`, run in the worker folder in the Makefile's order. `binarySha256` is the sha256 of the program.

Install the binary and its stamp together: binary 0755, stamp 0644, both owned by root (`apps/capture-worker/README.md:103-107`). At boot the Core checks that the stamp names this binary's sha256, its source, the worker version and the protocol. A mismatch is refused with `CAPTURE_WORKER_BUILD_STAMP_STALE`, `CAPTURE_WORKER_VERSION_MISMATCH` or `CAPTURE_WORKER_PROTOCOL_MISMATCH`.

Why it's like this: at the start of the audit the deployed worker predated the four-camera change and refused every 4+4 plan, while its boot check still passed (D15). The stamp (D44, D48) makes a stale worker a boot failure instead of a failed session.

6.1.6 The e2e series

`make -C apps/capture-worker e2e` builds everything and runs the whole series on fixtures, every check twice and every negative control twice (`apps/capture-worker/README.md:115-153`).

TABLE 6.6 Exit codes of the e2e series

EXIT CODE	MEANING
0	Every verdict is the expected one
1	At least one verdict is wrong, a check crashed, timed out or printed nothing, or a listener was left behind
2	The series couldn't run: the build failed, or no two free ports in 8554-8564
75	The heavy-host lock wasn't free within 30 minutes; nothing ran

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Things to know before you start it:

- It takes about 35 minutes on an idle host. Before each timing-sensitive check it waits while the 1-minute load is above 8 or a `whisper-cli` / `omlx` process is alive.
- It takes `.locks/heavy-host.lock` through `/usr/bin/lockf`. Only a caller that already holds the lock passes `E2E_NO_LOCK=1`.
- It binds two loopback ports and reads `ps` and `lsof`, so it fails inside a sandbox.
- Results land in `.cache/m3worker/e2e/<tag>/`: `summary.txt` (last line `AGGREGATE EXIT CODE n`), `summary.json` and one `<check>.out` per check.
- Options go through `E2E_ARGS`, for example `E2E_ARGS="--list"` or `E2E_ARGS="--only 'av1_|neg_av' --runs 1"`.
- Run it again after any FFmpeg upgrade.

HEADS-UP The gated end-to-end D5 tests can't pass on the bench host. Its loopback rule refuses pushers started as a normal user, and that's by design (D150, D153). D5 is proven by its parts; the first real 4+4 session is still a mandatory acceptance test.

6.1.7 The capture relay

`build/capture-relay` is a separate program in `src/relay/` (`apps/capture-worker/README.md:244-256`). The Core starts one per camera.

- It reads one start line on stdin, pulls the camera over RTSP on one TCP connection, answers a Digest challenge only, and pushes the video unchanged to the worker's loopback ingest.
- Two modes: `record` (push to the ingest) and `probe` (open the camera, report codec, size, rate and parameter sets at the first keyframe, then exit 0 without pushing).
- It writes fixed-shape report lines on stdout: `push_established`, `probe`, `camera_lost`, `camera_reopened`. A line is at most 6,400 bytes and never holds an address, path, account or camera text.
- After a camera loss it keeps its one connection to the worker and reopens the camera after 2, 2, 4, 8, 16 and 30 s, six tries per loss. The new stream must carry the same parameter sets as the announced one, or the relay ends with `STREAM_INVALID`. A second refused Digest answer ends it with `AUTH_REFUSED`; a missing stream path ends it with `LOCATOR_NOT_FOUND`.
- It never listens, accepts, opens a datagram socket or resolves a name. It binds exactly once: its one camera socket, to the source address of the start message and port 0.
- The release build carries `-D_FORTIFY_SOURCE=2 -fstack-protector-strong -fPIE`. `make relay-asan` and `make relay-fuzz` are part of every `make`.

HEADS-UP A camera that restarts with different encoder settings inside a session isn't recorded on after the restart. The parameter sets must come back byte-identical (README.md:1150, Known limit 55).

6.1.8 The capture guard

`build/capture-guard` (`src/guard/guard.c`) exists because the FFmpeg audio helper doesn't end when its stdin ends and survives a SIGKILL of its parent (`apps/capture-worker/README.md:258-270`).

```
capture-guard <absolute path of the program> [its arguments ...]
```

- It starts one child with `execve`: no shell, no search path, arguments and environment unchanged, core-file limit 0.
- It ends the child (SIGTERM, then SIGKILL after 2 s) when its own stdin ends, when its parent changes, or on SIGTERM, and passes the child's exit status on.
- Its own exit codes: 120 usage, 121 core-file limit, 122 stdin is no pipe or socket, 123 not started, 126 not executed.
- On Linux the child also gets the parent-death signal. That path has never been compiled on the development Mac.

6.1.9 Linux status

`make -C apps/capture-worker` builds and passes in the Ubuntu 24.04 container of `scripts/host/container/` (clang 18, glibc 2.39, a self-built FFmpeg 9.0.2) (`apps/capture-worker/README.md:272-279`).

- The sync level is the platform's, and it's stated in every camera file. macOS states `F_FULLFSYNC` (with `FSYNC` as a stated fallback); Linux has no such call, so every camera file states `FSYNC`.
- `make fixtures`, the fixture runs and `make e2e` have not been built or run on Linux.

HEADS-UP On the development Mac the worker links the Homebrew FFmpeg 9.0.2, which is a GPL-3.0-or-later build. The bench build lives in `/opt/dems/ffmpeg-9.0.2`. How a real-capture Core may be distributed with FFmpeg is an open decision (README.md:649; `THIRD_PARTY_NOTICES.md`).

6.1.10 Production notes

- The worker runs as a child of `dems-core`, never as a unit of its own.
- Rebuild on the host only when `apps/capture-worker` changed: `make clean`, then the build command of chapter 8 under the heavy-host lock.
- Worker and Core deploy together. Never run a new worker beside a Core that expects the old version, and roll them back as a pair (D44).

6.2 LCD backend

6.2.1 What it is

The LCD backend drives the ESP32 front panel. It's a single Python 3 file, `apps/lcd-backend/app/dems_backend.py`, using the standard-library HTTP server and `pyserial` 3.5. Pillow is used for logo conversion when it's installed, with a pure-PNG fallback (`apps/lcd-backend/README.md:5, 17`).

TABLE 6.7 LCD backend facts

FACT	VALUE
Listen	`127.0.0.1:8090`, no authentication
Panel link	USB serial at 115200 baud, one JSON line per message, panel protocol v2
Firmware	Not in this repository. **Unknown:** the ESP32 firmware's version and source; the import archive held no firmware, and firmware/hardware compatibility still needs an attended check
Role	Display only. The Core owns the recording truth; the backend's `rec=start` endpoint doesn't start a DEMS recording

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

6.2.2 Routes

From the header of `app/dems_backend.py` (`apps/lcd-backend/app/dems_backend.py:12-20`; `apps/lcd-backend/README.md:15`):

TABLE 6.8 LCD backend routes

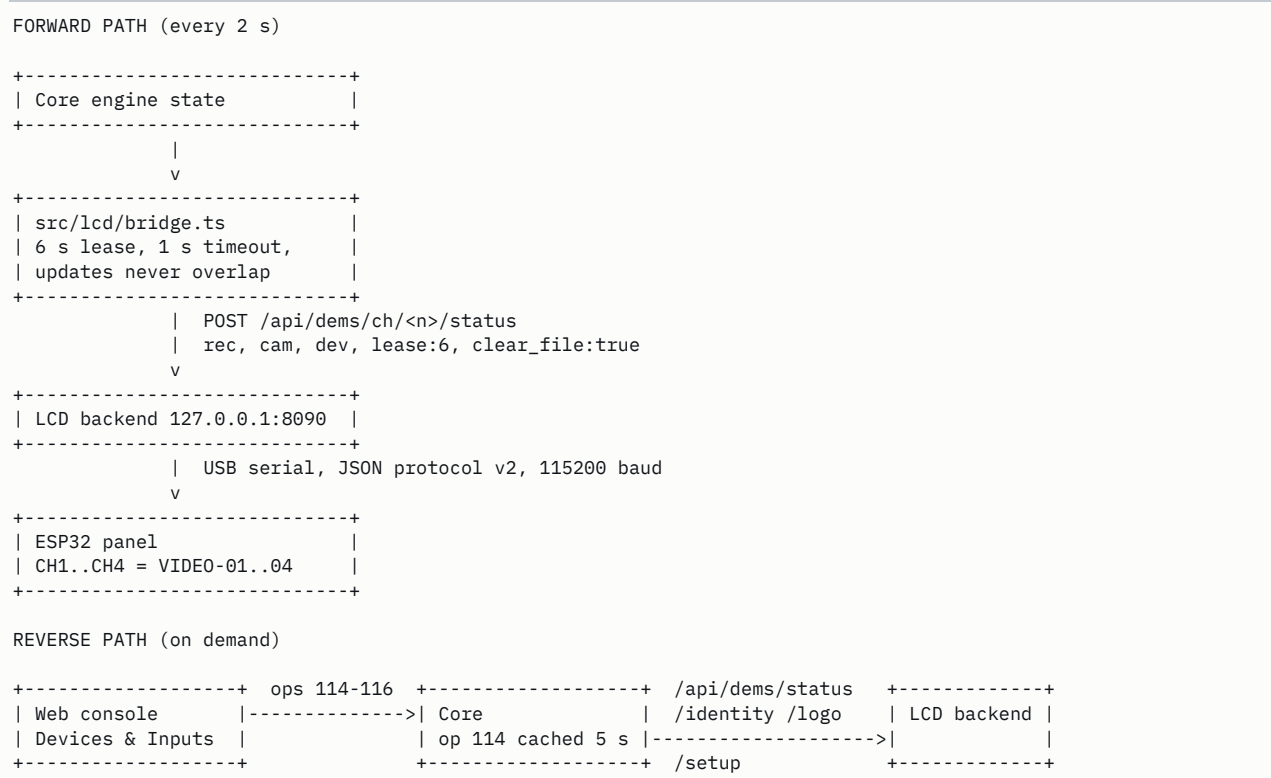
ROUTE	USE
`GET /`	Monitor page
`GET /healthz`	Health
`/api/dems/ch/<id>/status`	Per-channel state; the Core posts here

ROUTE	USE
`/api/dems/status`	Aggregate status; the Core reads it, and it's the smoke-check URL
`/api/dems/log`	Log
`/api/dems/identity`, `/api/dems/logo`, `/api/dems/setup`	Pre-delivery panel setup, proxied by the Core

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

6.2.3 How the Core drives it

FIGURE 6.2 The LCD bridge, both directions



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

- Every 2 s the Core posts each channel's `rec`, `cam`, `dev`, `lease:6`, `clear_file:true` to `DEMS_LCD_BACKEND_URL` (`http://127.0.0.1:8090` on the bench). Each request has a 1 s timeout and updates don't overlap (`apps/lcd-backend/README.md:70`).
- Nothing else goes out: no evidence file name or path, case name, operator identity, camera credential or session token.
- On an orderly shutdown the Core sends STOP on every known channel (`bridge.close()` sends an all-stop with `cam:false`).
- Op 114 reads the panel's firmware word, cached for 5 s. Ops 115 and 116 proxy the panel setup behind the vendor-account gate; the backend checks each write against the Core's session routes under `/api/v1/auth` at `DEMS_CORE_URL`. The Core stores no panel setup itself (`apps/core-server/src/lcd/README.md:7`).

6.2.4 Channel behaviour

TABLE 6.9 Panel channels

PANEL	DEMS SOURCE
CH1	VIDEO-01 (whatever physical camera that track is assigned)
CH2	VIDEO-02
CH3	VIDEO-03
CH4	VIDEO-04

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

CH3 and CH4 carry VIDEO-03 and VIDEO-04 since the Core records four video tracks (bench D60). With `DEMS_CH_MODE=track` (the code default) one track is one channel. With `device` the backend merges tracks of the same physical camera into one channel; the bench unit sets `device` (`apps/lcd-backend/README.md:66`).

Panel words stay in Indonesian, as the firmware shows them:

TABLE 6.10 Panel words

WORD ON THE PANEL	ENGLISH	WHEN
`REKAM`	recording	Only after durable capture, with an assigned online source, a LIVE stream and a REC/DEGRADED track. STOPPING, lost cameras, failed starts and an unavailable Core never show REKAM
`WASPADA`	caution	The backend's health check is not OK; the README names an unreadable media root as one cause (below)
`BACKEND DIAM`	backend silent	Shown in red only when this backend stops or the USB cable is pulled, not when the Core stops

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Timing rules:

- REKAM expires 6 s after the last update, plus up to 1 s for the reaper. A crashed or stopped Core drops REKAM within about 7 s.
- "Responding" needs a status request from the ESP32 within the last 10 s and a serial reply sent. That confirms communication, not what the screen physically shows.
- Since W4 the camera lamp is lit only for a camera stated REACHABLE, or for an open stream when no reach is stated (D234, D246).

WARNING Never publish port 8090. The API has no authentication, and its `rec=start` endpoint moves real relay contacts. Keep `DEMS_BIND=127.0.0.1` (`apps/lcd-backend/README.md:137`).

HEADS-UP If the media root isn't readable by the service user, the health check reports `dir tak terbaca` ("directory unreadable") and the panel stays on `WASPADA` forever. Give `dems-core` read access to the media root (`apps/lcd-backend/README.md:126-127`).

6.2.5 Unit and operations

The installed unit is `deploy/bench-config/systemd/dems-lcd.service` (chapter 8). It runs as user `dems-core` with supplementary group `dialout`, reads `/etc/dems/core.env` and then its own `Environment=` lines, which win. `apps/lcd-backend/dems-lcd.service` is an older copy; don't install it (`apps/lcd-backend/README.md:102`).

- Follow the log with `journalctl -u dems-lcd -f`. Device lines are prefixed `panel:`.
- Stop `dems-lcd` before you flash ESP32 firmware. It's the single holder of the serial port.
- To switch only the Core side off, remove `DEMS_LCD_BACKEND_URL` from `/etc/dems/core.env` and restart `dems-core`. The 6 s lease then releases any channel that was REKAM.
- Run one backend per panel, and configure one Core only to publish to it. Two Cores would overwrite each other's status.

`apps/lcd-backend/compose.yaml` and its `Dockerfile` give an optional Docker bench service. Inside the container it binds `0.0.0.0`, and only host loopback is published. No Docker deployment was performed; the native unit is the bench's form (`apps/lcd-backend/README.md:74-87`).

6.2.6 Tests

```
python3 -B -m unittest discover -s apps/lcd-backend/tests
npm run test -w @dems/core-server -- src/lcd
```

The tests stub the serial import and never open USB. The Core-side test launches the real Python handler, sends recording and stop states and watches the lease expire (`apps/lcd-backend/README.md:91-96`).

6.2.7 Configuration

All settings are environment variables read by `app/dems_backend.py`. On the bench they come from `/etc/dems/core.env` plus the unit's `Environment=` lines (`apps/lcd-backend/README.md:147-163`).

TABLE 6.11 LCD backend variables

VARIABLE	CODE DEFAULT	PURPOSE
`DEMS_BIND`, `DEMS_PORT`	`127.0.0.1`, `8090`	Listen address and port. Keep loopback
`DEMS_DEV`	`/dev/ttyACM0`	Serial device of the panel
`DEMS_USB_SERIAL`, `DEMS_USB_VIDPID`	empty	Find the panel by USB serial number (and vendor:product id) when `DEMS_DEV` is gone
`DEMS_BAUD`, `DEMS_DTR`, `DEMS_RTS`, `DEMS_USB_RETRY_S`	`115200`, `1`, `1`, `2`	Serial line settings and reopen backoff
`DEMS_REKAM_DIR` (falls back to `DEMS_MEDIA_DIR`)		Media root whose free space drives disk health
`DEMS_MIN_FREE_GB`, `DEMS_CRIT_FREE_GB`	`20`, `5`	Free-space warning and critical thresholds
`DEMS_CH_NAMES`	`CH1,CH2,CH3,CH4`	Channel names; the count is the number of names
`DEMS_CH_MODE`	`track`	`track` or `device`

VARIABLE	CODE DEFAULT	PURPOSE
DEMS_LEASE_S, DEMS_CH_STALE_S, DEMS_PANEL_SILENT_S, DEMS_PUSH_S	0, 0, 10, 1.0	Default start lease, stale-channel warning, panel-silent limit, push interval
DEMS_SATKER, DEMS_CLIENT	empty	Identity lines 3 (work unit) and 2 (client name) of the panel header
DEMS_IDENTITY_FILE, DEMS_LOGO_DIR	empty, then /var/lib/dems-lcd	Where identity and logos persist across restarts
DEMS_PROVISION_KEY	empty	Secret that guards writing the client name (header X-Provision-Key). Generate at deploy time; never commit
DEMS_CORE_URL, DEMS_CORE_CONSOLE_ORIGIN	http://127.0.0.1:4420, http://127.0.0.1:5173	Core address for the setup page's sign-in check
DEMS_SETUP_TTL_S	900	Lifetime of a setup page session
DEMS_LOGO_*	see the file	Logo upload to the panel

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

6.3 Transcriber

The transcriber (@dems/transcriber) is the Core's speech-to-text pipeline. It's a **library**, not a server: it opens no port, knows no database and has no HTTP route. The Core imports it from apps/core-server/src/transcription/ and runs it on the MIC-01 audio (apps/transcriber/README.md:7-19).

The pipeline, in order:

1. Audio intake, then voice activity detection with Silero VAD, which cuts the audio into chunks.
2. Speech to text per chunk, through one adapter seam with two backends.
3. The phantom-tail clamp, media time, chunk checkpoints with resume, and a mirror of the Core's transcript_chunks.
4. A diarization post-pass for two speakers.
5. The canonical dems.transcript/1 document with its hash and renderings, written once.

The language is fixed to Indonesian (TRANSCRIBER_LANGUAGE = 'id').

TABLE 6.12 Transcriber backends

BACKEND	HOW IT RUNS
cpu-local	whisper-cli built with the GPU backends off, started through nice at a lowered priority, so capture always outranks STT (src/cpu-priority.ts)
omlx-server	An oMLX server reached over loopback HTTP only (src/backends/omlx-http.ts is the only module that calls fetch)

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

Configuration picks the backend. The transcriber never switches by itself.

6.3.1 Configuration

The transcriber reads no environment variables. The Core hands it two things (apps/transcriber/README.md:58-63):

- **STT selection**, edited by operators through the Core (table `stt_config`, ops 90-92): backend, loopback endpoint (oMLX only), model, and a credential **reference**. The reference points to where the key is kept; the key itself is never in the configuration.
- **Host runtime**, which no operator edits: where the engine programs and model files are, and thread counts. The Core reads it from `<data dir>/transcriber/host.json`. Without that file, `cpu-local` is refused as `CPU_LOCAL_RUNTIME_MISSING`.

The `cpu-local` engine in use is whisper.cpp 1.9.5 with the model `ggml-large-v3-turbo.bin` (README.md:796). In `host.json`, `modelVersions` is a value the installer declares: nothing in a model file states its version, and a model without an entry seals its sessions without a transcript document (README.md:786-795).

A real-capture Core whose STT configuration can't be resolved stops at start with `TRANSCRIBER_CONFIG_UNRESOLVABLE`. Speaker labels are "unavailable" in this build: the running Core composes no speaker-labelling engine.

HEADS-UP The sources give no transcription speed or throughput figure, on purpose (README.md:798). Don't quote one.

6.3.2 Commands

TABLE 6.13 Transcriber commands

WHAT	COMMAND (REPOSITORY ROOT)
Unit tests	`npm test -w @dems/transcriber` (Vitest; needs no engine)
Typecheck	`npm run typecheck -w @dems/transcriber`
Lint	`npm run lint -w @dems/transcriber`
Real engine runs	`npm run test:real:cpu-local -w @dems/transcriber` and its siblings (`vitest.real.config.ts`); one at a time, heavy, not part of `npm test`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

There's no run command, and in production the transcriber has no unit and no port. It runs inside the `dems-core` process (`apps/transcriber/README.md:67`). In development the engines and models live under the git-ignored `.cache/stt/`.

6.4 Mock Core

The mock Core (`@dems/mock-core`) is the stand-in server of the first demo. It isn't the Core server and holds no data (`apps/mock-core/README.md:7-14`).

- Fastify, bound to `127.0.0.1:4400` only. Host and port are constants of the shared contract (`MOCK_CORE_HOST`, `MOCK_CORE_PORT`).
- `GET /api/v1/health` answers in the shared `HealthResponse` form.
- `GET /ws` sends one heartbeat a second (`HEARTBEAT_INTERVAL_MS` = 1000) with an increasing `seq`, the server time and the client count. An `Origin` outside the allowlist gets HTTP 403.
- The only configuration is `LOG_LEVEL` (default `info`).

The console's Simulated mode and the tablet's Simulated and Heartbeat sources use it, as do tests. A tablet reaches it through `adb reverse` of port 4400. It's not deployed on the bench host and has no unit (`apps/mock-core/README.md:51-53`).

TABLE 6.14 Mock Core commands

WHAT	COMMAND
Run with reload	`npm run dev:mock`
Probe the heartbeat	`npm run probe:ws` (exit 0 on valid heartbeats, 1 on timeout or invalid message, 2 on HTTP 403)
Tests	`npm test -w @dems/mock-core`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

CHAPTER 7

7 Development workflow

This chapter gets you from a fresh clone to a running Core, console and tablet on your own machine. It also covers how the tests are split up, the rule that keeps database tests away from real data, and the two branches you must not mix.

Decision numbers in this chapter come from the audit log unless they're marked "bench". "AUDIT §n" points to section n of `docs/AUDIT_REPORT_2026-11-06.md` in the repository.

7.1 Requirements

TABLE 7.1 Development requirements (README.md:37-45)

ITEM	VERSION OR SETTING	NOTES
Node	26.8.2 (`.node-version`; root `engines >=26.8.2 <27`)	The bench host runs the Core with its own Node 26.12.1 under `/opt/dems`
npm	11.19.1 (`packageManager`)	Workspaces; `.npmrc` sets `save-exact=true`
FFmpeg	9.0.2	The capture worker builds and links against it
C toolchain	A C compiler, `make`, `pkg-config`	For `apps/capture-worker`
PostgreSQL	18, port 4432	Development uses the project-local cluster from the npm package `embedded-postgres` (`npm run pg:*`), data in the git-ignored `.data/`
Python	3, with `pyserial` 3.5	For `apps/lcd-backend`
Tablet toolchain	Expo `~57.0.26`, React Native 0.86.3, JDK 17, Android SDK platform 36, build-tools 36.0.0, NDK 27.1.12297006	Chapter 5

SUMBER README.md:37-45; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The npm workspaces are `packages/*` and `apps/*`. All commands below run from the repository root unless a folder is named.

7.2 Setup

The short path to a Core and a LIVE console (README.md:59, 478-482):

```
npm install                      # also applies the mobile library patch (postinstall)
npm run pg:init                  # once: project cluster in .data/postgres, three roles, three credential
files
npm run pg:start                 # after every reboot; nothing starts PostgreSQL at login
npm run db:create -- --database dems_demo
npm run db:migrate -- --database dems_demo
npm run db:seed -- --database dems_demo
npm run dev:server               # Core on http://127.0.0.1:4420 (or: npm run start:server)
DEMS_CORE_PROXY_PORT=4420 VITE_DEMS_CORE_MODE=live npm run dev:web
```

Then open `http://127.0.0.1:5173` (not `localhost`, see chapter 4) and check the Core:

```
curl -s http://127.0.0.1:4420/api/v1/health   # "status":"ok" ... "ready":{"database":true,"engine":true}
```

On `dems_demo` the root README runs migrate, seed and reset under the migration lock: `/usr/bin/lockf -k .locks/db-migrate.lock npm run db:migrate -- --`

database dems_demo. lockf is a macOS tool; on Linux, make sure no other migration or reset runs at the same time.

7.2.1 The pg:* scripts

TABLE 7.2 Project PostgreSQL scripts (README.md:450-475)

COMMAND	WHAT IT DOES
npm run pg:init	Once: creates the cluster in .data/postgres with roles dems_admin (superuser, provisioning only), dems_owner (create, migrate, seed, reset, tests) and dems_app (the running Core), and leaves it running. Refuses (exit 1) if a cluster or any credential file already exists; it never overwrites
npm run pg:start	Starts the cluster; does nothing if it already runs. Refuses if a foreign process holds 4432
npm run pg:stop	pg_ctl -m fast -w stop on .data/postgres only
npm run pg:status	Exit 0: running and every check passed; 3: stopped; 4: no cluster
npm run pg:remove-data	Stops PostgreSQL and deletes .data/ (every database, every credential file). Can't be undone

SUMBER README.md:450-475; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The cluster binds 127.0.0.1:4432, TCP only, with no Unix socket. The listen address, port and socket directory are passed on the pg_ctl command line, so an edit of postgresql.conf can't open another interface (README.md:450-475).

WARNING pg:init writes three credential files into .data/, mode 0600: pg.pwfile, pg.owner.env and pg.env. Nothing asks you to open them. Never print, copy, commit or pass them as a command argument (README.md:436).

7.2.2 Database commands

TABLE 7.3 Database commands

COMMAND	NOTES
db:create	Leaves an existing database as it is
db:migrate	Applies pending migrations; applies nothing the second time
db:seed	Demo seed. Refuses an already seeded database ("Seeding again is a reset"). --factory writes the identity only, with no operator password (chapter 8)
db:reset	Back to the seed state. Refuses while that database's Core runs. Keeps 18 tables, including paired devices, credentials, sessions and capture configuration
db:status	Shows the migration state
db:generate	drizzle-kit generate. A script of the @dems/core-server workspace only (npm run db:generate -w @dems/core-server); the root package.json doesn't have it

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Each command takes exactly one of dems_demo, dems_lane_1, dems_lane_2, dems_lane_3 (README.md:483-496).

7.2.3 Lanes

The repository can run four Cores side by side on one machine. Each has its own port, database and media root (README.md:501-517).

TABLE 7.4 Development instances

INSTANCE	PORT, DATABASE	CAPTURE OPERATION	DEMO CONTROLS	MEDIA ROOT (**DEMS_MEDIA_DIR**)
main	4420, dems_demo	Simulated	on	.data/media/dems_demo/sessions
1	4421, dems_lane_1	Simulated	on	.data/media/dems_lane_1/sessions
2	4422, dems_lane_2	Development capture (DEMS_CAPTURE_MODE=real, DEMS_CAPTURE_DEV_SOURCES=1, generated test sources)	off	.data/media/dems_lane_2/sessions
3	4423, dems_lane_3	Simulated	off	.data/media/dems_lane_3/sessions

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

```
scripts/lane/lane.sh start main     # waits until ready
scripts/lane/lane.sh status         # all four: port, database, capture operation, media root
scripts/lane/lane.sh reset 2        # db:reset under the migration lock, with the right media root
scripts/lane/lane.sh stop main
```

The matching Vite lanes are 5176-5178.

TIP Use `scripts/lane/lane.sh`, not a bare `npm run start:server`. Every database records the media root it was started with. A bare start records the old shared root `.data/media/sessions`, and the next reset then stops with `RESET_STORE_ROOT_CONFLICT`. The script always passes the same environment to `start` and `reset`.

7.2.4 Development capture (lane 2)

Lane 2 records generated test sources through the real worker. It needs a one-time setup per machine (README.md:692-700):

TABLE 7.5 One-time setup of development capture

STEP	COMMAND
A private copy of FFmpeg as the helper of the development sources (`.data/helpers/ffmpeg`); run again after an FFmpeg upgrade	`npm run capture:dev-setup`
Build the capture worker	`make -C apps/capture-worker`
A development signing key in `.data/signer` (certificate says `DEVELOPMENT KEY - NOT FOR PRODUCTION`)	`npm run evidence:dev-key -w @dems/core-server`
The transcriber host file, written by hand	`.data/transcriber/host.json` (chapter 6)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

A `REAL_PRODUCTION` Core refuses to start with a development key. Lane 2's reset seals two demo packages with the development key; one of them has a fragment altered on purpose, so verify has something to catch.

7.3 Typecheck, lint and tests

The root commands run every workspace (README.md:52-58):

```
npm run typecheck    # tsc -b in core-web, tsc --noEmit elsewhere
npm run lint         # oxlint in the TypeScript workspaces, eslint in apps/mobile
npm test             # unit tests: vitest, and jest in apps/mobile
```

One workspace at a time: `npm test -w @dems/core-server` (the Core's `unit` project), `npm test -w @dems/core-web`, `npm test -w @dems/shared`, `npm test -w @dems/mobile`.

7.3.1 What `npm test` does not run

`npm test` is unit suites only. These scripts cover the rest (README.md:1177-1186; `package.json:53-58`):

TABLE 7.6 Test scripts outside `npm test`

SCRIPT	WHAT IT RUNS	NEEDS, AND ABOUT HOW LONG
`npm run test:db`	The Core's `db` Vitest project (`*.db.test.ts`, `test/acceptance/`)	The project PostgreSQL on 4432 (7.4)
`npm run test:seal-e2e`	The built worker records, the Core seals, verify checks (3 tests)	Built worker, `capture:dev-setup`, PostgreSQL; about 1 minute
`npm run test:dev-session-e2e`	A development session on generated sources: start, durable, stop, seal	As above; under half a minute
`npm run test:kill-matrix`	Every active cell of the kill matrix (35)	As above; about 10 minutes
`npm run test:capture-worker`	`make -C apps/capture-worker`: static rules, self-test, C unit tests, build	C toolchain and FFmpeg libraries; under a minute
`npm run test:capture-worker-e2e`	The worker's end-to-end series	Two free loopback ports; about 35 minutes
`npm run test:regression`	Everything, one step after another, stopping at the first failure (16 steps)	Everything above; about 75 minutes. `DEMS_REGRESSION_SKIP_WORKER_E2E=1` skips the worker series

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

None of them opens a camera or a microphone.

HEADS-UP `test:seal-e2e` and `test:dev-session-e2e` (`package.json:55-56`), the worker e2e (`apps/capture-worker/Makefile:298`) and the DB step of `test:regression` (`scripts/test/regression.sh:37`) call `/usr/bin/lockf`, a macOS tool. On Linux these scripts don't run as written.

7.3.2 Test counts

Be careful with numbers here:

- The last fully green regression ran at task 3, commit `713d83f`: 17,494 tests passed (README.md:615). That's history.
- The audit ran targeted gates only. DBW4 (2026-11-06): shared 3013/3013, core-web 1904/1904, core-server 2922 passed plus 9 skipped (AUDIT §5; D244). Rows 236/237 of Mission 3: mobile jest 3523/3523 (AUDIT §6).
- **Unknown:** full-suite counts at `59a683ed`. No full suite ran on this commit; the end-of-mission full gate the audit planned never ran.

7.4 The DB test window

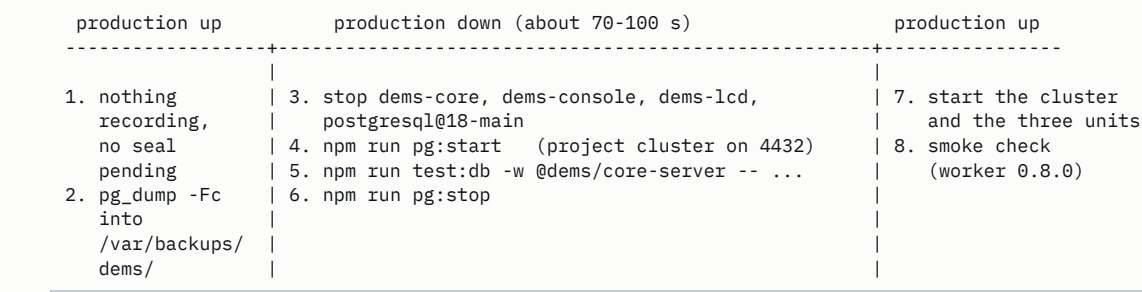
The rule (`apps/core-server/README.md:88`):

- Files named `*.db.test.ts` and everything under `test/acceptance/` run only through `npm run test:db`.
- Its global setup (`test/global-setup.db.ts`) attaches to PostgreSQL on `127.0.0.1:4432`. If nothing answers, it fails; it never skips.
- It creates throw-away databases for the run and drops them at the end.

So run DB tests only against the project-local cluster (`npm run pg:start`), never against a database that holds real data.

On the bench host, port 4432 belongs to the production cluster `postgresql@18-main`. DB tests there run only inside a **DB test window** (D32): production stops, the project cluster takes port 4432 for the length of the test run, then production comes back (README.md:64-69). Figure 7.1 shows the sequence.

FIGURE 7.1 DB test window on the bench host



SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The step-by-step runbook, with commands, is in chapter 8. Measured windows ran about 70 s (DBW3D, D154) and about 96 s (DBW4, D244).

WARNING The window script's own worker check still expects `capture-worker/0.7.0`, but production runs 0.8.0 (D244). Pin the smoke check to 0.8.0 by hand. An older pin gave a false red in DBW4. Fix the script before the next window.

More things to know:

- The heavy scripts take lock files under `.locks/` at the repository root: `heavy-host.lock` (one heavy run at a time) and `db-migrate.lock` (one migration writer at a time) (README.md:68).
- The gated D5 end-to-end tests can't pass on the bench host. The nft loopback rule refuses pushers that don't run as `dems-core`, and running them as `dems-core` is refused by the product's own trust rule (README.md:69; D150, D152, D153).

7.5 Branches

TABLE 7.7 Branches

BRANCH	WHAT IT IS
`task4/input-av-core-port`	The team's branch
`audit/fix-20261105`	The audit's branch: fix waves W1-W4 and Mission 3, on top of the team's state at the audit's start
`main`	Holds the audit branch's history (merged as `8b829aba`), now at `59a683ed`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The team branch and the audit branch are not merged with each other (README.md:185-189; AUDIT §9).

WARNING Don't merge or rebase one onto the other without a decision. When that decision is made, open a pull request between them so the history stays reviewable.

CHAPTER 8

8 Deploy and operations

This chapter is for whoever installs or runs the bench host: one Linux machine that runs the Core, the web console and the LCD front panel from `/opt/dems/core`. It starts with the layout, then gives seven runbooks: fresh install, `core.env`, migrations, the DB test window, a lockstep deploy, smoke checks and rollback.

The unit files and the nft rule in `deploy/bench-config/` are byte-for-byte copies of what was installed on the bench host on 2026-11-06 (`deploy/bench-config/README.md:5`). This chapter quotes `dems-core.service` and the nft rule in full and the other units as excerpts. In the `dems-lcd.service` excerpt the two lines that name the bench's USB serial adapter carry placeholders instead.

Decision numbers in this chapter come from the audit log unless they're marked "bench". "AUDIT §n" points to section n of `docs/AUDIT_REPORT_2026-11-06.md` in the repository.

8.1 Bench host layout

DEMS runs from `/opt/dems/core`, a copy of the repository tree at a deployed commit. It's **not** a git checkout. The audit worked in a separate git worktree and copied files into `/opt/dems/core` (README.md:73).

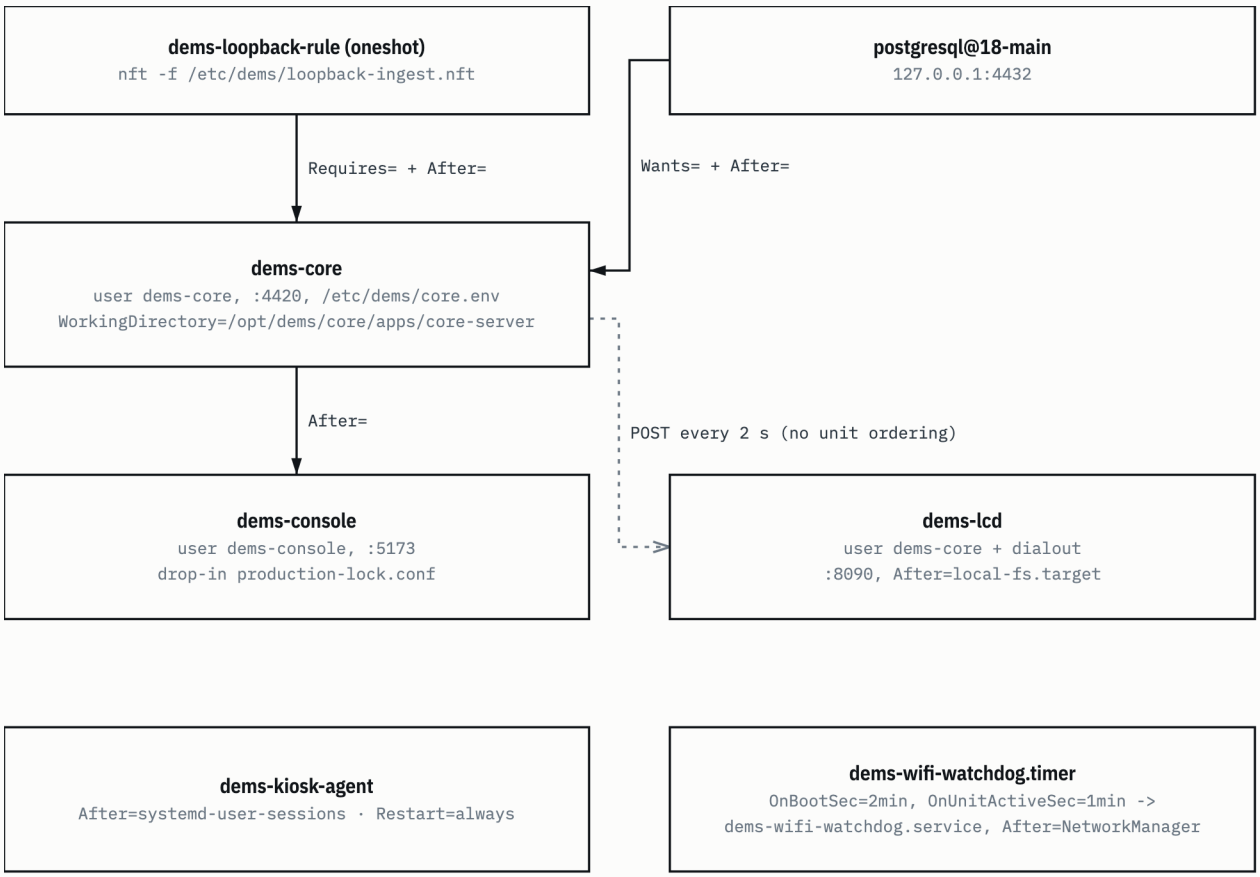
TABLE 8.1 Units on the bench host

UNIT	USER	PORT / ROLE	READS
`dems-core`	`dems-core`	4420; health `GET /api/v1/health`	`/etc/dems/core.env`
`dems-console`	`dems-console`	5173 (Vite server)	Its own `Environment=` lines and the drop-in `production-lock.conf`
`dems-lcd`	`dems-core`, plus group `dialout`	8090; status `GET /api/dems/status`	`/etc/dems/core.env`, then its own `Environment=` lines
`postgresql@18-main`	(PostgreSQL)	4432, database `dems_demo`	System cluster config
`dems-loopback-rule`	root	Oneshot; loads nft table `inet dems_loopback`	`/etc/dems/loopback-ingest.nft`
`dems-kiosk-agent`	root	Root side of the Edge kiosk physical-key lock	Runs `/usr/local/lib/dems/dems-kiosk-agent.py`
`dems-wifi-watchdog.service` + `.timer`	root	Bench Wi-Fi watchdog, every minute, first run 2 min after boot	Runs `/usr/local/sbin/dems-wifi-watchdog`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Figure 8.1 shows how the units order against each other, as their `[Unit]` sections state it.

FIGURE 8.1 Units on the bench host and their ordering.



SUMBER DEMS repository, commit 59a683ed. Redrawn with datum_diagram.py from the box drawing of the source chapter.
PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

Only `dems-core` has a hard dependency (`Requires=dems-loopback-rule.service`): if the rule's unit fails, the Core doesn't start. The loopback rule also states `Before=dems-core.service` from its side. `dems-lcd` has no ordering against `dems-core`; it just receives posts when the Core is up.

8.1.1 dems-core.service

```
[Unit]
Description=DEMS Core
After=network-online.target time-sync.target dems-loopback-rule.service postgresql@18-main.service
Wants=network-online.target postgresql@18-main.service
Requires=dems-loopback-rule.service

[Service]
Type=simple
User=dems-core
Group=dems-core
SupplementaryGroups=audio dems-netalias
WorkingDirectory=/opt/dems/core/apps/core-server
EnvironmentFile=/etc/dems/core.env
ExecStart=/opt/dems/node-26.12.1/bin/node /opt/dems/core/node_modules/tsx/dist/cli.mjs src/index.ts
KillMode=mixed
UMask=0077
LimitCORE=0
NoNewPrivileges=yes
CapabilityBoundingSet=
AmbientCapabilities=
TimeoutStopSec=60

[Install]
WantedBy=multi-user.target
```

The Core runs straight from TypeScript through `tsx`; there's no build step.

8.1.2 dems-console.service

The lines that matter (the rest mirrors the hardening of `dems-core`):

```
User=dems-console
WorkingDirectory=/opt/dems/core/apps/core-web
Environment=DEMS_CORE_PROXY_PORT=4420 VITE_DEMS_CORE_MODE=live HOME=/nonexistent
ExecStart=/opt/dems/node-26.12.1/bin/node /opt/dems/core/node_modules/vite/bin/vite.js --configLoader runner
```

Plus the drop-in `/etc/systemd/system/dems-console.service.d/production-lock.conf`, which sets `Environment=VITE_DEMS_PRODUCTION_LOCK=1` (chapter 4).

Why a dev server in production: a console built with `vite build` is always SIMULATED, and only the dev server can run LIVE (chapter 4).

8.1.3 dems-lcd.service

The installed unit's service lines, with the two hardware values replaced. It also carries `Restart=always`, `RestartSec=3`, `UMask=0077`, `NoNewPrivileges=yes` and `KillMode=mixed`:

```
User=dems-core
Group=dems-core
SupplementaryGroups=dialout
WorkingDirectory=/opt/dems/core/apps/lcd-backend
EnvironmentFile=/etc/dems/core.env
Environment=DEMS_BIND=127.0.0.1
Environment=DEMS_PORT=8090
Environment=DEMS_DEV=/dev/serial/by-id/<your-adapter>
Environment=DEMS_USB_SERIAL=<serial>
Environment=DEMS_CH_MODE=device
Environment=DEMS_IDENTITY_FILE=/var/lib/dems-lcd/identity.json
Environment=DEMS_BAUD=115200
Environment=DEMS_LEASE_S=6
Environment=DEMS_PUSH_S=1.0
ExecStart=/usr/bin/python3 -B /opt/dems/core/apps/lcd-backend/app/dems_backend.py
```

On your machine, find the adapter with `ls /dev/serial/by-id/`, or delete both hardware lines to fall back to `/dev/ttyACM0` (`deploy/bench-config/README.md:24`). The `by-id` path is used because the `ttyACM` number moves when the cable is re-plugged.

8.1.4 The nft loopback rule

`deploy/bench-config/nftables/loopback-ingest.nft`, installed as `/etc/dems/loopback-ingest.nft`:

```
table inet dems_loopback
delete table inet dems_loopback
table inet dems_loopback {
    chain output {
        type filter hook output priority filter; policy accept;
        oifname "lo" ip daddr 127.0.0.1 tcp dport 8554-8564 meta skuid != "dems-core" counter reject with tcp
reset
    }
}
```

Only the user `dems-core` may connect to the capture ingest ports 8554-8564 on loopback; everyone else gets a TCP reset. The first two lines make reloading idempotent. `dems-loopback-rule.service` loads it with `nft -f` and deletes the table on stop.

8.1.5 What's not in the repository

The repository doesn't hold (`deploy/bench-config/README.md:22`):

- the kiosk agent program `/usr/local/lib/dems/dems-kiosk-agent.py`;
- the Wi-Fi watchdog program `/usr/local/sbin/dems-wifi-watchdog`;
- the Node runtime under `/opt/dems/node-26.12.1`;
- the FFmpeg build under `/opt/dems/ffmpeg-9.0.2`, and the worker binaries;
- the PostgreSQL 18 server.

Unknown: what the kiosk agent and the Wi-Fi watchdog do beyond their unit descriptions. The kiosk agent is "the root side of the Edge kiosk physical-key lock". The watchdog's description says it reloads the `rtw88` Wi-Fi driver or reboots when the gateway is lost. Neither program's source is in the repository.

8.2 Prerequisites

From `deploy/bench-config/README.md:26-31`:

TABLE 8.2 Host prerequisites

WHAT	DETAIL
Users and groups	`dems-core` (group `dems-core`), `dems-console` (group `dems-console`), and the group `dems-netalias`
Tree	The repository at the deployed commit in `/opt/dems/core`, with its `node_modules`
Programs	Node at `/opt/dems/node-26.12.1/bin/node` (the path the units use), `python3`, `nft`
Database	PostgreSQL 18 as `postgresql@18-main`, on `127.0.0.1` port 4432. The Core refuses port 5432 (`FOREIGN_DATABASE_PORT`)
Capture	FFmpeg 9.0.2 at `/opt/dems/ffmpeg-9.0.2`; the built worker, relay, guard and audio helper at absolute paths you'll name in `core.env`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP The bench README lists the supplementary groups of `dems-core` as `audio`, `dems-netalias` and `dialout`. The installed `dems-core.service` sets only `audio dems-netalias`. `dialout` comes from `dems-lcd.service`, which adds it for that unit alone. **Unknown:** whether the `dems-core` account itself is also meant to be in `dialout`; a comment in `dems-lcd.service` says it isn't and doesn't need to be.

The repository gives no commands for creating the users or for putting the tree and its `node_modules` in place the first time. Use your distribution's tools.

8.3 Runbook: fresh install

Run as an administrator on the target host. Step 1 is the bench README's "Prerequisites on the host". Steps 2-4 and 6-8 follow its seven-step guide (guide steps 1-2, 3, 4, 5, 6 and 7, in that order). Step 5, the signing key, isn't in that guide; it comes from the root README, and 8.3.3 gives the installer's order for it.

1. **Prerequisites.** Everything in 8.2 is in place.
2. **Environment file.** Create and fill `/etc/dems/core.env` (runbook 8.4).
3. **Database roles, database and connection files.** See 8.3.1.
4. **Migrations and seed.** Run 8.5.1, then seed with the factory seed and give every operator a password (8.3.2).
5. **Production signing key.** See 8.3.3.
6. **Units and the nft rule.** See 8.3.4.

7. **Verify.** Run the smoke checks (8.8).
8. **Lock the secrets down.** See 8.3.5.

8.3.1 Roles, database and connection files

Role names come from `scripts/pg/lib.mjs` and `apps/core-server/src/db/schema/boot-checks.ts`:

TABLE 8.3 Database roles and database

WHAT	NAME	ATTRIBUTES
Owner role (runs migrations, owns the database)	`dems_owner`	`LOGIN NOSUPERUSER CREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS`
Runtime role (the running Core)	`dems_app`	`LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS`
Database for port 4420	`dems_demo`	Owner `dems_owner`, `TEMPLATE template0 ENCODING 'UTF8' LOCALE 'C'`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Generate two passwords the way the project's own provisioning does (32 random bytes as base64url, at least 24 characters):

```
openssl rand -base64 32 | tr '+/' '-_' | tr -d '='    # run twice: one for dems_owner, one for dems_app
```

Create the roles on the system cluster. `\password` prompts for the value, so it never lands in shell history or the process list:

```
sudo -u postgres psql -p 4432 -c "CREATE ROLE dems_owner LOGIN NOSUPERUSER CREATEDB NOCREATEROLE NOREPLICATION
NOBYPASSRLS"
sudo -u postgres psql -p 4432 -c "CREATE ROLE dems_app LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION
NOBYPASSRLS"
sudo -u postgres psql -p 4432 -c "\password dems_owner"
sudo -u postgres psql -p 4432 -c "\password dems_app"
```

Authentication on `127.0.0.1` must be `scram-sha-256`.

Write the two connection files into the data directory, `<repository root>/.data`, which is `/opt/dems/core/.data` on the bench. The data directory itself is mode 0700. Create each file with `install -m 0600` first and edit it afterwards, so it never exists with a wider mode.

`/opt/dems/core/.data/pg.env`, read by the Core; owner `dems-core`, mode 0600. The Core refuses the file if any group or other bit is set:

```
DEMS_PG_HOST=127.0.0.1
DEMS_PG_PORT=4432
DEMS_PG_DATABASE=dems_demo
DEMS_PG_APP_ROLE=dems_app
DEMS_PG_APP_PASSWORD=<the dems_app password>
```

`/opt/dems/core/.data/pg.owner.env`, read by the `db:*` commands; mode 0600, owned by the account that runs those commands:

```
DEMS_PG_HOST=127.0.0.1
DEMS_PG_PORT=4432
DEMS_PG_DATABASE=dems_demo
DEMS_PG_OWNER_ROLE=dems_owner
DEMS_PG_OWNER_PASSWORD=<the dems_owner password>
```

Then create the database (as `dems_owner`, through `pg.owner.env`):

```
cd /opt/dems/core
npm run db:create -w @dems/core-server -- --database dems_demo
```

8.3.2 Seed and operator passwords

After the migrations (8.5.1), seed a production unit with the **factory** seed. It writes no operator password. It reads `DEMS_INSTALLATION_DEVICE_ID`, `DEMS_INSTALLATION_NAME` and `DEMS_DEVICE_CERT_FINGERPRINT` from its environment (`deploy/bench-config/README.md:150`):

```
npm run db:seed -w @dems/core-server -- --database dems_demo --factory
```

Then give each operator a password. The command needs a terminal and never takes the password as an argument:

```
npm run operator:set-password -w @dems/core-server -- --database dems_demo --data-dir /opt/dems/core/.data --
username <username>
```

WARNING `REAL_PRODUCTION` refuses to boot while any operator still has the published demo password, and also when that count can't be read (boot check 4c, `CAPTURE_DEMO_OPERATOR_PASSWORD_IN_PRODUCTION`; `DEMO_PASSWORD_IN_PRODUCTION` in `checks.ts`). That's why a production unit uses the factory seed and `operator:set-password`, never the demo seed. Running `db:seed` (demo) or `db:reset` afterwards writes the demo password again.

8.3.3 Production signing key

A `REAL_PRODUCTION` Core refuses to start without a usable signing key in `<data directory>/signer` (boot check 4). The key is made by:

```
npm run evidence:production-key -w @dems/core-server -- --data-dir /opt/dems/core/.data
```

What it does (README.md:960-965):

- makes a software Ed25519 key with a self-signed certificate valid 1,825 days;
- asks for the organisation and the installation device id at prompts, not as arguments;
- asks no secret;
- refuses a store that already holds anything (`KEY_EXISTS`) and a reserved installation id (`INSTALLATION_ID_RESERVED`).

Run it and `operator:set-password` from the repository root, as the user the Core runs as (never as root), with the Core stopped.

The installation id you type must match `DEMS_INSTALLATION_DEVICE_ID` in `core.env`, or boot check 4b-2 refuses with `CAPTURE_SIGNING_KEY_INSTALLATION_MISMATCH`. Never use `DEMS-TEST-01`, `DEMS-IR-01` or `DEMS-JKT-07`; they're reserved for tests and documentation (check 4b-3).

The bench guide's seven steps never call `evidence:production-key`. The order of a first production boot, per the Docker installer notes (installer unverified on the bench), is the following. `deploy/docker/README.md`, section "First production boot", maps the steps of the installer note `scripts/host/INSTALLER-NOTE-FIRST-PRODUCTION-BOOT.md` (README.md:956) like this:

1. Installer step 1: the host profile.
2. Installer steps 2 and 3: the database.
3. Installer steps 5 and 6: a first boot in SIMULATED operation. Unassign the four camera slots in the console, and switch transcription off unless `.data/transcriber/host.json` exists. Then stop the Core and the console.

4. Installer step 7: `operator:set-password` first, then `evidence:production-key`, both with the Core stopped (the root README calls them X-2 and X-1).
5. Installer step 8: set `DEMS_CAPTURE_MODE=real` and start the Core in `REAL_PRODUCTION`.
6. Installer steps 9 to 12: the helper, cameras, microphones, sessions.

The Docker README maps no installer step 4. The runbook above keeps the same order for passwords and key (its steps 4 and 5) but, like the bench guide, has no SIMULATED first boot. Either way the key has to exist before `dems-core` starts in `REAL_PRODUCTION`.

WARNING There's no key escrow for the session keys in `<data directory>/credential-store`. Lose that folder and every sealed recording is unreadable. The signing key has the opposite risk: a copy of it can sign evidence elsewhere. Don't put the signer key files into backups, images or support bundles (README.md:984-991; chapter 9).

8.3.4 Units and the nft rule

```
cd /opt/dems/core/deploy/bench-config
sudo install -o root -g root -m 0644 nftables/loopback-ingest.nft /etc/dems/loopback-ingest.nft
sudo nft -c -f /etc/dems/loopback-ingest.nft          # syntax check only
sudo install -o root -g root -m 0644 systemd/*.service systemd/*.timer /etc/systemd/system/
sudo install -d -o root -g root -m 0755 /etc/systemd/system/dems-console.service.d
sudo install -o root -g root -m 0644 systemd/dems-console.service.d/production-lock.conf
/etc/systemd/system/dems-console.service.d/
sudo systemctl daemon-reload
sudo systemctl enable --now dems-loopback-rule.service
sudo systemctl enable --now dems-core.service dems-console.service dems-lcd.service
sudo systemctl enable --now dems-kiosk-agent.service dems-wifi-watchdog.timer   # bench extras, only where
their programs are installed
```

Between the `install` lines and the `systemctl` lines, edit the two hardware lines in `/etc/systemd/system/dems-lcd.service` for your adapter (8.1), so the deployed tree stays a clean copy of the commit. You don't load the nft rule by hand; `dems-loopback-rule.service` does it.

8.3.5 Keep the secrets private

From `deploy/bench-config/README.md:193-198`:

- Never commit `/etc/dems/core.env`, `pg.env` or `pg.owner.env`, filled or partly filled, to any repository. This one is public.
- Keep `core.env` at root:dems-core 0640 and both connection files at 0600.
- Share a filled file only privately, for example with `scp` over SSH directly between team machines. Never by e-mail, chat, an issue, a pull request or a paste service.
- Backup copies (`core.env.bak-*`) carry the same secrets and the same rules.

8.4 Runbook: core.env from the template

The template `deploy/bench-config/core.env.example` was built from the code only (`apps/core-server/src/config.ts`, `apps/core-server/src/plugins/config/load-config.ts`, `apps/lcd-backend/app/dems_backend.py`). It holds placeholders `<set-me>` and the code's own defaults. Both `dems-core` and `dems-lcd` read the filled file.

Step 1. Install the template.

```
sudo install -d -o root -g root -m 0755 /etc/dems
sudo install -o root -g dems-core -m 0640 deploy/bench-config/core.env.example /etc/dems/core.env
```

Why root:dems-core 0640: both units that read it run as group `dems-core`, and systemd reads the file as root before dropping privileges. Nobody else needs it.

Step 2. Fill in every variable with `sudoedit /etc/dems/core.env`. Replace every `<set-me>`. For production, `DEMS_CAPTURE_MODE=real`, and `DEMS_DEMO_CONTROLS` and `DEMS_CAPTURE_DEV_SOURCES` stay `0`.

What each group needs (`deploy/bench-config/README.md:50-78`):

TABLE 8.4 core.env variables by group

GROUP	VARIABLE	RULE
Ports	`DEMS_CORE_PORT`	4420 (4421-4423 are lanes). Any other value stops the boot. The host is fixed to 127.0.0.1 in code
	`DEMS_LCD_BACKEND_URL`	Must be `http://127.0.0.1:<port>`; never a remote host
	`DEMS_LOG_LEVEL`	`fatal`, `error`, `warn`, `info`, `debug`, `trace` or `silent`; default `info`
Capture	`DEMS_CAPTURE_MODE`	`real` for production. `simulated` is the default and starts no worker
	`DEMS_CAPTURE_DEV_SOURCES`, `DEMS_DEMO_CONTROLS`	`0` in production
	`DEMS_CAMERA_AUTOATTACH`	`0`, or `1` together with `DEMS_CAMERA_SEGMENT_CIDR`
	`DEMS_CAMERA_SEGMENT_CIDR`	One IPv4 `/24` literal, for example `192.168.x.0/24`. The boot stops if it's missing while the sweep is on
Paths	`DEMS_CAPTURE_WORKER`, `DEMS_CAPTURE_RELAY`, `DEMS_CAPTURE_GUARD`, `DEMS_CAPTURE_AUDIO_HELPER`	Absolute paths of the installed production programs. `parseCapturePath` stops the boot otherwise
	`DEMS_CAPTURE_ALIAS_SOCKET`	Absolute path of the address helper's socket. Unset means helper "not configured"
	`DEMS_MEDIA_DIR`	The media root. Writable by `dems-core`, owner-only (0700); the LCD backend also reads its free space
Identity	`DEMS_INSTALLATION_DEVICE_ID`	A capital letter, then capitals, digits or hyphens, at most 32 characters (for example `DEMS-SITE-21`). Not a reserved id
	`DEMS_INSTALLATION_NAME`	1 to 120 printable characters, no leading or trailing space
	`DEMS_SATKER`	The work unit on line 3 of the LCD panel
Secrets	`DEMS_CAMERA_DEFAULT_USER` + `DEMS_CAMERA_DEFAULT_PASSWORD`	Optional pair, read only to test and store an auto-adopted camera. The password is the one the camera already has; copy it from the camera's own setup, don't generate it. 1 to 256 printable characters
	`DEMS_PROVISION_KEY`	Optional, LCD backend: guards writing the client name (`X-Provision-Key`). Any non-empty string; generate one with `openssl rand -hex 32`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

What's deliberately **not** in `core.env` (`core.env.example:73-79`):

- No session secret: the cookie name is derived from the port (`dems_session_<port>`).
- No signer-key path: the signing and session keys live in the Core's key store.

- No database password: it's in `pg.env` (8.3.1). Any `PG*` variable in the environment is removed at start.
- `NODE_OPTIONS` is only reported in the boot log. Don't set it.

The web console's variables (`DEMS_CORE_PROXY_PORT`, `VITE_DEMS_CORE_MODE`, `VITE_DEMS_PRODUCTION_LOCK`, `VITE_DEMS_CORE_ORIGIN`) aren't read from `core.env` either; they're set in `dems-console.service` and its drop-in.

HEADS-UP The LCD backend's own `DEMS_PORT` and `DEMS_BIND` are set in `dems-lcd.service`. The unit's `Environment=` lines win over `core.env`, so changing them in `core.env` does nothing.

Step 3. Check before you start anything.

TIP Two quick checks catch most mistakes: `sudo grep -n '<set-me' /etc/dems/core.env` should print nothing, and `sudo stat -c '%U:%G %a' /etc/dems/core.env` should print `root:dems-core 640`.

8.5 Runbook: migrations

The migrations are SQL files from `0000_prelude` to `0044_recording_fragments_audio_tracks` in `apps/core-server/src/db/migrations/`, applied in journal order. Each table migration is followed by a `_privileges` migration that grants the runtime role what it needs. The bench ledger holds 45 entries since the W3 deploy (D156).

8.5.1 On a fresh install

```
cd /opt/dems/core
npm run db:migrate -w @dems/core-server -- --database dems_demo
npm run db:status  -w @dems/core-server -- --database dems_demo
```

The documented form takes the migration lock: `/usr/bin/lockf -k .locks/db-migrate.lock npm run db:migrate -w @dems/core-server -- --database dems_demo`. `lockf` is a macOS tool; on Linux, make sure no reset or other migration runs at the same time (`deploy/bench-config/README.md:148`).

8.5.2 In a deploy

Only when the commit to deploy adds a migration (README.md:93):

1. **Dry run.** Take a fresh dump. Restore it into a scratch database. Apply the migration there with the project's own migration code, under the migration lock, as the owner role. Compare probes before and after. Drop the scratch database.
2. **Backup.** Take a `pg_dump -Fc` of `dems_demo` into `/var/backups/dems/` (root, mode 0600) as part of the deploy backup (8.7, step 4).
3. **Apply** the migration during the install step, with the units stopped (8.7, step 5).
4. **Check** the ledger with `db:status` in the smoke check (8.8).

The repository doesn't record the exact commands of the dry run or the probe queries the audit compared.

HEADS-UP On the W1 deploy the host had no route to run migrations (`DEPLOY_MIGRATE_ROUTE_MISSING`; production untouched). The audit used a reviewed throwaway harness instead: the local socket as `postgres`, `SET ROLE dems_owner`, the project's own `applyMigrations`, tried on a scratch database first (D72).

8.5.3 Rules

- Never edit an applied migration. Repairs go forward as new migrations (D22; `src/db/migrations/README.md:81`).
- Never delete a migration file while the ledger lists it (README.md:97).
- Rolling back a migration means restoring the pre-deploy dump (README.md:97). **Unknown:** any other way to revert one; the sources mention no down migrations.

8.6 Runbook: DB test window

DB tests need port 4432, which on the bench host belongs to production. So they run only inside an announced window, with production stopped (D32). Downtime is about 70-100 s (README.md:99-108). Chapter 7 explains why.

Run it from a git working tree of the repository that has a project cluster in its `.data/`, not from `/opt/dems/core`.

1. **Check that it's safe.** No session is recording and no seal is pending.
2. **Back up.** Take a fresh `pg_dump -Fc` of `dems_demo` into `/var/backups/dems/` (root, mode 0600). The exact invocation the audit used isn't recorded; as a sketch, run as a role that can read every table:
   ```
   pg_dump -Fc -p 4432 -d dems_demo -f /var/backups/dems/<label>-<timestamp>.dump
   ```
3. **Stop production.** Port 4432 is free afterwards.
   ```
   sudo systemctl stop dems-core dems-console dems-lcd postgresql@18-main
   ```
4. **Start the project cluster** from the working tree:
   ```
   npm run pg:start
   ```
5. **Run the listed test files:**
   ```
   npm run test:db -w @dems/core-server -- <listed test files> --maxWorkers=2
   ```
6. **Stop the project cluster:**
   ```
   npm run pg:stop
   ```
7. **Start production:**
   ```
   sudo systemctl start postgresql@18-main
   sudo systemctl start dems-core dems-console dems-lcd
   ```
8. **Smoke check** (8.8): health 200, console and LCD 200, one `CAPTURE_BOOT_CHECKS_PASSED` with `REAL_PRODUCTION`, ledger unchanged. Pin the expected worker version to `capture-worker/0.8.0`. Restore the dump only if the smoke check fails.

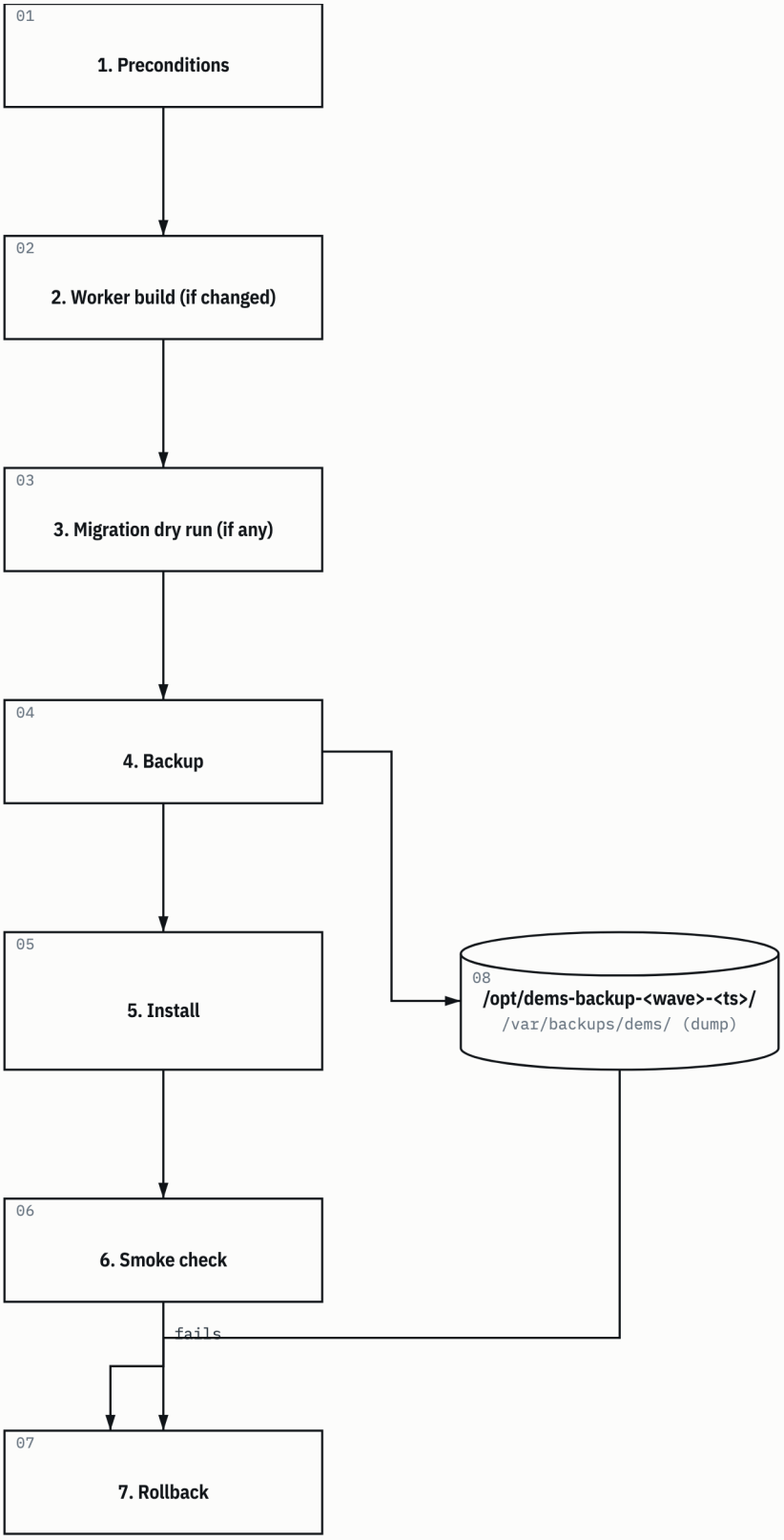
WARNING The window script's own worker check still expects 0.7.0 (D244). Production runs 0.8.0, so that check gives a false red. Fix the script, or check the version by hand, before the next window.

HEADS-UP A window restarts the Core, which ends every sign-in, web and tablet. Announce it to the operators.

8.7 Runbook: lockstep deploy

This is how the audit deployed each fix wave: one lockstep copy of the changed files of one commit (README.md:87-97). Each wave's deploy took between about 1.2 s and 9.2 s of downtime (AUDIT §4). Figure 8.2 shows the flow.

FIGURE 8.2 Lockstep deploy, with the backup feeding the rollback.



SUMBER DEMS repository, commit 59a683ed. Redrawn with datum_diagram.py from the box drawing of the source chapter. Arrows without a label show the order of steps, as in the source drawing. **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Step 1. Preconditions.

- The worktree is clean at the commit to deploy.

- No session is recording, no seal is pending, and no capture process is running.
- List the changed paths:

  ```
  git diff --name-status --no-renames <deployed commit> <new commit>
  ```

- Check that every modified path in `/opt/dems/core` still has the sha256 of the deployed commit's copy, and that every added path is absent. If not, someone changed the install by hand; stop and find out.

Step 2. Worker build, only when `apps/capture-worker` changed. One fresh build of the committed tree, under the heavy-host lock:

```
cd apps/capture-worker
make clean
PKG_CONFIG_PATH=/opt/dems/ffmpeg-9.0.2/lib/pkgconfig make check selftest unit build/capture-worker relay-check
```

Check that `build/capture-worker.stamp` names the worker version, the protocol, the source hash and the binary hash (chapter 6).

Step 3. Migration dry run, only when a migration is in the diff (8.5.2).

Step 4. Backup.

- Copy every file that will be replaced, and the worker pair, with `cp -a --parents` into `/opt/dems-backup-<wave>-<timestamp>/`.
- Write a `SHA256SUMS` manifest, the list of added paths, and the owners before.
- When a migration runs, also take a `pg_dump -Fc` of `dems_demo` into `/var/backups/dems/` (root, mode 0600).

Step 5. Install.

1. Stop `dems-core` and `dems-console`. Stop `dems-lcd` too if its files change. PostgreSQL stays up.
2. For each file, write the commit's copy from `git show <commit>:<path>` with `install -o <owner> -g <group> -m 0644` to a `.<name>.new` file next to the target. Then `chmod g-w,o-w`, then `mv -f` over the old file. The worker binary and stamp go in as 0755 and 0644, root-owned.
3. Check that every installed file has the sha256 of the commit's copy.
4. Apply the migration, if there is one.
5. Start the units.

The audit's install script isn't in the repository. As a sketch, one file:

```
git show <commit>:<path> > "$TMPDIR/<name>"
sudo install -o <owner> -g <group> -m 0644 "$TMPDIR/<name>" /opt/dems/core/<dir>/.<name>.new
sudo chmod g-w,o-w /opt/dems/core/<dir>/.<name>.new
sudo mv -f /opt/dems/core/<dir>/.<name>.new /opt/dems/core/<path>
```

Step 6. Smoke check (8.8).

Step 7. Rollback if the smoke check fails (8.9).

WARNING Worker and Core files deploy together, and a rollback is a pair rollback. Never run a new worker beside a Core that expects the old version (D44). The Core would refuse it at boot anyway (stamp, version and protocol checks), and you'd be down until you fix it.

8.8 Runbook: smoke checks

Run after every install, deploy, window or rollback. All of these must hold (README.md:96; `deploy/bench-config/README.md:179-189`):

TABLE 8.5 Smoke checks

CHECK	EXPECTED
Units	`dems-core`, `dems-console`, `dems-lcd`, `postgresql@18-main` active, with no restarts since the start
Boot line	Exactly one `CAPTURE_BOOT_CHECKS_PASSED` with `"operation":"REAL_PRODUCTION"`, `"checks":"real"` and the expected worker version (`capture-worker/0.8.0`)
Refusals	No stamp, version or protocol refusal; no log line at level 50 or 60
HTTP	200 from `4420/api/v1/health`, from `5173/` and from `8090/api/dems/status`
Health body	`"status":"ok"` and `"ready":{"database":true,"engine":true}`
Ledger	As expected (45 entries at `59a683ed`)
nft	Table `inet dems_loopback` present

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

```
systemctl is-active dems-core dems-console dems-lcd postgresql@18-main
systemctl show -p NRestarts dems-core dems-console dems-lcd
journalctl -u dems-core -b --no-pager | grep CAPTURE_BOOT_CHECKS_PASSED
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:4420/api/v1/health     # expect 200
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:5173/                  # expect 200
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8090/api/dems/status    # expect 200
curl -s http://127.0.0.1:4420/api/v1/health                                      # read the body
sudo nft list table inet dems_loopback
cd /opt/dems/core && npm run db:status -w @dems/core-server -- --database dems_demo
```

HEADS-UP Health answers 200 as long as the process answers. Readiness is in the body, under `ready` (`src/modules/health/README.md:7`). Read the body, not just the status code. **Unknown:** what other fields the body carries; this manual relies only on `status`, `ready.database` and `ready.engine`.

A start without the `CAPTURE_BOOT_CHECKS_PASSED` line is not a healthy production start, whatever the curls say. If the Core stopped during its database checks, the journal has one line `"the server stopped: boot"` with a `"check"` naming the step (B-1 to B-9) and nothing listens (chapter 3).

8.9 Runbook: rollback

Not needed so far (README.md:97).

1. Stop the units.
2. Copy the files back from `/opt/dems-backup-<wave>-<timestamp>/`. The backup keeps the full paths because it was made with `cp -a --parents`. Check every restored file against the backup's `SHA256SUMS`.
3. Delete the paths the deploy added (the backup holds their list).
4. **Only if a migration ran:** restore the pre-deploy dump from `/var/backups/dems/`. Never delete a migration file while the ledger lists it. The exact `pg_restore` invocation the audit planned isn't recorded in the repository.
5. Start the units.
6. Run the smoke check again (8.8).

WARNING A rollback that restores the database silently unpairs every tablet that paired after the dump was taken (R225-F1, README.md:176). Before you restore, list the tablets paired after the dump, so you know which ones need to pair again.

WARNING A Core restart ends every sign-in. A console restart ends the web sign-ins (README.md:85). Any rollback does at least one of these.

8.10 Docker

`deploy/docker/` holds a task-4 Docker image and installer. Its status is UNVERIFIED (`deploy/docker/README.md`, "Status").

WARNING Docker is not the bench deployment, and no test suite ran on it. The Linux host profile that `REAL_PRODUCTION` asks for can't be satisfied inside a container: it forbids a container runtime on the production host (`deploy/docker/README.md`, section "First production boot"). **Unknown:** how the Docker installer relates to the bench's native systemd install. This manual gives no Docker procedure.

8.11 Ops facts

TABLE 8.6 Day-to-day facts

TOPIC	FACT	SOURCE
Core log	`journalctl -u dems-core`; persistent errors also go to `apps/core-server/logs/errors.log`	`apps/core-server/README.md:132`
LCD log	`journalctl -u dems-lcd -f`; device lines are prefixed `panel:`	`apps/lcd-backend/README.md:116`
Console stop	Stopping `dems-console` leaves it `failed` with status 143; it starts cleanly	README.md:85
Media store	Owner-only (0700), owned by the Core's user. Hidden folders `.quarantine`, `.transcriber`, `.sealing`, `.capture-session` inside it. A real-capture Core refuses to start otherwise (`CAPTURE_EVIDENCE_STORE_UNUSABLE`)	README.md:713
Evidence volume	Must support hard links, exclusive file creation and a full durability sync; a real-capture Core probes all three at start	README.md:742
Moving the media store	Needs a manual rename of the session-key folder; no command does it	README.md:729
Preview CPU	4 live preview pipelines at 26.5-31.5 % of one core each, about 120 % in total, with `dems-core` itself at 4 % (console Dashboard open). All viewers of a camera share its one pipeline. Load during a recording with previews open is still to be measured	AUDIT §5; D249

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP The CPU figure was taken while jest runs were also using the host (1-minute load 2.79). It isn't a clean idle number, and it says nothing about load during a recording.

CHAPTER 9

9 Security notes

This chapter collects what protects DEMS today, where the protection stops, and what's still open. Most mechanisms are explained in full elsewhere; here you get the security view and a pointer.

Decision numbers in this chapter come from the audit log unless they're marked "bench". "AUDIT §n" points to section n of `docs/AUDIT_REPORT_2026-11-06.md` in the repository.

9.1 Network exposure

Every DEMS listener binds `127.0.0.1` (README.md:434; `apps/core-server/README.md:22`):

TABLE 9.1 Listeners

LISTENER	BIND	HOW IT'S HELD THERE
Core server, 4420	`127.0.0.1`	Constant `LISTEN_HOST` in `apps/core-server/src/config.ts`; no variable changes it
Web console (Vite), 5173	`127.0.0.1`	Dev server config, strict port
LCD backend, 8090	`127.0.0.1`	`DEMS_BIND=127.0.0.1` in `dems-lcd.service`
PostgreSQL, 4432	`127.0.0.1`, TCP only, no Unix socket	Bench: system cluster config. Dev: address, port and socket directory on the `pg_ctl` command line
Capture ingest, 8554-8564	`127.0.0.1`	Worker code, plus the nft rule: only uid `dems-core` may connect
Mock Core, 4400 (dev only)	`127.0.0.1`	Shared contract constant

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

The root README's rule for development applies to the bench too: don't expose any of these on another interface, through a port forward or through a tunnel (README.md:434). If another PC ever needs the console, that's a separate, temporary decision: expose only the console port, bound to the LAN interface address rather than `0.0.0.0`, so the camera network can't reach it (`apps/lcd-backend/README.md:139-141`).

The tablet is the exception. It reaches the Core over the LAN on port 4420, in cleartext `http` and `ws` (residual R263-TLS, chapter 5).

HEADS-UP **Unknown:** how a tablet on the LAN reaches a Core whose listener is fixed to `127.0.0.1`. The sources don't document a forwarder, proxy or bind change, and LAN `http`/`ws` from a real device hasn't been proven (AUDIT §6). Whatever closes this gap will widen the network exposure in Table 9.1; review it as a security change when it's made.

9.2 Request security

Every request to the Core passes the plugin pipeline described in chapter 3 (`apps/core-server/src/plugins/README.md:13-22`). The security-relevant stages:

TABLE 9.2 Security-relevant pipeline stages

STAGE	WHAT IT DOES
Origin guard	Accepts only the exact console origins (`WEB_CONSOLE_ORIGINS`: `http://127.0.0.1:5173`, lanes 5176-5178). Anything else gets 403 `ORIGIN_NOT_ALLOWED`
Authenticate	A cookie or a bearer token resolves to an operator, a device or anonymous
Throttle	20 audited refusals within 60 s block the key
Permission guard	A missing permission is refused with one audited `ACCESS_DENIED` row
Error envelope	A 500 answers `INTERNAL_ERROR` with no detail
Response headers	Cache-Control, X-Content-Type-Options, CSP, CORP, Referrer-Policy, plus a request id

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Roles are Investigator, Supervisor and Technician (README.md:425). Permissions such as `SESSION_START`, `PAIRING_MANAGE`, `USER_ADMIN` and `CAPTURE_COMMISSION` (Technician only) gate the console pages (chapter 4), but the Core's permission guard is what actually refuses.

Two vendor identities, `VENDOR_ROOT` and `VENDOR_ACCOUNT`, gate the operator administration trio (create, update, delete user) and the LCD panel setup (ops 115 and 116) (bench D54-D59, as reconstructed in the audit's decision map). For the individual operation numbers of the admin trio, see the shared operation registry; the sources disagree on them.

Other rules worth knowing:

- A web sign-in expires after 15 minutes idle and after 8 hours at most (README.md:540).
- A body of op 91 (`updateSttConfig`) carrying `credential` is refused with `403 NOT_PERMITTED` (`WEB_CONSOLE_ONLY`) from a tablet session. Only the web console may set the STT credential (S99-09).
- Device commands that would only be simulated on a Core that captures for itself are refused, instead of being audited as "(simulated)" (D89).

9.3 Device credentials and tokens

Pairing (chapter 5):

- The pairing code is one-time: six digits, valid 5 minutes, works once, shown once (README.md:554). These values come from the README; the code constants weren't read for this manual.
- Three failed redeems within 300 s lock redeeming for 60 s (`apps/core-server/README.md:145`).
- The device credential lives in the tablet's secure store, bound to the Core address. Changing the address deletes it (D263).
- Revoking a device closes its socket with code 4403.

Signed paths carry their own bearer token:

TABLE 9.3 Signed paths

PATH	TOKEN FROM	LIFETIME AND BINDING

PATH	TOKEN FROM	LIFETIME AND BINDING
`/media/<token>`	Op 88	A 60 s media session that the player renews on demand (README.md:1109, Known limit 14)
`/exports/<token>`	Op 87	The download link lives 60 s and is bound to the sign-in. A download already running isn't cut when the link expires; it's cut when the sign-in ends
`/preview/<token>`	Op 112	Camera live preview. The URL is a bearer value and is never logged or stored by the console

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

The tablet's runtime log blanks the token of a signed media URL before storing a record (`apps/mobile/src/runtime/README.md`, `log-media-token.test.ts`).

9.4 Evidence at rest

WARNING A recording is plaintext on the media store from the start of the session until its seal completes. The worker writes the camera files unencrypted; the transcriber's raw audio tap under `<media store>/.transcriber/sessions/<sessionId>/` is plaintext too. If the Core or the machine stops in between, the recording stays plaintext until recovery seals it. The owner-only media store (0700) is the only protection. A stolen or imaged disk exposes every unsealed recording and everything in the quarantine (README.md:708-715).

What happens at and after the seal (chapter 3):

- Each fragment is encrypted as one AES-256-GCM message. The nonce is random, so uniqueness under one session key is probabilistic, not proven (Known limit 23).
- Plaintext is deleted only by proof: the manifest names it, the sealed counterpart exists, and the file's bytes hash to the manifest's plaintext hash. A file that can't be proven goes to `.quarantine/<sessionId>/` (folders 0700, files 0600), in plaintext, with a CRITICAL alert.
- The audit extract inside a package (`audit-session-extract.jsonl`) is not covered by the signature. Verify says so on every package (`AUDIT_EXTRACT_NOT_SIGNED`) (README.md:751).
- An export leaves a decrypted archive `EXP-<id>.zip` on the Core's disk. No retention rule exists (S99-03).

The two keys have opposite risks (README.md:984-991):

TABLE 9.4 The two keys

	SIGNING KEY	SESSION KEYS
Where	`<data directory>/signer`	`<data directory>/credential-store`
Does	Signs packages, exports and transcript revisions	Decrypts the sealed recordings
Risk	A **copy**: someone can sign evidence elsewhere	A **loss**: every sealed recording is unreadable for ever
Backup	Never copy the key file and its wrap secret into a backup, image or support bundle; the certificate is public	No escrow exists (Known limit 37); the user's open decision

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

The signing key is a software Ed25519 key in files on the host, with a self-signed certificate (README.md:977-982):

- No certificate authority stands behind it, and no revocation channel exists.
- The organisation and installation names in the certificate are what the installer typed.
- The signing time is the signer's own clock.
- A receiver trusts a package only through the certificate fingerprint handed over out of band. A compromised key is announced the same way: the receiver passes a list of fingerprints to `evidence:verify-export ... --compromised <sha256>` (Known limit 54).

The host account the Core runs as has full control of every operator account, of the audit log and of the signing key. The boot checks on the key and the operators (4, 4b, 4b-2, 4b-3, 4c) are necessary conditions, not proof of correct provisioning (README.md:977-982).

9.5 Host hardening

TABLE 9.5 Hardening in the bench units (`deploy/bench-config/systemd/`)

DIRECTIVE	dems-core	dems-console	dems-lcd
Own service user	`dems-core`	`dems-console`	`dems-core`
`UMask=0077`	yes	yes	yes
`NoNewPrivileges=yes`	yes	yes	yes
`CapabilityBoundingSet=` and `AmbientCapabilities=` (empty)	yes	yes	no
`LimitCORE=0`	yes	yes	no

SUMBER deploy/bench-config/systemd/; DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

`dems-kiosk-agent` runs as root with none of these lines, and the Wi-Fi watchdog runs as root too. Their programs aren't in the repository (chapter 8).

Other host-level rules:

- **The nft loopback rule.** Only uid `dems-core` may connect to `127.0.0.1` TCP 8554-8564; everyone else gets a TCP reset. So no other local user can push a stream into the worker's ingest.
- **The rule's state.** The Core reads the state of `dems-loopback-rule.service`, not the nft table itself. An unprivileged process can't list the table, and giving the Core that capability would also let it change the table. A table deleted by hand while its unit stays active isn't noticed (README.md:1143, Known limit 48).
- **Worker program paths.** The capture programs must be owned by root or the service user and not writable by group or others; a worker program that fails the path rule is refused at boot (`CAPTURE_WORKER_PROGRAM_UNUSABLE`, chapter 3).
- **Build stamp.** The Core refuses a worker whose stamp doesn't match its binary, source, version or protocol (chapter 6).
- **Connection file mode.** The Core refuses `pg.env` if any group or other bit is set (`CONNECTION_FILE_MODE` in `connection-file.ts`, `deploy/bench-config/README.md:109`).
- **Relay and guard.** The relay never listens and binds once; the guard execs one child with no shell and no search path (chapter 6).

9.6 Secrets handling

TABLE 9.6 Where the secrets live

SECRET	WHERE IT LIVES	RULE
`core.env` (camera default password, provisioning key)	`/etc/dems/core.env`	root:dems-core 0640; never committed; shared only over SSH between team machines (`deploy/bench-config/README.md:193-198`)
Database passwords	`pg.env`, `pg.owner.env` in the data directory	0600, enforced for `pg.env`; never in `core.env`
Signing key	`<data directory>/signer`	Never in backups or bundles
Camera credentials	The Core's credential store	Typed in the web console only; write-only, never shown again (README.md:704)
STT server key	Outside DEMS	DEMS holds a credential reference, never the key (README.md:785)
Release keystore (tablet)	Git-ignored `apps/mobile/signing/`	Password generated by the script, never printed (chapter 5)

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

What the repository does to keep secrets out of git and logs:

- `.gitignore` ignores `*.p8`, `*.wrap` and `*.blob` by form (README.md:603).
- The shared log-blanking rule blanks the userinfo of a URL carrying a password: `rtsp://user:password@host` is logged as `rtsp://***@host` (S99-04). A password with a raw `/` isn't covered (Known limit 44).
- The development `pg:init` writes its three credential files with mode 0600 and never prints them.

9.7 Demo-era leftovers

The old README contains published demo credentials for synthetic accounts, and the seed source holds them too. They are not live on production, and production refuses to boot while any operator still has that password (boot check 4c, README.md:975; AUDIT §9, C1). This manual doesn't reproduce them.

9.8 Open security items

None of these is fixed at `59a683ed`. Each is a decision for the project owner unless the table says otherwise.

TABLE 9.7 Open security items

ITEM	WHAT'S OPEN	SOURCE
R263-TLS	Tablet to Core traffic is cleartext `http`/`ws` on the LAN. TLS is the proper fix	README.md:177; D263
R174-G5	Biometric sign-in is off in release builds; enabling it needs Keystore-bound authentication proven on a real tablet	README.md:178; D199
R225-F1	A database rollback silently unpairs tablets paired after the dump. Open fix: a runbook step that lists them first	README.md:176; D204
R225-F2	Cores have no unique id per installation	same

ITEM	WHAT'S OPEN	SOURCE
R225-F3	The tablet should compare the stored Core id before sending its credential; the address binding of D263 covers part of it	same
Plaintext before the seal	Encrypting while recording isn't built	Known limit 1
No key escrow	Losing the credential store loses every sealed recording	Known limit 37
S99-02	The Core opens a local device for any caller with the session-start permission; the confirmation is a client-side rule only	README.md:652
S99-03	No retention rule for the decrypted export archives the Core keeps	README.md:653
S99-05	The key of a replaced camera credential is never destroyed	README.md:1139
S99-06	The signer's private-file read is weaker than the key store's	README.md:1139
S99-08	How the worker loads its FFmpeg libraries, and how a real-capture Core may be shipped with a GPL-3.0-or-later FFmpeg	README.md:649, 1139
SEC35-26	Personal data in real evidence: no legal basis, retention or erasure rule, and no impact assessment exist. No recorded decision accepts this risk	README.md:651
`/opt/dems/core` layout	The deployed tree is group-writable (213 directories and 11,698 files at the audit); flagged for a security review	D75
Items for review	The camera-link file read, the stamp read, the default camera credential, the cleartext plugin and `camera-stream-transport` were routed to a security review	D41, D48, D64, D183, D217, D263
Publication cautions C1-C7	Demo credentials in history (C1), an author mailbox in commit metadata (C2), deployment details in older docs (C3), about 1.6 GB of history (C4), screenshots and recordings not viewed one by one (C5), a missing ignore rule for a root signing folder (C6), two committed `.pyc` files (C7)	AUDIT §9

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

WARNING The security review that the audit's end-of-mission gate would have included never ran (AUDIT §1, "Not done"). The items above are what was recorded, not the result of a full review.

CHAPTER 10

10 Design decisions and history

Chapters 1 to 9 describe DEMS at commit `59a683ed`. This chapter explains why parts of it look the way they do: for each decision, the design it replaced, what changed, who asked, and what happened afterwards.

10.1 How to read this chapter

Two decision series share the same numbers.

- The team's **bench log** (D1-D64 and later) is kept on the bench host. Bench D49, D51, D53, D58, D60, D61 and D70 shaped the system before the audit.
- The **audit log** (D1-D283) belongs to the 2026-11-05/06 audit, fix and tablet task, and reuses the numbers. Audit D60 is a fault-cleared fix; bench D60 is the change to four video tracks.

The repo READMEs use both. "D60" in `apps/capture-worker/README.md` or `apps/lcd-backend/README.md` means bench D60. This document writes "bench Dnn" or "audit Dnn" wherever the context doesn't settle it, and every record in 10.3 names its series.

The full logs aren't in this repository. The audit log lives with the task documents outside the repo; only the sanitized summary `docs/AUDIT_REPORT_2026-11-06.md` is in it. The bench log wasn't available for this document, and records that depend on it say so.

Most audit rulings are the controller's. The audit was run by an agent controller. On 2026-11-05 the user handed over full autonomy on one condition, that every decision be logged:

> "lakukan semua pekerjaan secara full otonom, saya tidak akan interupsi apapun, semua keputusan bisa kamu ambil sendiri, saya ikut rekomendasi kamu, asalkan kamu mencatat lognya"
>
> *The user, 2026-11-05, audit D12*

In English: do all the work fully autonomously; I won't interrupt; you may take every decision yourself; I follow your recommendation, as long as you log it. The same day the user added that no questions should come back until the work was done:

> "apapun pekerjaannya jangan tanyakan saya lagi sampai semua selesai ya"
>
> *The user, 2026-11-05, audit D16*

"Controller ruling under D12" below means the controller picked its own recommended option and logged the reason. Even under D12, some things stayed off-limits: typing any password other than the one credential the user supplied, deleting data permanently, work that needs hands on the hardware, and weakening evidence integrity or security without a logged reason.

One request is described, never quoted. In audit D3 the user supplied the production credential in chat (not reproduced). The controller never wrote it to any file or log.

Ground rule for overrides (audit D6(b)). A user request may override an earlier spec or design rule, logged each time, but never security rules, boot checks, identity gates, evidence or audit integrity, the bench rulings, or the user's word on hard actions.

Quotes. Requests are quoted exactly as the audit log holds them, in the user's language (mostly Indonesian), with an English rendering after the quote. Three requests are logged only in English and carry the label "(as logged; original wording not recorded)". Where the log holds only a paraphrase, the record says so and quotes nothing.

10.2 Timeline

Figure 10.1 places the records of 10.3 on a timeline, oldest first.

FIGURE 10.1 Decision timeline, 2026-10-23 to 2026-11-06

```
2026-10-23     bench D51/D53    live camera preview, idle mic meters    ADR-3
2026-11-04     bench D60        four video tracks VIDEO-01..04
2026-11-04/05  bench D61        four mic channels MIC-01..04
      |
2026-11-05     audit D5         Camera N <-> Mic N pairing              ADR-1, ADR-2
               audit D25/D47    console lock, "Not reported"            ADR-5
      |
2026-11-06     audit D148       mission order swapped                   ADR-7
               audit D153       D5 proof limit                          ADR-11
               audit D163       release builds LIVE-only                ADR-9
               audit D199       biometric off in release                ADR-13
               audit D204       credential-binding finding              ADR-12
               audit D231/D232  previews everywhere, fixes first        ADR-3, ADR-8
               audit D263       configurable Core address               ADR-10
               audit D270       public repository                       ADR-14
               audit D273       all missions stopped                    ADR-17
               audit D274/D275  environment as a template               ADR-15
```

SUMBER Audit decision log, kept outside the repository (see chapter 10), and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

10.3 Decision records

10.3.1 ADR-1: Camera N to Mic N pairing

audit D5 · 2026-11-05

Original design. Three earlier designs sat on top of each other.

- Task 3 recorded one session audio track, `AUDIO-01`, and let an operator switch its source (op 81), stored as intervals.
- Bench D49 (2026-10-23) removed the web "move microphone" control. The control came back at the audit's starting point.
- Bench D61 (2026-11-04/05) widened the microphones to four channels `MIC-01..04`. Parts of the code still worked with the two old roles, PRIMARY and STANDBY (ADR-4 shows one leftover).

Change.

- Fixed pairs `VIDEO-0N` <-> `MIC-0N`, set in the capture setup. Each camera file carries the audio of its own mic.
- The pairing goes by channel number; the setup chooses which microphone device sits on each channel (audit D27).
- Op 81 is retired and answers `409 AUDIO_SOURCE_FIXED_BY_PAIRING`. `SWITCH_AUDIO_SOURCE` is gone from the capture protocol (audit D136).

- Changing a mic in the selector is a confirmed swap: one op 78 call with a two-line body, written as one channel-swap audit row (audit D30 K-6, D125, D126).

Request.

> "mic ditentukan dari setup, diconfigurasi mapping pair camera 1-mic 1 sampai camera 4-mic4"
>
> *The user, 2026-11-05, audit D5*

In English: the mic is decided in the setup, configured as a pair mapping from camera 1-mic 1 up to camera 4-mic 4.

Reason and alternatives considered.

- Pairing by channel number matches the user's wording and needs no new pairing storage: the existing `mic_01..04` settings already say which device sits on each channel (audit D27).
- Free cross-pairing (any camera with any mic, option DB-D5-3) was rejected as beyond the request (audit D27).
- Transcription stays on Pair 1 only. Transcribing four channels was ruled out for CPU and memory on the bench host, and recorded as a limit (audit D28, DQ-3).

Impact.

- Capture protocol 6 and `capture-worker/0.8.0`, changed in lockstep (audit D117).
- Migration `0044_recording_fragments_audio_tracks` lets `recording_fragments` accept `AUDIO-01..04` (audit D118). It was planned as 0043 and renumbered when the migration order was fixed (audit D29).
- The plan pairs each camera with its mic (audit D124); the worker runs one encoder per mic channel (audit D135); the web shows "Camera N <-> Mic N" rows and the swap (audit D125, D126).
- All of it shipped in W3, commit `37891824`: 197 files, about 7.8 s of downtime at the deploy.

History.

- Audit D21 declared the per-channel mic files and had them sealed as part of the same design (ADR-2).
- Audit D30 K-3: if a paired mic can't open, the recording still starts with a silent track in the camera file. An interview is never lost to one failed mic.
- Audit D153 limits the proof: D5 is proven by its parts only (ADR-11).
- Audit D235/D240 fixed the level meters for channels 3 and 4 (ADR-4).
- Mission 3 carried the pairing to the tablet: op 81 retired, the paired-mic notices, the swap body and its confirm dialog (tablet rows 185-189).

10.3.2 ADR-2: Evidence manifest /2.1, and quarantine instead of deletion

audit D21, D30, D45, D128 · 2026-11-05/06

Original design. Manifest schema `/2`. A package sealed only camera files, and the audio lived inside them as one track.

Change.

- Schema `dems.evidence-manifest/2.1`; `manifestVersion` stays 2 (audit D45).

- `AUDIO-01..04` (the `MIC-0N.m4a` files) are sealed and signed. A missing mic is stated under the signature as `audioTracksMissing`, with the reason `NOT_ASSIGNED` or `NOT_OPENED` (audit D45, D128).
- An unaccounted `MIC-0N.m4a` is moved to quarantine, never deleted.
- Legacy `/2` packages still verify, with byte-identical results. The verifier reads both schemas and raises the notice `AUDIO_TRACKS_MISSING` for a stated missing mic (audit D30 K-2, D119).

Request. Controller rulings under D12 (the delegation is quoted in 10.1), following the user's D5 request (ADR-1).

Reason and alternatives considered.

- Audit D21 chose to declare and seal the mic files (option (a)) rather than leave them outside the package, because they hold the per-pair audio of D5.
- Audit D45 (R145-3) adds no completeness rule, so a failed mic file never makes an honest package unsealable; the package states what is missing.

Impact. Sealing and shared-verifier rows 145, 153 and 154. Before the deploy, the new verifier and the deployed one gave identical results on six real legacy packages (audit D122).

History. One residual is still open: when the source-facts file is missing and every mic failed, the session seals as `/2` (R148-5, audit D132).

10.3.3 ADR-3: Live previews on every surface

bench D51, D53, then audit D231, D239 (+D241) · 2026-10-23 to 2026-11-06

Original design.

- The task-3 spec rule OD-12 said: no live preview.
- Bench D51/D53 (2026-10-23) reversed it at the user's request. They added an MJPEG live preview (about 8 fps, 640x360) and idle mic meters, with recording taking priority over preview.
- A later team ruling, bench D70, said "only the main feed decodes video". It kept the Dashboard tiles and the Session Monitor thumbnails as still pictures. **Unknown:** its date; the sources of this document don't give it.

Change.

- The Dashboard camera tiles, the active-session thumbnail and the Session Monitor thumbnails use `ReachGatedCameraPreview` with the live preview (audit D239).
- Streams close when the tab is hidden, on Session New too (W4-8, audit D241).
- The Session Monitor re-checks, once per mount, a camera unreachable for 60 s or more.

Request.

- Bench D51: the original request isn't available in the sources of this document. The audit's decision map reconstructs the user's wording as "Video live bergerak" (as reconstructed in the audit's decision map), in English "moving live video".
- Audit D231 records the user's observation as a paraphrase, not a quote: the Dashboard and the Session Monitor showed cameras as READY with no

picture while Devices & Inputs showed one, and the user asked why it wasn't the same code.
- The fix wave itself followed the user's D232 instruction, "fixes first" (quoted in ADR-8).

Reason and alternatives considered. It was the same component all along; the live picture is opt-in per caller, and the thumbnails never opted in. The cost reason behind bench D70 had mostly gone: the shared contract caps preview pipelines at 1 per camera and 4 in total, viewers at 4 per camera and 16 in total, and extra viewers share a camera's one pipeline (audit D231). Kept for later: measure the CPU at recording time.

Impact. W4 commit `8962f16b`. Measured after the deploy: each of 4 preview pipelines used 26.5-31.5 % of one core, about 120 % of one core for all four (audit D249).

History. Tablet row 196 mounts the live preview at four sites (audit D219), so web and tablet follow one rule. The load during a recording with previews open is still to be measured in the 4+4 acceptance recording.

10.3.4 ADR-4: One mic status reader, and mic reach stated by the Core

audit D226, D238, D243 (+D235, D240) · 2026-11-06

Original design.

- One mic had two readers: "Audio inputs" said READY while "Registered microphones" said "Not checked" (audit D226).
- Idle levels were forwarded by role: channel 1 as PRIMARY, channels 2-4 as STANDBY. The shared code mapped STANDBY to `MIC-02` only, so `MIC-03` and `MIC-04` never showed a level. This was a leftover of bench D61's two roles (audit D235).
- Nothing in the Core stated whether a mic was reachable (audit D238).

Change.

- One reader for mic status (`slotSourceOf` plus `trackReachOf`) (audit D238).
- Levels carry the mic id and resolve by id first. The wire protocol is unchanged (audit D240).
- The Core states mic reach from its own idle level reading, throttled: in the test, 2,440 levels produced 6 writes (audit D243).

Request. The fix wave followed the user's D232 instruction, "fixes first" (quoted in ADR-8). The two-readers problem itself is an observation made during judging (audit D226), not a request.

Reason and alternatives considered.

- Audit D240 rejects a fallback to the role for an unknown mic id: it would bring the misrouting back.
- Audit D243 accepts that levels during a recording don't refresh reach: idle meters are off then, the mic-loss watch covers a loss, and the recording path gets no extra load.

Impact. W4 commit `8962f16b`. Files in `apps/core-server/src/capture-composition.ts`, the idle meter, the shared capture telemetry, and the web devices pages.

History. Residual R-W4-7: a mic lost in the middle of a recording keeps its old "Ready" until the next meter reading or pre-check.

10.3.5 ADR-5: Production console lock and "Not reported"

audit D25, D29, D47 · 2026-11-05

Original design. The console offered Simulated wherever it ran as a dev server, which is how production runs it. Bench D58 had removed the demo-operators disclosure from the login card but kept the mode selector.

Change.

- `VITE_DEMS_PRODUCTION_LOCK=1` forces LIVE and removes every Simulated affordance (audit D47; the variable was named in audit D29 Q6).
- On a `REAL_PRODUCTION` Core, anything without a real data source shows "Not reported". No synthetic values are drawn (audit D25 N3). This overrides bench D58's kept mode selector, and the override is logged.
- The pre-check's Core-link warning on an own-capture Core (bench D48) stays; only its reason was reworded (audit D29 Q2).

Request. Controller rulings under D12 (quoted in 10.1).

Reason and alternatives considered. The root cause sat in the web runtime: the console decided the mode as "selectable when dev server and not packaged", and production is exactly a dev server (audit D47). A packaged build was no way out: a console built with `vite build` is always SIMULATED (section 4.2).

Impact. The drop-in `deploy/bench-config/systemd/dems-console.service.d/production-lock.conf`; rule 1b in `apps/core-web/src/runtime/core-mode.ts`; W1 commit `b11950d6`.

History. Mission 3 applied the same rules to the tablet (audit D47, lane item B6; tablet rows 197-200).

10.3.6 ADR-6: Worker version, build stamp and the lockstep rule

audit D15, D44, D48, D117 · 2026-11-05/06

Original design. The boot checked only the worker's version and protocol. The deployed worker predated the four-camera change but still reported 0.6.0 and protocol 5, so it passed the boot and then refused every 4+4 plan with `PLAN_INVALID`.

Change.

- `capture-worker/0.7.0` with a build stamp (audit D44). The stamp file holds four lines: `workerVersion`, `protocolVersion`, `sourceSha256`, `binarySha256`.
- Boot check 5b `CAPTURE_WORKER_BUILD_STAMP_STALE`, run on `REAL_PRODUCTION` (audit D48).
- A new rule: a change to the plan shape bumps the capture protocol in the same commit (audit D25 N5).
- `capture-worker/0.8.0` with protocol 6 for D5 (audit D117).
- Worker and Core deploy together, and roll back together. A new worker never runs beside a Core that expects the old one (audit D44).

Request. Controller ruling under D12 (quoted in 10.1). It was the first fix of the mission (audit D15).

Reason and alternatives considered. A rebuild alone fixed the symptom. Audit D15 also asked for an upstream guard: a version or protocol bump on a plan-format change, or a build-stamp check. Both were adopted.

Impact. W1 commit `b11950d6` (worker 0.7.0 plus stamp); W3 commit `37891824` (worker 0.8.0, protocol 6, stamp from one fresh build).

History.

- For stamp mismatches this closes README Known limit 21 (an older worker failing only at the first session).
- Residual R-1: the stamp is checked at boot only, not again at session start (audit D76).

10.3.7 ADR-7: Mission order swapped

audit D148 · 2026-11-06

Original design. M1 audit and fix, then M2 end-to-end UI judging, then M3 tablet adaptation:

> "tambahan misi, setelah 2 misi panel penjurian dan perbaikannya selesai, lanjutkan ke penyesuaian mobile apps / tablet, sesuaikan dengan dems core yang baru"
>
> *The user, 2026-11-05, audit D8*

In English: an extra mission; after the two missions (the judging panel and its fixes) are done, continue with adapting the mobile app / tablet to the new DEMS Core. The user then allowed tablet work to start early where judging needed it:

> "jika diperlupan penyesuaian apps tablet terlebih dahulu sebelum penjurian saya ijinkan"
>
> *The user, 2026-11-05, audit D13*

In English: if the tablet app needs adapting before the judging, I allow it.

Change. M3 runs before M2.

Request.

> "tukar setelah misi1, misi3 terlebih dahulu"
>
> *The user, 2026-11-06, audit D148*

In English: swap; after mission 1, mission 3 first.

Reason and alternatives considered. With M3 first, M2 would judge the web, the LCD and the adapted tablet in one pass.

Impact and history. Audit D262 postponed M2 and asked for a release APK for a real-device test (the log holds this request as a paraphrase only). Audit D273 stopped all missions (ADR-17). M2 never ran as a panel.

10.3.8 ADR-8: Fixes first (the W4 wave)

audit D232 (+D236, D247) · 2026-11-06

Original design. Judging would continue, and fixes would queue for later waves.

Change.

- The queued web, LCD and shared fixes ran at once, as wave W4.

- After W4, one Core update, then a pause for the user's own check (audit D236).
- Work resumed on the user's "Checked, continue" (audit D247, as logged).

Request.

> "better insert fixes like this first, then patch"
>
> *The user, 2026-11-06, audit D232 (as logged; original wording not recorded)*
>
> "after the judging finishes, pause to update the Core"
>
> *The user, 2026-11-06, audit D236 (as logged; original wording not recorded)*

Reason and alternatives considered. No reason is logged beyond the requests. The alternative, queuing the fixes for M2, is what the change replaced.

Impact. Rows W4-1 to W4-8; commit `8962f16b`; about 1.2 s of downtime at the deploy. Chapter 11 lists the rows.

History. The W4 changes are recorded in ADR-3 and ADR-4. M2 itself never ran (ADR-17).

10.3.9 ADR-9: Release builds are LIVE-only

audit D163 (+D169, D174, D199, D208) · 2026-11-06

Original design. The 0.1.0 release APK was Simulated-only, and Heartbeat was disabled in release. Live existed only in debug builds.

Change.

- Release builds carry only the Live source (audit D163 RT-1). They never restore the simulator snapshot and offer no switch to Simulated (audit D169, D174, D199, D208).
- Debug builds and jest keep Simulated and Heartbeat, reachable through `?core=` links and tests.

Request. Controller ruling under D12 (quoted in 10.1), following the user's D8 request to adapt the tablet to the new Core (quoted in ADR-7).

Reason and alternatives considered. Audit D21 (item 4) reopened the old Simulated-only rule and set the target, following the user's tablet direction (audit D4, D8, D13): a release tablet build must carry the Live connection. For the network side, RT-2 first accepted a cleartext config limited to `127.0.0.1` and `localhost` (audit D163, D183).

Impact. Mission 3 cycle 1, commit `cf59429e`.

History. Audit D263 (ADR-10) replaced the loopback-only cleartext with base-config cleartext, so that a release build can reach a Core on the LAN.

10.3.10 ADR-10: Configurable Core address, with LAN cleartext

audit D263 (+D264, D265, D266, D267) · 2026-11-06

Original design. The app reached the Core only at `http://127.0.0.1:4420` through `adb reverse`. The cleartext plugin allowed exactly `127.0.0.1` and `localhost` (audit D183).

Change.

- A Core address card on the Pair screen: host and port, default `127.0.0.1:4420`, so the `adb reverse` path keeps working. Settings shows it read-only.

- One runtime source of the Core origin (`apps/mobile/src/runtime/core-address.ts`) for HTTP, WS, media and preview.
- Changing the address first deletes the stored device credential, the owed sign-out and the Core snapshot. A different Core means a new pairing.
- Input validation: private, loopback or link-local IPv4, or a hostname; port 1-65535. Public IPv4, IPv6, a scheme or a path are refused (audit D265).
- The release base config permits cleartext (audit D264).

Request.

> "on the tablet make a configuration to input the host address, do not hard-code it"
>
> *The user, 2026-11-06, audit D263 (as logged; original wording not recorded)*

Reason and alternatives considered.

- Android's network security config can't allow cleartext by IP range. To reach a LAN Core over `http`/`ws`, a release build has to permit cleartext in the base config.
- The compensations: public IPv4 addresses are refused in the app, the credential is bound to the address, and the interview-room LAN is a closed network.
- TLS to the Core is the proper fix. It is left open as residual R263-TLS, for the user to decide.

Impact. `apps/mobile/src/runtime/core-address.ts`; `apps/mobile/plugins/with-dems-core-cleartext.js`, which replaced the loopback-only plugin. Commit `f41972f9` (29 files, audit D266); the release APK for a real-device test was built from it (audit D267).

History. It realises part of R225-F3, "never send a credential to another Core" (ADR-12). Residuals R236-R1 to R5 were accepted with it. There is no real-device proof yet.

10.3.11 ADR-11: The D5 proof limit

audit D153 (+D150, D152) · 2026-11-06

Original plan. Prove D5 with the gated end-to-end tests, run on the bench host.

Change.

- The gated D5 end-to-end tests can't pass on the bench, because the product's own trust rules refuse every setup that was tried:
  - run as a user other than `dems-core`, the nft rule resets every connection to the ingest ports (audit D150);
  - run as the service user, the programs are refused: they must be owned by root or the running user (`FILE_OWNER`, audit D152);
  - root-owned copies under `/var/tmp` are refused with `DIRECTORY_WRITABLE`, because a group- or other-writable parent isn't trusted (audit D153).
- D5 is proven by its parts: unit tests, DB windows including migration 0044, the real worker on media pairs, the verifier on six real packages, and the sealing unit tests.
- The first real 4+4 session is a mandatory acceptance test before operational use.

Request. Controller ruling under D12 (quoted in 10.1).

Reason and alternatives considered. The alternative was to loosen the firewall rule or the trust rules for the test. D12 doesn't permit weakening security, so both stayed as they are and the residual was accepted instead (audit D152, D153).

Impact. The residual is carried in every report since. The 4+4 recording needs a fourth mic on the four-input audio interface (audit D229) and every camera answering (audit D228).

10.3.12 ADR-12: Pairing and credential binding

audit D204 (+D202, D225, D226) · 2026-11-06

Original design. The tablet sent its stored device credential to whichever Core answered at its configured origin. On a 401, the app deleted the credential silently.

Finding.

- The emulator's credential came from a development Core with its own database and the same installation id. Production answered 401 and the app deleted the credential (audit D204, correcting the reading in audit D202).
- A tablet is silently unpaired by (a) a `pg_restore` rollback to a dump taken before it paired, (b) a move to another Core, or (c) a reset of the pairing tables.
- A Core restart or a normal deploy doesn't unpair a tablet.

Change. Nothing in this mission beyond the address binding of ADR-10. Three fixes stay open:

- R225-F1: a deploy runbook step that lists tablets paired after the dump before any DB-restoring rollback;
- R225-F2: a unique Core id per installation;
- R225-F3: the tablet compares the stored Core id before it sends its credential (depends on F2).

Request. Controller ruling under D12 (quoted in 10.1). For the re-pairing, the controller stopped before typing a production password and asked the user to sign in personally (audit D225). It then issued one pairing code, which appears in no document, and re-paired the emulator (audit D226). A real tablet already paired with production was left untouched.

Impact. No code change. The three fixes are open items (chapter 12).

History. ADR-10 covers the "another Core" case in part.

10.3.13 ADR-13: Biometric sign-in off in release

audit D199 · 2026-11-06

Original design. The app had biometric sign-in: a simulated security-key sign-in in the demo, backed by a biometric module in the Core (`modules/identity/biometric`).

Change. A release build offers password sign-in and the Live source. Biometric sign-in stays gated off.

Request. Controller ruling under D12 (quoted in 10.1).

Reason and alternatives considered. Turning it on needs authentication bound to the Android Keystore, proven on a real tablet. An emulator can't prove that.

Impact. Mission 3 cycle 1, commit `cf59429e`.

History. Residual R174-G5. README Known limit 16 already said biometric sign-in was never gate-tested.

10.3.14 ADR-14: Public repository for the team audit

audit D270 (+D268, D269, D271, D272, D276) · 2026-11-06

Original design. Audit D268 (logged as a paraphrase only) planned a private publication through GitHub Desktop with "Keep this code private" on. The repository the user created turned out to be public, and the controller put the publication on HOLD (audit D269).

Change. Publish publicly now, and make the repository private after the team audit.

Request.

> "sekarang saya buka publik push dulu saja, nanti akan ada audit tim, nanti akan saya private jika sudah selesai"
>
> *The user, 2026-11-06, audit D270*

In English: I'll open it to the public now, just push first; there will be a team audit, and I'll make it private once that is done.

Reason and alternatives considered. A public push can't be taken back (forks, caches, indexing). The controller's condition: push only if a scan of the full history finds no secret, credential, signing material, personal data or file over 100 MB. The result was CAUTION with no blocker, items C1-C7 (audit D272, listed in chapter 11).

Impact. Merge `8b829aba` into `main`; four pushes, the last at `59a683ed` (audit D276-D283).

History. Open: the audit report's next step 1 is to make the repository private.

10.3.15 ADR-15: Environment published as a template, not as secrets

audit D274, D275 (+D277) · 2026-11-06

Original request. The user asked for a push with all the env and the final configuration:

> "push dengan semua env dan konfigurasi akhir"
>
> *The user, 2026-11-06, audit D274*

Change.

- The systemd units and the nft rule, which hold no secrets, are published as byte-identical copies.
- `core.env.example` is built from the code only, with placeholders. The real `/etc/dems/core.env` is never opened or published.
- A deploy-time self-configuration guide explains how to fill in the template (audit D275).

Request. The user then set the form:

> "push dengan format template dan tulis di readme untuk konfig sendiri saat akan di deploy"
>
> *The user, 2026-11-06, audit D275*

In English: push it in template form and write in the README how to configure it yourself at deploy time.

Reason and alternatives considered. Publishing the real `core.env` would hand anyone access to the production Core and its evidence. It would also break the standing rule never to open or quote `core.env` values.

Impact. `deploy/bench-config/`, commit `2bcc4c53`. No redactions were needed in the copied files (audit D277). The guide hasn't been executed on a host, because the bench was already provisioned; the README says so.

History. The audit report's next step 1 repeats it: share the real environment file privately, never through git.

10.3.16 ADR-16: How changes reach production

audit D1, D2, D32 · 2026-11-05

Original design. None recorded. The context: the bench host is also the production install, and other agents were working on the same host tree when the audit started (audit D2).

Change.

- Audit D1: the full verification chain (verifier, security review, test expert, senior code review, full suites, full UI judging) runs once, at the end. During the work: targeted tests, typecheck, and a smoke check after each deploy.
- Audit D2: fixes are made in a separate branch and worktree, never in `/opt/dems/core`, and deployed file by file with a backup first.
- Audit D32: the DB test project attaches to `127.0.0.1:4432`, the production cluster on the bench. Fix rows never run `*.db.test.ts`; DB tests run only in announced windows (dump, stop, test on the project cluster, restore, smoke).

Requests.

> "gate full hanya dikerjakan di akhir misi, saat misi berjalan jika dibutuhkan pakai gate minimal saja"
>
> *The user, 2026-11-05, audit D1*

In English: the full gate is done only at the end of the mission; while the mission runs, use only minimal gates where needed.

> "Branch di host + pause agen lain (Recommended)"
>
> *The user, 2026-11-05, audit D2*

The user picked this option label: a branch on the host, and pause the other agents. Audit D32 is a controller ruling under D12.

Impact. Every wave went through a worktree commit, a backup, a lockstep deploy and its DB windows (chapter 11).

History. Audit D273 stopped the missions before the D1 full gate ran (ADR-17). The full gate is still outstanding.

10.3.17 ADR-17: All missions stopped

audit D273 · 2026-11-06

Original design. M1, then M3, then M2 (ADR-7), closed by the D1 full gate (ADR-16).

Change.

- M3 closes without the cycle-2 re-judge on the emulator. Its fixes are proven by jest, tsc and eslint only (UNVERIFIED-ON-DEVICE); the user's real-device test of the `f41972f9` release APK replaces the re-judge.
- M2 is postponed.
- The D1 full gate doesn't run.

Request.

> "stop all mission, fokus ke push ke github dan dokumentasi komprehensif akhir"
>
> *The user, 2026-11-06, audit D273*

In English: stop all missions; focus on the push to GitHub and the final comprehensive documentation.

Reason and alternatives considered. No reason is logged beyond the request.

Impact. The open items in chapter 12.

History. No later decision reopened the missions.

CHAPTER 11

11 Audit changelog 2026-11-05/06

This chapter is the record of the audit, fix and tablet-adaptation task that produced the current `main`. Every decision number here belongs to the **audit** series unless it says "bench" (see 10.1 on the two series). The full decision log is outside the repository; `docs/AUDIT_REPORT_2026-11-06.md` is the public summary this chapter is built from.

11.1 Missions

TABLE 11.1 Missions of the audit

MISSION	CONTENT	STATUS
M1	Audit and fix of the team's updates (commits `e05a1567..53513b0c` plus the tablet LIVE-only work), and the new D5 pairing	Closed (D158)
M2	End-to-end UI judging panel	Postponed (D262, D273); only its fix items ran, as W4
M3	Tablet app aligned to the new Core (D8)	Code complete; cycle 2 not proven on a device (D273)

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

11.2 Method

- 163 functions in 11 channels were inventoried and judged; the failures were triaged into 56 defect rows.
- The defects were fixed in waves W1, W2 and W3, and later W4.
- Each wave was committed in a separate worktree and deployed to the bench install file by file, with a backup first.
- DB tests ran only in announced DB windows (dump, stop, test, restore, smoke), because the DB test project attaches to the production port (D32).

11.3 The first fix: a stale worker

The deployed capture worker had been built before the four-camera, four-mic changes (bench D60/D61). It refused every 4+4 plan with `PLAN_INVALID`, yet passed the boot, because it still reported version 0.6.0 and protocol 5. It was rebuilt and deployed, and the faulted session was recovered. To stop a repeat, worker 0.7.0 got a build stamp and boot check 5b `CAPTURE_WORKER_BUILD_STAMP_STALE` (D15, D44, D48; ADR-6).

11.4 Waves W1-W3

W1 (closed D89): 12-item pre-check for cameras 1-4 and mics 1-4 (D53); migration 0042 repairs the camera and mic foreign keys (D36); migration 0043 keeps the evidence-chain audit types through an operator purge and never leaves the chain head NULL (D51, D68); the engine adopts a resealed chain (D43); mic ids derived from the capture configuration (D39, D82, D87); source facts (D42); camera link watch with a sweep on link-up (D41, D61); default camera

credential offered only inside the camera /24 (D64); production console lock and "Not reported" (D47, D52, D57); system principal hardening (D37); op 66 refused on an own-capture Core (D83); LCD logo validation before save (D35, D79); OpenAPI regenerated (D59).

W2 (closed D115): `checkedAt` and reach state (D90, D94); device status from the assigned device plus its reach in web, Core and pre-check (D97, D100, D104); registered lists (D92); reach-gated previews (D95, D98); system principal permissions frozen and purge-kept accounts terminal (D91, D93); device commands refused instead of writing "(simulated)" audit rows (D96).

W3, the D5 pairing (closed D158): contract (D116); capture protocol 6 with worker 0.8.0 in lockstep (D117); migration 0044 for `AUDIO-01..04` fragments (D118); plan pairs (D124); per-channel worker encoder (D135); sealing `/2.1` with a signed missing-mic statement and quarantine instead of deletion (D128); the verifier reads `/2.1` and legacy packages verify byte-identical (D119, D122); op 81 retired (D136); pair rows and a confirmed swap in the web (D125, D126); durability counts camera files only (D151).

11.5 Wave W4

TABLE 11.2 W4 rows

ROW	CHANGE	D
W4-1	Dashboard and Session Monitor thumbnails use the reach-gated live preview; streams close on a hidden tab. The thumbnails had never opted in (bench D70)	D231, D239
W4-2	"View device" route; PoE switch and recording lamp not counted on self-capture; one mic status reader	D238
W4-3	Diagnosis: idle levels were forwarded by role, and STANDBY mapped only to `MIC-02`, so channels 3/4 never showed a level	D235
W4-6	Levels carry the mic id and resolve by id first; wire protocol unchanged	D240
W4-7	The Core states mic reach from its idle level reading	D243
W4-8	Session New preview closes on a hidden tab	D241
W4-4	LCD camera lamp lit only on a stated REACHABLE, or with no stated reach and a live stream	D234
W4-5	Stale "8 checks" comments now say twelve	D233
U-18	Not reproduced; parked with a capture plan	D242

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Gate DBW4 was green: shared 3013/3013, core-web 1904/1904, core-server 2922 plus 9 skipped (D244). Commit `8962f16b`, 33 files (D245). CPU after the deploy: 4 preview pipelines at 26.5-31.5 % of one core each (D249).

11.6 Mission 3: tablet rows

TABLE 11.3 Mission 3 accepted rows, condensed

ROWS	CHANGE	D
169-171, 174, 175, 223	Tablet HOLD lane adopted; release builds LIVE-only; no Simulated switch in release	D160, D169, D174, D199, D208, D190

ROWS	CHANGE	D
172, 224	Sign-out gaps closed; owed sign-out kept and paid on link-up	D188, D211
181-184	12-item pre-check (crash fixed), live readiness, 4-pair previews, count pins	D167, D197, D172, D206
185-189	Op 81 retired; paired-mic notices; swap body; confirm dialog	D177, D170, D200, D175, D210
190-192, 222	Roster and status from the assigned device; registered-device detail route	D173, D187, D195, D220
193-196	Live camera preview: renderer spike, reach gate, one transport file, 4 mounts	D202, D185, D218, D219
197-200	Core id and "Not reported" rules	D168, D213, D205, D212
201-203	Device commands blocked on own capture; offline alert; fault-cleared mark	D198, D203, D182
204-208	8-track order, verify notice bound 10, pair audio in integrity and verify views	D180, D184, D171, D209, D201
209-211, 226, 227	Pair words, hygiene, integration pass, attention line	D194, D191, D207, D214, D215
176, 177	Loopback cleartext plugin (later replaced); spec line	D183, D181
229-235	Cycle 2: defects F1-F6 and L7 from judging round 1	D250-D256, D258
236, 237	Core server address setting; release cleartext config	D265, D264

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Gates: jest 3428/3428 at gate 213 (D221); gate 218 green after one test-expectation fix-up (D258); jest 3523/3523 for rows 236/237 (D265).

11.7 Deploys

TABLE 11.4 Deploys to the bench install

DEPLOY	COMMIT	FILES	DOWNTIME	MIGRATIONS / WORKER	D
W1	`b11950d6`	173	about 9.2 s	0042 + 0043; worker 0.7.0 + stamp	D73
W1 cycle 2	`7bc77bfa`	20	about 2.5 s	none	D87
W2	`5191c8d7`	90	about 5.3 s	none	D110
W2 cycle 2	`e5dea409`	7	console only	none	D115
W3	`37891824`	197	about 7.8 s	0044; worker 0.8.0 + stamp	D156
W4	`8962f16b`	33	about 1.2 s	none	D246

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Each deploy backed up the replaced files to `/opt/dems-backup-<wave>-<timestamp>/` with a `SHA256SUMS` manifest, plus a `pg_dump` into `/var/backups/dems/` when it migrated. The host had no migrate route, so the migrations went through a reviewed throwaway harness, tried first on a scratch database (D72).

11.8 Results

Each wave was re-judged by channel after its deploy. The public audit report lists the PASS rows; the rows for channels 2/3 and 4/5 come from the decision log (D80, D78).

TABLE 11.5 Re-judge results after each wave

WAVE	RE-JUDGE	RESULT	D
W1	P0-1b (the stale worker)	11 PASS, 0 FAIL	D76
W1	Channels 1 and 7	33 PASS, 1 FAIL (op 66, fixed afterwards in W1)	D81
W1	Channels 2 and 3	11 PASS, 5 FAIL, 11 BLOCKED-HARDWARE	D80
W1	Channels 4 and 5	No new FAIL	D78
W1	Channel 6	9 PASS, 2 FAIL from one defect (fixed afterwards in W1)	D77
W2	Channel 2	28 PASS, 0 FAIL	D113
W2	Security channel	18 PASS, 0 FAIL	D112
W3	Channels 4 and 3	18 PASS, 0 FAIL	D158
W3	Channel 5	0 FAIL; the live package check BLOCKED-HARDWARE	D157

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 11.6 Tablet judging round 1 (90 scored lines, D237)

VERDICT	COUNT
PASS	56
FAIL (6 defects, F1-F6)	10
BLOCKED-HARDWARE	7
BLOCKED-CREDENTIAL	1
BLOCKED (no-save limit)	2
data-absent	14

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed
PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

Cycle 2 fixed all six defects. Round 2 never judged them: the emulator was running a stale JS bundle (D262), and the user then stopped the work (D273). Cycle 2 is proven by jest, tsc and eslint only (UNVERIFIED-ON-DEVICE).

WARNING Mission 1's own residual still stands. The gated D5 end-to-end tests can't run on the bench, so D5 is proven by its parts. The first real 4+4 session is a mandatory acceptance test before operational use (D153; ADR-11).

11.9 Commit chain

Figure 11.1 shows the commits of the audit branch and the four pushes that put them on `main`.

FIGURE 11.1 Commit chain of the audit branch and its publication on `main`

```
audit/fix-20261105
  53513b0c (team state at audit start)
    -> b11950d6  W1
    -> 7bc77bfa  W1 cycle 2
    -> 5191c8d7  W2
    -> e5dea409  W2 cycle 2
    -> 37891824  W3 (D5)
    -> cf59429e  M3 cycle 1
    -> 8962f16b  W4
    -> d6146ce4  M3 cycle 2
    -> f41972f9  Core address setting
    -> e9f48469  README
         |
         v  merged into main
main
  8b829aba  merge of the audit branch (push 1)
  2bcc4c53 / 6a92132c  bench config, audit report (push 2)
  52c845d6 / 5738f7cf  app READMEs (push 3)
  b885d838 / 59a683ed  folder READMEs (push 4)
```

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

11.10 APKs

The APK files are kept outside the repository. Hashes are given so you can check a copy you receive.

TABLE 11.7 Tablet APKs built by the audit

APK	SHA256	STATE
debug, from `cf59429e`	`448b3f3726aab93c15c39b93eb0b45fdd9266a16208 4ff1fcfadada1ce5c09e1`	Installed on the emulator (D224)
release, from `cf59429e`	`1aa42b423ce115aca22c803b0a908e235efff269494 7f1deef34cd6f2b429813`	Built and checked, not installed (D225)
release, from `f41972f9`	`21315b538da7bdcaa1519a951b0eb6e11ae09f680b6 80ed05cdbe6ab87f76258`	For the real-device test. Demo signing key (signer SHA-256 `995c1e389ed7df89f3e79a1690b2c14dc7376d928249bbf01 de3c83272fd1647`), not debuggable, includes the Core address screen (D267)

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Real-device caveats: replacing a debug install needs an uninstall, which removes the pairing; LAN `http`/`ws` from a real device isn't proven yet; tablet builds older than the W1-W3 contracts can't talk to this Core (D90).

11.11 Publication

1. The user asked for a push with README documentation (D268) and chose a public repository for the team audit, to be made private afterwards (D270; ADR-14).
2. README commit `e9f48469` (D271).
3. A scan of the full history gave **CAUTION, no blocker** (D272). No real secret, key, keystore, token, signing material, DB dump or APK was found. The demo accounts named in the old README don't exist on production, and production refuses to boot with a demo password. The caution items:
   - C1: demo credentials in the README and the seed source (not live);
   - C2: an author mailbox address in commit metadata;

- C3: deployment details in older docs;
- C4: about 1.6 GB of history;
- C5: screenshots and recordings not viewed one by one;
- C6: no ignore rule for a root signing folder;
- C7: two committed `.pyc` files.

4. The audit branch was merged into `main` with unrelated histories allowed, keeping the initial commit: merge `8b829aba`, 142 commits on `main` (D272).

TABLE 11.8 Pushes to the public repository

PUSH	RANGE ON `main`	CONTENT	D
1	`b3fe29fd..8b829aba`	Full history of the audit branch	D276
2	`8b829aba..6a92132c`	`deploy/bench-config/` (commit `2bcc4c53`) and `docs/AUDIT_REPORT_2026-11-06.md`	D278
3	`6a92132c..5738f7cf`	READMEs for each app folder (commit `52c845d6`)	D279
4	`5738f7cf..59a683ed`	Folder-level READMEs (commit `b885d838`, 149 files); 156 READMEs under `apps/` afterwards	D283

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

11.12 Deviations and lapses

TABLE 11.9 Deviations and controller lapses

WHAT HAPPENED	D
A draft session was created on production during tablet judging, because the brief demanded a marker reachable only by pressing Next. It was armed, nothing was recorded, and it was cancelled. Its three audit rows stay in the hash chain as the honest trail	D237
A shared-file edit during tablet judging made the bundler hot-reload, which signed the tablet out. A code-freeze rule was added	D237
The bundler served a stale bundle, so the cycle-2 re-judge was void	D262
The controller stopped before entering the production sign-in credential and asked the user to sign in	D225, D226
Scratch files on the host or in the docs during read-only rows; broad process kills that may have ended other rows' runs	D17, D49, D50, D80, D83
W1 deployed in one lockstep deploy, not in the planned order; the migrate route was missing, so a reviewed harness was used	D71, D72
The Core id update was skipped because its safety check could not be proven	D72
The pause of the other writers on the host was decided by the controller under autonomy	D17
Earlier controller readings were corrected	D75, D89, D204, D229
The full end-of-mission gate (D1) never ran	D1, D273
End-of-task cleanup (test sessions, host scratch, emulator pairing) not done	D45, D122, D267

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

CHAPTER 12

12 Known issues and roadmap

This is what is still open at `59a683ed`. Decision numbers are from the audit series unless marked "bench". Known-limit numbers refer to the "Known limits" table of the root `README.md`.

12.1 Open items

TABLE 12.1 Open items and residuals

ITEM	WHAT IS OPEN	SOURCE
R225-F1	A deploy runbook step that lists the tablets paired after the dump, before any DB-restoring rollback. Without it, a `pg_restore` silently unpairs them	D204
R225-F2	A unique Core id per installation. Cores don't have one yet	D204
R225-F3	The tablet compares the stored Core id before it sends its credential (depends on F2). The D263 address binding covers the "another Core" case in part	D204, D263
R263-TLS	Tablet-to-Core traffic is cleartext `http`/`ws` on the LAN. TLS is the proper fix; the user decides	D263
R174-G5	Biometric sign-in is off in release builds. Turning it on needs Keystore-bound authentication proven on a real tablet	D199
R224-1	If the secure write of an owed sign-out fails, the debt is forgotten after a cold start (it is logged)	D211
R224-2	Unpairing doesn't delete a stored owed sign-out; it clears at the next socket that isn't an operator's	D211
U-18	A web view that may keep a recovered session on screen. Not reproduced; parked with a capture plan for the next real failed start	D242
4+4 acceptance recording	The first real session with four cameras and four mics is mandatory before operational use. It needs a fourth mic and every camera answering, and the user's go, because it writes evidence	D153, D228, D229
Mission 2	The end-to-end UI judging panel (web, LCD, adapted tablet) is postponed by the user	D262, D273
DB-window script	The script's smoke check still expects worker 0.7.0; production runs 0.8.0. Fix it before the next window	D244
R-W4-7	A mic lost in the middle of a recording keeps its old "Ready" until the next meter reading or pre-check	D243
R-1	The worker build stamp is checked at boot only, not again at session start	D76
R148-5	With the source-facts file missing and every mic failed, a session seals as manifest `/2` instead of `/2.1`	D132

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

WARNING Two of these bite during routine operations. A rollback that restores the database unpairs every tablet paired after the dump (R225-F1), and the DB-window script will flag the correct worker as wrong until it is updated (D244).

12.2 Blocked items, by kind

TABLE 12.2 Items blocked by something outside the code

KIND	ITEM	D
Credential	Storing the camera network interface needs a Technician account	D80
Credential	The live probe of the LCD setup page needs a vendor sign-in	D89
Credential	Biometric sign-in (R174-G5)	D199
Environment	The gated D5 end-to-end tests can't run on the bench host: the product's trust rules refuse the test setups	D153
Environment	Some camera regression cases need a test-only `libx264` build	D142
Hardware	The PoE replug test (unplug, carrier flip, re-sweep)	row 47
Hardware	The real 4+4 recording: the audio interface has four inputs, but only three mics were plugged	D229
Hardware	One camera didn't answer on the camera network	D228
Device	The cycle-2 tablet re-judge never ran on a fresh bundle	D262, D273

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

12.3 Long-standing product limits worth knowing

The README's Known limits table has 58 rows. These are the ones you are most likely to run into. They are given as the README states them at task 4; later bench runs may have changed some of them in part.

TABLE 12.3 Selected product limits

KNOWN LIMIT	WHAT IT MEANS FOR YOU
1	Recordings and the transcriber's raw audio tap are plaintext on the media store until the seal completes. The owner-only media store is the only protection
2	A recording that can't be sealed ends INCOMPLETE; its files stay in `.quarantine/<sessionId>/` in plaintext, and the product can't bring them back into a seal
7	The audit extract inside a package isn't covered by the package signature
13	Composite video export isn't available: no encoder is selected until the FFmpeg licence decision is taken
37	No key escrow. Losing `<data directory>/credential-store` makes every sealed recording unreadable
46	A production boot from this repository is a bench boot on a seeded database, with the demonstration cases and evidence in the same register and audit chain
47	The signing key is a software key in files of the host, with a self-signed certificate: no hardware key store, no certificate authority, no revocation channel
58	No session with a production camera had been recorded and sealed when the limit was written (task 4). This document doesn't claim more than that; the 4+4 acceptance recording is still open

SUMBER Audit record 2026-11-05/06 (docs/AUDIT_REPORT_2026-11-06.md) and the repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Two open decisions sit next to these limits:

- **FFmpeg licence.** The capture worker links a GPL-3.0-or-later FFmpeg build dynamically. Who may receive a real-capture Core, and whether a composite encoder is selected, waits on this decision.
- **Personal data (SEC35-26).** No legal basis, retention rule, erasure rule or impact assessment exists for personal data in real evidence. No recorded decision of the user accepts this risk.

12.4 Roadmap

The audit's recommended next steps, in order:

1. Make the repository private after the team audit. Share the real environment file only privately, never through git.
2. Test the `f41972f9` release APK on a real tablet: uninstall any debug build, enter the Core address on the Pair screen, pair, then check the F1-F6 screens and the live preview. This replaces the cycle-2 re-judge.
3. Run the 4+4 acceptance recording once a fourth mic is connected and every camera answers. Record, seal, verify through op 41, probe each audio file, and measure the CPU with previews open.
4. Run the D1 full gate once: verifier, security review, test expert with the full DB suite, senior code review.
5. Resume Mission 2 if wanted.
6. Decide the open items in 12.1, starting with R263-TLS, R225-F1/F2 and the publication cautions C1-C7.
7. Clean up the test artifacts, on the user's word.

HEADS-UP Before step 4 (or any other DB window), update the DB-window script's worker check to `capture-worker/0.8.0` (D244). Otherwise the smoke check fails on a healthy Core.

12.5 Later hygiene

None of these change behaviour. The retired `session.audioSource.*` translation keys can be removed now that Mission 3 is done. Some helpers are read only by tests, the "Try again" prop is optional, and simulator leftovers remain for cameras 3-4. On the tablet, the image library logs every MJPEG preview frame at DEBUG level, so debug logs grow fast (D138, D214, D239, D247, D253, D256).

CHAPTER 13

13 Appendix

Reference tables for ports, environment variables, units, scripts and codes, plus a glossary. Every value comes from the repository at `59a683ed`; the source column or the lead-in tells you where to look. Section numbers in the tables point to the chapter that explains the item; "ADR-n" points to record n in 10.3.

13.1 Ports

Every DEMS listener binds `127.0.0.1`. Nothing below should be exposed on another interface.

TABLE 13.1 Ports

PORT	SERVICE	BIND	SOURCE
4420	Core server, main instance (`/api/v1`, `/ws`, `/media`, `/exports`, `/preview`)	127.0.0.1	`apps/core-server/README.md`
4421-4423	Core lane instances (dev)	127.0.0.1	`README.md`, Ports
4432	PostgreSQL 18, TCP only, no Unix socket	127.0.0.1	`README.md`, Ports
5173	Web console (Vite)	127.0.0.1	`apps/core-web/README.md`
5176-5178	Vite lanes (dev), forwarding to 4421-4423	127.0.0.1	`README.md`, Ports
4173	Vite preview of a build (dev)	127.0.0.1	`apps/core-web/README.md`
8090	LCD backend (no authentication)	127.0.0.1	`apps/lcd-backend/README.md`
8554-8564	Capture ingest, RTSP over TCP; the nft rule admits only user `dems-core`	127.0.0.1	`apps/capture-worker/README.md`
4400	Mock Core (dev and demo only)	127.0.0.1	`apps/mock-core/README.md`
8081-8089	Metro bundler and its lanes (dev)	127.0.0.1	`README.md`
4410-4419	Packaged web demo launcher of the first demo	127.0.0.1	`README.md`
5432	Refused by the Core (`FOREIGN_DATABASE_PORT`)	n/a	`deploy/bench-config/README.md`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

WARNING The LCD backend's API has no authentication, and its `rec=start` drives real relay contacts on the panel. Never publish port 8090.

13.2 Environment variables

The production template is `deploy/bench-config/core.env.example`. It lists every key with its allowed values. Never commit a filled-in copy.

TABLE 13.2 Core server (`/etc/dems/core.env`, read once at start)

NAME	VALUES / DEFAULT	SECRET	READ BY
`DEMS_CORE_PORT`	`4420`; `4421-4423` for lanes; anything else stops the boot	no	Core
`DEMS_DEMO_CONTROLS`	`1` on; unset, empty or `0` off. Production: `0`	no	Core
`DEMS_LOG_LEVEL`	`fatal`, `error`, `warn`, `info` (default), `debug`, `trace`, `silent`	no	Core
`DEMS_LCD_BACKEND_URL`	`http://127.0.0.1:<port>` only; unset disables the bridge	no	Core
`DEMS_CAPTURE_MODE`	`simulated` (default, no worker) or `real`	no	Core
`DEMS_CAPTURE_WORKER`	absolute path of `capture-worker`; unset = no worker configured	no	Core
`DEMS_MEDIA_DIR`	absolute path of the media root; default `<data dir>/media/sessions`	no	Core, LCD backend
`DEMS_CAPTURE_DEV_SOURCES`	`1` = development sources exist; production `0`	no	Core
`DEMS_CAPTURE_HELPER_FFMPEG`	absolute path of the helper program (ffmpeg) for development sources	no	Core
`DEMS_CAPTURE_DEV_FAKE_MIC`	`tone` (default) or `speech`; development fence only	no	Core
`DEMS_CAPTURE_RELAY`	absolute path of `capture-relay`	no	Core
`DEMS_CAPTURE_GUARD`	absolute path of `capture-guard`	no	Core
`DEMS_CAPTURE_ALIAS_SOCKET`	absolute path of the address helper's socket; unset = helper not configured	no	Core
`DEMS_CAPTURE_AUDIO_HELPER`	absolute path of the audio helper; unset = no production mic can open	no	Core
`DEMS_INSTALLATION_DEVICE_ID`	a capital letter, then capitals, digits or hyphens, 32 characters at most	no	Core, factory seed
`DEMS_INSTALLATION_NAME`	1-120 printable characters, no leading or trailing space	no	Core, factory seed
`DEMS_CAMERA_AUTOATTACH`	`1` turns the smart-attach sweep on; default off	no	Core
`DEMS_CAMERA_SEGMENT_CIDR`	one IPv4 `/24`, such as `192.168.x.0/24`; required while the sweep is on	no	Core
`DEMS_CAMERA_DEFAULT_USER`	user name the cameras already have; used only to test and store an auto-adopted camera	no	Core
`DEMS_CAMERA_DEFAULT_PASSWORD`	1-256 printable characters; logged as `[redacted]`	**yes**	Core
`DEMS_DEVICE_CERT_FINGERPRINT`	device certificate fingerprint; read at seed time only	no	`db:seed -- factory`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

Not in the environment: the database connection (next table), the signing and session keys (the Core's key store), and a session secret (none exists; the cookie is named `dems_session_<port>`). `NODE_OPTIONS` is only reported in the boot log. Any `PG*` variable is removed at start.

TABLE 13.3 Connection files in `<repository root>/.data` (mode 0600)

FILE	KEYS	READ BY

FILE	KEYS	READ BY
pg.env	DEMS_PG_HOST (must be 127.0.0.1), DEMS_PG_PORT (4432), DEMS_PG_DATABASE, DEMS_PG_APP_ROLE, DEMS_PG_APP_PASSWORD (**secret**)	Core; refuses the file if any group or other bit is set
pg.owner.env	the same host, port and database keys, plus DEMS_PG_OWNER_ROLE, DEMS_PG_OWNER_PASSWORD (**secret**)	db:* commands

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.4 LCD backend (apps/lcd-backend/app/dems_backend.py; /etc/dems/core.env, then the unit's own Environment= lines, which win)

GROUP	NAME	DEFAULT
Network	DEMS_BIND, DEMS_PORT	127.0.0.1, 8090
Serial	DEMS_DEV	/dev/ttyACM0; the bench unit sets /dev/serial/by-id/<your-adapter>
Serial	DEMS_USB_SERIAL, DEMS_USB_VIDPID	empty; set DEMS_USB_SERIAL=<serial> of your adapter
Serial	DEMS_BAUD, DEMS_DTR, DEMS_RTS, DEMS_USB_RETRY_S	115200, 1, 1, 2
Disk	DEMS_REKAM_DIR (falls back to DEMS_MEDIA_DIR), DEMS_MIN_FREE_GB, DEMS_CRIT_FREE_GB	-, 20, 5
Channels	DEMS_CH_NAMES, DEMS_CH_MODE	CH1,CH2,CH3,CH4, track (bench: device)
Channels	DEMS_LEASE_S, DEMS_CH_STALE_S, DEMS_PANEL_SILENT_S, DEMS_PUSH_S	0 (bench: 6), 0, 10, 1.0
Identity	DEMS_SATKER, DEMS_CLIENT, DEMS_IDENTITY_FILE	empty
Identity	DEMS_PROVISION_KEY (**secret**; guards writing DEMS_CLIENT, header X-Provision-Key)	empty; generate with openssl rand -hex 32
Logo	DEMS_LOGO_DIR, DEMS_LOGO_* (CHUNK, ACK_S, GAP_S, MAX_TRIES, REPUSH_MIN_S, WAIT_S, AUTOPUSH, MAX_BYTE, MAX_W, MAX_H)	directory of the identity file, else /var/lib/dems-lcd; see the template
Setup page	DEMS_CORE_URL, DEMS_CORE_CONSOLE_ORIGIN, DEMS_SETUP_TTL_S	http://127.0.0.1:4420, http://127.0.0.1:5173, 900
Docker only	DEMS_LCD_RECORDINGS_DIR, DEMS_LCD_SERIAL_DEVICE, DEMS_LCD_DIALOUT_GID, DEMS_LCD_UID, DEMS_LCD_GID	none; see apps/lcd-backend/README.md

SUMBER apps/lcd-backend/app/dems_backend.py; /etc/dems/core.env, then the unit's own Environment= lines, which win; DEMS repository, commit 59a683ed
PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.5 Console, tablet app, mock Core and tests

NAME	WHERE	PURPOSE
DEMS_CORE_PROXY_PORT	console, vite.config.ts	4420-4423 turns the proxy to the Core on; any other value stops the dev server
VITE_DEMS_CORE_MODE	console	live or simulated: default data mode when nothing is stored
VITE_DEMS_PRODUCTION_LOCK	console	exactly 1: production lock (drop-in production-lock.conf)
VITE_DEMS_CORE_ORIGIN	console	same-origin, for the packaged build only; unset on the bench
EXPO_PUBLIC_USE_RN_FETCH	tablet, dev only	read by the debug probes

NAME	WHERE	PURPOSE
METRO_PORT	tablet, dev only	Metro port of the start:lane script
JAVA_HOME, ANDROID_HOME	tablet build	read by scripts/build-release-apk.sh
LOG_LEVEL	mock Core	logger level, default info
DEMS_SEAL_E2E, DEMS_DEV_SESSION_E2E	tests	set by test:seal-e2e and test:dev-session-e2e
DEMS_REGRESSION_SKIP_WORKER_E2E	tests	1 skips the worker e2e series in test:regression
DEMS_FFMPEG	tests	path of the ffmpeg that drives the generated test camera

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

The capture worker, relay and guard read no environment variables. They take arguments and the start message from the Core.

13.3 systemd units

All files are in deploy/bench-config/systemd/ and install to /etc/systemd/system/. Chapter 8 (8.1, 8.3.4) shows the unit files and the install commands.

TABLE 13.6 Bench host units

UNIT	TYPE	USER / GROUPS	PORT	ORDERING AND CONFIG
dems-loopback-rule.service	oneshot, stays active	root	-	Before=dems-core.service; loads /etc/dems/loopback-ingest.nft; on stop deletes table inet dems_loopback
dems-core.service	simple	dems-core + audio, dems-netalias	4420	Requires=dems-loopback-rule.service; after network-online, time-sync, the loopback rule and postgresql@18-main; EnvironmentFile=/etc/dems/core.env; runs tsx src/index.ts with Node 26.12.1
postgresql@18-main	PostgreSQL 18 server unit	-	4432	not in this folder; installed with the PostgreSQL server
dems-console.service	simple	dems-console	5173	after network-online and dems-core; DEMS_CORE_PROXY_PORT=4420 VITE_DEMS_CORE_MODE=live HOME=/nonexistent; drop-in production-lock.conf
dems-lcd.service	simple, Restart=always	dems-core + dialout	8090	after local-fs; same core.env, then its own Environment= lines
dems-kiosk-agent.service	simple, Restart=always	root	-	root side of the kiosk physical-key lock; its program is not in the repository
dems-wifi-watchdog.service + .timer	oneshot, timer	root	-	every minute, first run 2 min after boot; reloads the Wi-Fi driver or reboots when the gateway is lost; program not in the repository

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

dems-core and dems-console both set UMask=0077, LimitCORE=0, NoNewPrivileges=yes and empty capability sets. dems-lcd sets UMask=0077 and NoNewPrivileges=yes (Table 9.5). The nft rule (8.1.4):

```
table inet dems_loopback
delete table inet dems_loopback
table inet dems_loopback {
    chain output {
        type filter hook output priority filter; policy accept;
        oifname "lo" ip daddr 127.0.0.1 tcp dport 8554-8564 meta skuid != "dems-core" counter reject with tcp
reset
    }
}
```

13.4 npm scripts

Run root scripts from the repository root. Root `package.json`:

TABLE 13.7 Root scripts

GROUP	SCRIPTS
Dev servers	`dev:server`, `start:server`, `dev:web`, `dev:mock`, `dev:mobile`, `android`, `emulator`, `emulator:phone`, `mobile:reverse`, `probe:ws`
Build	`build` (shared, mock-core, console), `build:android`, `theme:gen`; `postinstall` applies the mobile library patch
Quality	`typecheck`, `lint`, `test` (all workspaces)
Database	`db:create`, `db:migrate`, `db:seed`, `db:reset`, `db:status`, `test:db`
Project PostgreSQL	`pg:init`, `pg:start`, `pg:stop`, `pg:status`, `pg:remove-data`, `pg:remove`, `test:pg-scripts`
Heavy tests	`test:seal-e2e`, `test:dev-session-e2e`, `test:kill-matrix`, `test:capture-worker`, `test:capture-worker-e2e`, `test:regression`, `test:host-helper`
Capture and evidence	`capture:dev-setup`, `evidence:verify-export`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.8 Core server host commands (`npm run <script> -w @dems/core-server`)

SCRIPT	WHAT IT DOES
`dev`, `start`	`tsx watch src/index.ts`, `tsx src/index.ts`
`test`, `test:db`, `coverage`	Vitest unit project, DB project, coverage
`db:create`, `db:migrate`, `db:seed`, `db:reset`, `db:status`, `db:generate`	Database lifecycle; `db:generate` runs drizzle-kit
`evidence:production-key`, `evidence:rotate-key`, `evidence:mark-compromised`	Production signing key: create, rotate, mark compromised
`evidence:dev-key`	Development signing key
`evidence:verify-export`	Verify an export archive without a Core
`operator:set-password`	Set an operator's password from the host; prompts, never takes it as an argument
`start:camera-stub`, `test:db:drop-run`	Test helpers

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

HEADS-UP `test:seal-e2e`, `test:dev-session-e2e`, the worker e2e, the DB step of `test:regression` and the documented migrate lock call `/usr/bin/lockf`, a macOS tool (7.3.1). On Linux, make sure no other migration or heavy run is going at the same time.

13.5 Error and log code index

TABLE 13.9 Codes and where they are explained

CODE	KIND	SECTION
`B-1` .. `B-9`, log `the server stopped: boot`	DB boot checks	3.7
`FOREIGN_DATABASE_PORT`	boot refusal (port 5432)	3.8
`REAL_CAPTURE_WITH_DEMO_CONTROLS`, `DEV_SOURCES_NEED_REAL_CAPTURE`, `DEV_FLAG_WITH_LAN_LISTENER`, `DEV_SOURCE_WITHOUT_FLAG`	capture fence refusals	3.7
`CAPTURE_WORKER_NOT_CONFIGURED`, `CAPTURE_WORKER_PROGRAM_UNUSABLE`, `CAPTURE_EVIDENCE_STORE_UNUSABLE`, `CAPTURE_SIGNING_KEY_UNUSABLE`	capture boot refusals	3.7
`CAPTURE_WORKER_NO_HANDSHAKE`, `CAPTURE_WORKER_PROTOCOL_MISMATCH`, `CAPTURE_WORKER_VERSION_MISMATCH`, `CAPTURE_WORKER_VERSION_UNKNOWN`, `CAPTURE_WORKER_PROOF_BUILD`, `CAPTURE_WORKER_MEDIA_LIBRARY_MISMATCH`	worker boot refusals	3.7, 6.1
`CAPTURE_WORKER_BUILD_STAMP_STALE`	boot check 5b	3.7, 6.1, ADR-6
`CAPTURE_DEVELOPMENT_SIGNING_KEY_IN_PRODUCTION`, `CAPTURE_SIGNING_KEY_INSTALLATION_MISMATCH` (4b-2), `CAPTURE_INSTALLATION_ID_RESERVED_FOR_TESTS` (4b-3)	production key checks	3.7
`CAPTURE_DEMO_OPERATOR_PASSWORD_IN_PRODUCTION` (reason `DEMO_PASSWORD_IN_PRODUCTION`)	check 4c	3.7, 8.3.2
`CAPTURE_HOST_PLATFORM_NOT_SUPPORTED`, `CAPTURE_HOST_PROFILE_NOT_ASKED`	platform refusal / macOS log line	3.7
`CAPTURE_HOST_ALIAS_HELPER_ABSENT`	boot warning; start continues	3.7
`TRANSCRIBER_CONFIG_UNRESOLVABLE`	boot refusal	3.7, 3.12, 6.3
`CAPTURE_BOOT_CHECKS_PASSED`	the one healthy-start log line	3.7, 8.8
`RESET_BLOCKED_REAL_EVIDENCE`, `RESET_STORE_ROOT_CONFLICT`, `RESET_STORE_ROOT_UNKNOWN`, `RESET_TARGET_IN_USE`	DB reset refusals	3.8
`ORIGIN_NOT_ALLOWED` (403), `ACCESS_DENIED`, `INTERNAL_ERROR` (500)	request pipeline	3.3
`CAMERA_PROFILE_NOT_STATED`, `STREAM_INVALID`, `PLAN_INVALID`	camera assignment, relay, worker plan	3.5, 6.1, ADR-6
`CAMERA_LINK_DOWN`, `CAMERA_LINK_UP`	audit rows of the camera link watch	3.5
`CAPTURE_PAIRED_MIC_MISSING` (warn log), `PAIRED_MIC_MISSING` (notice)	D5 pair without its mic	3.6, 5.4
`409 AUDIO_SOURCE_FIXED_BY_PAIRING`	op 81, retired	3.4, 3.6, ADR-1
`SEAL_BLOCKED`	critical alert; session stays FINALIZING	3.9
`AUDIT_EXTRACT_NOT_SIGNED`, `HOST_PROFILE_NOT_ASKED`, `SIGNED_BY_COMPROMISED_KEY`, `REVISION_APPEND_PENDING`, `EXPORT_FOLDER_UNEXPECTED_FILE`, `AUDIO_ABSENT_ON_RESTART`, `VIDEO_ABSENT_IN_SESSION`, `AUDIO_TRACKS_MISSING`	verify notices	3.9, ADR-2

CODE	KIND	SECTION
`ACTIVE_KEY_CHANGED`	Core not restarted after a key rotation	3.9
`COMPOSITE_ENCODER_NOT_SELECTED`	composite export unavailable	3.9
`CPU_LOCAL_RUNTIME_MISSING`	transcriber host file missing	6.3
`FILE_OWNER`, `DIRECTORY_WRITABLE`	program trust rule refusals	ADR-11
Close `4403`, `4401`	tablet socket: device revoked, session ended	3.10, 5.2
`REKAM` ("recording"), `WASPADA` ("alert"), `BACKEND DIAM` ("backend silent")	LCD panel words	6.2

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

13.6 Glossary

TABLE 13.10 Glossary, hosts and modes

TERM	MEANING
Core	The Core server, `apps/core-server`. It holds the recording truth; the console, the tablet and the LCD panel display it
Core host	The machine that runs the Core, its database and its capture programs in the interview room
Bench host	The team's single Linux machine that runs DEMS from `/opt/dems/core`. It is also the production install
Lane	An extra development instance: Core 4421-4423, database `dems_lane_1..3`, Vite 5176-5178, run by `scripts/lane/lane.sh`
Data mode	Where a client gets its data: SIMULATED (a simulator inside the client) or LIVE (the Core)
Capture operation	How the Core records: `SIMULATED` (default), `REAL_DEVELOPMENT` (real capture with development sources) or `REAL_PRODUCTION`. Decided once, at start
Demo controls	Demonstration controls, `DEMS_DEMO_CONTROLS=1`; allowed only with SIMULATED capture
Development fence	The rule that keeps development sources and production sources apart; one session never mixes them

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.11 Glossary, capture

TERM	MEANING
Capture worker	The C program, linked against FFmpeg 9.0.2, that records up to four cameras with their mics. Child of the Core; protocol 6
Relay	`capture-relay`, one per camera. Pulls the camera over RTSP and pushes it to the ingest; never listens
Guard	`capture-guard`. Starts one child (the audio helper) and kills it when its input ends or its parent changes
Audio helper	The program that delivers mic audio (PCM) to the worker, started through the guard
Address helper (netalias)	The Linux host helper behind `DEMS_CAPTURE_ALIAS_SOCKET` (`scripts/host/netalias`). Absent: a boot warning. Present but wrong: a refused start
Ingest	The worker's RTSP listeners on `127.0.0.1:8554-8564`

TERM	MEANING
fMP4	Fragmented MP4, the camera file format
Fragment	One durably written piece of a recording file, stored as a `recording_fragments` row
Checkpoint	The worker's `RECORD` frame that reports a written fragment to the Core
Durable fragment	The first fragment on disk. A session shows "recording" only after it

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.12 Glossary, evidence

TERM	MEANING
Seal	Turning a stopped recording into a signed evidence package
Sealing pipeline	The six steps `CLOSE_TRACKS`, `FINALIZE_HASHES`, `FINALIZE_TRANSCRIPT`, `BUILD_MANIFEST`, `SIGN_MANIFEST`, `STORE_SEALED_PACKAGE`
Evidence package	The sealed folder `SESSION-<sessionId>/`: encrypted fragments, manifest, signature, certificate
Manifest	`session-manifest.json`. Schema `/2` before D5; `/2.1` adds the per-pair audio tracks and the signed missing-mic statement
Signer store	`<data dir>/signer`, the signing key. Its risk is a copy
Credential store	`<data dir>/credential-store`, the session keys. Its risk is a loss; there is no escrow
Session key	The key that encrypts one session's fragments (AES-256-GCM)
Quarantine	`.quarantine/<sessionId>/`, where unsealable or unaccounted files are moved. Never deleted
Verify	The Core's check of a sealed package (op 41)
Export	A decrypted, signed archive `EXP-<id>.zip` for handing over
Receiver	Whoever gets an export; checks it without a Core using `evidence:verify-export`

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.13 Glossary, devices and state

TERM	MEANING
Pairing	Binding a tablet to a Core with a one-time code issued in the console; the device appears as `DEV-000N`
Device credential	The secret a paired tablet keeps in its secure store, bound to the Core address
Owed sign-out	A sign-out the tablet could not deliver; it is kept and sent when the link returns
Reach	Whether the Core last found a device reachable: `REACHABLE`, `NOT_REACHABLE`, or "Not checked"
"Not reported"	What a `REAL_PRODUCTION` Core shows for anything that has no real data source
Pre-check	The 12-item check run before a session starts, including cameras 1-4 and mics 1-4

SUMBER DEMS repository, commit 59a683ed **PROVENANS** TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.14 Glossary, pairs and notices

TERM	MEANING
D5 pair	Camera N with Mic N, fixed in the capture setup (audit D5)
`VIDEO-0N` / `MIC-0N` / `AUDIO-0N`	Camera track N / mic channel N / the per-pair audio file `MIC-0N.m4a` as track N

TERM	MEANING
PAIRED_MIC_MISSING	Notice: a pair records without its mic, with a silent audio track

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.15 Glossary, operations

TERM	MEANING
Production lock	VITE_DEMS_PRODUCTION_LOCK=1: the console is always LIVE
DB test window	An announced stop of production to run DB tests on the bench (dump, stop, test, restore, smoke)
Lockstep deploy	Worker and Core files deployed, and rolled back, together
Build stamp	capture-worker.stamp: worker version, protocol, source and binary hashes; checked at boot
Smart attach	DEMS_CAMERA_AUTOATTACH=1: the Core sweeps the camera /24 every 60 s and on link-up, adopts ONVIF cameras with the default camera credential, and assigns them to the first empty track
Commissioning	Technician-only setup of capture hardware on the Core host (/admin/commissioning, permission CAPTURE_COMMISSION)
Installation id	DEMS_INSTALLATION_DEVICE_ID, the identity a signed manifest states; ids reserved for tests are refused in production
REKAM / WASPADA	LCD panel words: "recording" and "alert"

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual

TABLE 13.16 Glossary, audit vocabulary

TERM	MEANING
M1 / M2 / M3	Audit missions: audit and fix; end-to-end UI judging (postponed); tablet adaptation
W1-W4	Fix waves of the audit
Residual (R-codes)	A known, accepted gap carried in the reports, such as R263-TLS or R225-F1
Dnnn	A decision number. Bench series (the team's log) and audit series (D1-D283) reuse the same numbers; write "bench" or "audit"

SUMBER DEMS repository, commit 59a683ed PROVENANS TERUKUR, read from the sources named; nothing was measured for this manual